

EtherCAT 规范

第六部分：应用层协议规范

ETG.1000.6 S (R) V1.0.2

ETG 号	ETG.1000.6
类型	S (规范)
状态	R (颁布)
版本	V1.0.2
中文版本号	C01

作者	ETG
联系方式	info@ethercat.org
英文文件名	ETG1000_6_S_R_V1i0i2_EcatALProtocols.doc
日期	6.24.2013
中文文件名	ETG1000_6_CHN_EcatALProtocols_C01.doc

版权

Copyright © EtherCAT Technology Group. All Rights Reserved.

禁止在没有得到授权的情况下复制, 传阅和使用这份文档, 以及对其内容的交流。违者有责任支付其赔偿金。在授予专利、实用新型或外观设计的事件中, 所有的权力受到保留。并且该文档服从于技术变化。

商标

EtherCAT® 以及 Safety over EtherCAT®是由德国的Beckhoff Automation GmbH许可的已注册商标和专利技术。

原文修改履历

版本号	注释
1.0	引用于只供 ETG 使用的 IEC-规范 61158-6 Type 12 部分
1.0.1	修正问题与澄清, 请查看 ETG1000_ES_D_V0i6_EcatSpecErrata.pdf。该文件没有修改点。
1.0.2	修正问题与澄清, 请查看 ETG1000_V1i0i2_ES_R_V0i7_EcatSpecErrata.doc 与 IEC SC65C MT 9 编辑注释。该文件没有修改点。

中文版修改履历

版本号	注释
C01	初版

目录

1	范围	9
1.1	本标准的范围以及所对应的 IEC 标准	9
1.2	概述	9
1.3	规范	10
1.4	一致性	10
2	规范性引用文件	10
3	术语, 定义, 符号, 缩略语和约定	11
3.1	参考模型术语和定义	11
3.2	服务协议术语和定义	11
3.3	应用层定义	12
3.4	通用符号和缩写	17
3.5	附加的符号和缩略语	17
3.6	约定	18
4	应用层协议规范	23
4.1	操作原理	23
4.2	节点参考模型	23
5	FAL 语法说明	25
5.1	编码原则	25
5.2	数据类型和编码规则	25
5.3	AR 编码	28
5.4	SII 编码	33
5.5	等时同步 PDI 编码	38
5.6	CoE 编码	41
5.7	EoE 编码	79
5.8	FoE 编码	88
6	FAL 协议状态机	94
6.1	总体结构	94
6.2	AP 上下关系状态机(AP-Context state machine)	96
6.3	FAL 服务协议机(FAL service protocol machine, FSPM)	96
6.4	应用关系协议机(Application Relationship Protocol Machines, ARPMs)	96
6.5	DLL 映射协议机(DLL mapping protocol machine, DMPM)	141
附录		143
	EtherCAT Technology Group (ETG)	143
图 1	特定字段的公共结构	19
图 2	类型描述实例	20
图 3	从站节点参考模型	24
图 4	Time Of Day 的编码	25
图 5	Time Difference 的编码	26
图 6	AL 控制请求结构	28
图 7	AL 控制响应结构	29

图 8 – AL 状态变化结构	31
图 9 – PDI 控制类型描述	32
图 10 – 同步配置类型描述	32
图 11 – 分布式时钟同步和锁存类型描述.....	39
图 12 – CoE 一般结构	41
图 13 – SDO 下载急送请求结构	42
图 14 – SDO 下载急送响应结构	43
图 15 – SDO 常规下载请求结构	44
图 16 – 下载 SDO 段请求结构.....	46
图 17 – 下载 SDO 段响应结构.....	47
图 18 – SDO 上传急送请求结构	47
图 19 – SDO 上传急送响应结构	48
图 20 – SDO 常规上传响应结构	50
图 21 – 上传 SDO 段请求结构.....	51
图 22 – 上传 SDO 段响应结构.....	51
图 23 – 中止 SDO 传输请求结构.....	52
图 24 – SDO 信息服务结构	55
图 25 – Get OD 列表请求结构.....	56
图 26 – 获取 OD 列表响应结构.....	56
图 27 – 获取对象描述请求结构.....	58
图 28 – 获取对象描述响应结构.....	58
图 29 – 获取条目描述请求的结构	59
图 30 – 获取条目描述响应结构.....	60
图 31 – SDO 信息错误请求结构	62
图 32 – EoE 一般结构.....	79
图 33 – EoE 时间戳结构	80
图 34 – EoE Fragment 请求结构.....	81
图 35 – Set IP Parameter 请求结构.....	83
图 36 – Set IP Parameter 响应结构.....	85
图 37 – Set Address Filter 请求结构	86
图 38 – Set Address Filter 响应结构	88
图 39 – FoE Read 请求结构.....	89
图 40 – FoE Write 请求结构	89
图 41 – FoE Data 请求结构	90
图 42 – Ack 请求结构	91
图 43 – Error 请求结构	92
图 44 – Busy 请求结构	94
图 45 – 协议机间的关系.....	95
图 46 – AR 协议机.....	96

图 47 – ESM 图.....	98
表 1 – PDU 元素的描述示例.....	20
表 2 – 属性描述实例.....	21
表 3 – 状态机描述元素.....	21
表 4 – 状态机元素的说明.....	22
表 5 – 状态机中使用的约定.....	22
表 6 – 位序列的传输语法.....	26
表 7 – Unsignedn 数据类型的传输语法.....	27
表 8 – Integern 数据类型的传输语法.....	28
表 9 – AL 控制描述.....	29
表 10 – AL 控制响应.....	29
表 11 – AL 状态编码.....	30
表 12 – AL 状态改变.....	31
表 13 – PDI 控制.....	32
表 14 – PDI 配置.....	32
表 15 – 同步配置.....	33
表 16 – 从站信息接口区域.....	33
表 17 – 从站信息接口类别.....	34
表 18 – 支持的邮箱协议类型.....	34
表 19 – 种类类型.....	35
表 20 – String 类别的结构.....	35
表 21 – General 类别的结构.....	36
表 22 – FMMU 类别的结构.....	37
表 23 – 每个元素的 SyncM 类别的结构.....	37
表 24 – 每个 PDO 的 TXPDO 和 RXPDO 的结构.....	38
表 25 – PDO Entry 的结构.....	38
表 26 – 分布式时钟同步参数.....	40
表 27 – 分布式时钟锁存数据.....	41
表 28 – CoE 元素.....	42
表 29 – SDO 下载急送请求.....	43
表 30 – SDO 下载急送响应.....	44
表 31 – SDO 常规下载请求.....	45
表 32 – 下载 SDO 段请求.....	46
表 33 – 下载 SDO 段响应.....	47
表 34 – SDO 上传急送请求.....	48
表 35 – SDO 上传加速响应.....	49
表 36 – SDO 常规上传响应.....	50
表 37 – 上传 SDO 段请求.....	51

表 38 – 上传 SDO 段响应	52
表 39 – 中止 SDO 传输请求	53
表 40 – SDO 中止代码	54
表 41 – SDO 信息服务	55
表 42 – 获取 OD 列表请求	56
表 43 – 获取 OD 列表响应	57
表 44 – 获取对象描述请求	58
表 45 – 获取对象描述响应	59
表 46 – 获取条目描述请求	60
表 47 – 获取条目描述响应	61
表 48 – SDO 信息错误请求	63
表 49 – 紧急请求	64
表 50 – 紧急错误代码	65
表 51 – 错误代码	65
表 52 – 诊断数据	66
表 53 – 同步管理器长度错误	66
表 54 – 同步管理器地址错误	66
表 55 – 同步管理器设置错误	66
表 56 – 通过邮箱进行 RxPDO 传输	67
表 57 – 通过邮箱进行 TxPDO 传输	67
表 58 – RxPDO 远程传输请求	68
表 59 – TxPDO 远程传输请求	68
表 60 – 命令对象结构	69
表 61 – 对象字典结构	69
表 62 – 对象代码定义	69
表 63 – 基本数据类型区	70
表 64 – 扩展数据类型区	71
表 65 – 枚举定义	72
表 66 – CoE 通信区	72
表 67 – Device Type	73
表 68 – Error Register	74
表 69 – Manufacturer Device Name	74
表 70 – Manufacturer Hardware Version	75
表 71 – Manufacturer Software Version	75
表 72 – Identity Object	75
表 73 – Receive PDO Mapping	76
表 74 – Transmit PDO Mapping	76
表 75 – Sync Manager Communication Type	77
表 76 – Sync Manager Channel 0~31	78

表 77 – Sync Manager Synchronization	79
表 78 – Initiate EoE 请求	80
表 79 – Initiate EoE 响应	81
表 80 – EoE Fragment 请求	82
表 81 – EoE 数据	83
表 82 – Set IP Parameter 请求	84
表 83 – Set IP Parameter 响应	85
表 84 – EoE Result 参数	86
表 85 – Set Address Filter 请求	87
表 86 – Set Address Filter 响应	88
表 87 – FoE Read 请求	89
表 88 – FoE Write 请求	90
表 89 – FoE Data 请求	91
表 90 – Ack 请求	92
表 91 – Error 请求	93
表 92 – Error 代码	93
表 93 – Busy 请求	94
表 94 – 状态转换与本地管理服务	98
表 95 – ESM 发出到 DL 的原语	99
表 96 – DL 发出到 ESM 的原语	99
表 97 – 应用发送到 ESM 的原语	100
表 98 – ESM 到应用的原语	100
表 99 – ESM 变量	100
表 100 – ESM 宏	101
表 101 – ESM 函数	102
表 102 – ESM 状态表	103
表 103 – ESM 函数	115
表 104 – 邮箱处理程序向 DL 发出的原语	116
表 105 – DL 向邮箱处理程序发出的原语	116
表 106 – 协议处理程序向邮箱处理程序发出的原语	116
表 107 – 邮箱处理程序向协议处理程序发出的原语	116
表 108 – 应用向 CoESM 发出的原语	118
表 109 – CoESM 向应用发出的原语	119
表 110 – CoESM 状态表	119
表 111 – 应用向 EoESM 发出的原语	130
表 112 – EoESM 向应用发出的原语	131
表 113 – EoESM 状态表	132
表 114 – 应用向 FoESM 发出的原语	136
表 115 – FoESM 向应用发出的原语	137

表 116 – FoESM 状态表	138
-------------------------	-----

1 范围

1.1 本标准的范围以及所对应的 IEC 标准

ETG.1000 系列文件是在 EtherCAT Technology group 范围内对 EtherCAT Technology 详细说明。它分为以下几个部分:

- ETG.1000.2: 物理层服务定义和协议规范
- ETG.1000.3: 数据链路层服务定义
- ETG.1000.4: 数据链路层协议规范
- ETG.1000.5: 应用层服务定义
- ETG.1000.6: 应用层协议规范

以上各文件依赖于 IEC 61158 系列文件 Type12 中相一致的部分。为了回避商标使用权的问题, EtherCAT 在 IEC 61158 中被称作 Type12。

1.2 概述

现场总线应用层 (FAL) 为用户程序提供一种访问现场总线通信环境的方法, FAL 被当作相应的应用程序之间的“窗口”。

针对 EtherCAT 现场总线的自动化环境和材料, 本部分为应用程序之间基本时间关键及非时间关键信息的交流提供公共要素。“时间关键”一词代表存在一个时窗, 在此时窗内, 要求完成一个或多个有明确定义的指定动作。在时窗内没有完成指定的动作, 有可能造成需要该动作的应用的失败, 甚至会影响设备, 厂房及人身安全。

本部分以一种抽象的方式定义了由不同类型的现场总线应用层提供的外部可视的服务, 它借助于:

- a) 定义在通信应用实体间传送的应用层协议数据单元的抽象语法,
- b) 定义在通信应用实体间传送的应用层协议数据单元的传输语法,
- c) 定义通信应用实体间可视的应用服务行为的应用上下关系状态机,
- d) 定义通信应用实体间可视的通讯行为的应用关系状态机。

本部分的目的是定义提供的协议以定义下列内容

- 1) 定义 ETG.1000.5 中定义的服务原语的 wire-representation,
- 2) 定义与传输相关的外部可视动作。

本部分规定 IEC 总线应用层的协议, 并与 OSI 基本参考模型 (ISO/IEC 7498) 及 OSI 应用层结构 (ISO/IEC 9545) 相一致。

包含在应用过程中的 FAL 应用实体 (AE) 提供 FAL 服务和协议。FAL AE 由一组面向对象的应用服务单元 (ASE) 和一个管理 AE 的层管理实体 (LME) 组成。ASE 提供操作一组应用过程对象 (APO) 类的通信服务。在 FAL ASE 中有一个管理 ASE, 它能提供一组用于 FAL 类实例管理的通用服务。

从应用的角度, 尽管这些服务定义了请求和响应怎样被发布和传送, 但它们都不包括关于正请求和响应中的应用发布和传送内容的规范。也就是, 应用的行为方面没有被定义。只有对它们可以发送/接收什么样的请求和响应进行了明确定义。这使得 FAL 用户在标准化这种对象行为时更具灵活性。除了这些服务之外, 在该部分中也定义了一些支持服务, 以提供对控制操作的某些方面的 FAL 的访问。

1.3 规范

本部分的主要目的是定义应用层协议的语法和行为, 其用于传送在 ETG.1000.5 中定义的应用层服务。

本应用层服务定义标准虽然没有与设备的一致性。然而通过执行相应的满足本标准中任意给定类型的应用层服务定义的应用层协议可以实现一致性。

1.4 一致性

本部分不规定具体的设备或产品, 也不具体限制工业自动化系统中的应用层实体的实现。

本应用层服务定义标准虽然没有与设备的一致性。然而通过执行本应用层协议规范可实现标准的一致性。

2 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件, 仅所注日期的版本适用于本文件。凡是不注日期的引用文件, 其最新版本(包括所有的修改单)适用于本文件。

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC 60559, *Binary Floating-point Arithmetic for Microprocessor Systems*

ISO/IEC 7498-1, *Information technology – Open Systems Interconnection – Basic Reference Model – Part1: The Basic Model*

ISO/IEC 7498-3, *Information technology – Open Systems Interconnection – Basic Reference Model – Part3: Naming and addressing*

ISO/IEC 8802-3, *Information technology – Telecommunications and information exchange between systems - Local and metropolitan area networks – Specific requirements – Part 3: Carrier sense multiple access with collision detection (CSMA/CD) access method and physical layer specifications*

ISO/IEC 9545, *Information technology – Open Systems Interconnection – Application Layer structure*

ISO/IEC 9899, *Programming Languages – C.*

ISO/IEC 10731, *Information technology – Open Systems Interconnection – Basic Reference Model – Conventions for the definition of OSI services*

IEEE 802.1D, 2004, *IEEE standard for local and metropolitan area networks – Common specifications – Media access control (MAC) Bridges*; available at <<http://www.ieee.org>>

IEEE 802.1Q, 1998, *IEEE standard for Local and metropolitan area networks – Virtual bridged local area networks Bridges*; available at <<http://www.ieee.org>>

IETF RFC 768, *User Datagram Protocol*; available at <<http://www.ietf.org>>

IETF RFC 791, *Internet Protocol darpa internet program protocol specification*; available at <<http://www.ietf.org>>

3 术语, 定义, 符号, 缩略语和约定

为了方便起见, 使用下列术语, 定义, 符号, 缩略语和约定。

3.1 参考模型术语和定义

本部分基于 ISO/IEC 7498-1 与 ISO/IEC 7498-3 中的概念, 并使用下列定义的术语:

3.1.1	对应(N)实体	correspondent (N)-entities	[7498-1]
	对应 AL 实体	correspondent AL-entities (N=7)	
3.1.2	(N) 实体	(N)-entity	[7498-1]
	AL 实体(N=7)	AL-entity (N=7)	
3.1.3	(N) 层	(N)-layer	[7498-1]
	AL 层(N=7)	AL-layer (N=7)	
3.1.4	层管理	layer-management	[7498-1]
3.1.5	对等实体	peer-entities	[7498-1]
3.1.6	原语名称	primitive name	[7498-3]
3.1.7	AL 协议	AL-protocol	[7498-1]
3.1.8	AL 协议数据单元	AL-protocol-data-unit	[7498-1]
3.1.9	复位	reset	[7498-1]
3.1.10	路由	routing	[7498-1]
3.1.11	分段	segmenting	[7498-1]
3.1.12	(N)服务	(N)-service	[7498-1]
	AL 服务(N=7)	AL-service (N=7)	
3.1.13	AL 服务数据单元	AL-service-data-unit	[7498-1]
3.1.14	AL 单工传输	AL-simplex-transmission	[7498-1]
3.1.15	AL 子系统	AL-subsystem	[7498-1]
3.1.16	系统管理	systems-management	[7498-1]
3.1.17	AL 用户数据	AL-user-data	[7498-1]

3.2 服务协议术语和定义

本部分还使用了用于数据链路层的 ISO/IEC 10731 中的下列术语定义:

3.2.1	接受者	acceptor
3.2.2	不对称服务	asymmetrical service
3.2.3	证实(原语) 请求者.交付(原语)	confirm (primitive); requestor.deliver (primitive)
3.2.4	交付(原语)	deliver (primitive)
3.2.5	AL 服务原语	AL-service-primitive;
3.2.6	AL 服务提供者	AL-service-provider
3.2.7	AL 服务用户	AL-service-user
3.2.8	指示(原语) 接受者.交付(原语)	indication (primitive); acceptor.deliver (primitive)
3.2.9	请求(原语) 请求者.提交(原语)	request (primitive); requestor.submit (primitive)
3.2.10	请求者	requestor
3.2.11	响应(原语) 接受者.提交(原语)	response (primitive); acceptor.submit (primitive)
3.2.12	提交(原语)	submit (primitive)
3.2.13	对称服务	symmetrical service

3.3应用层定义

3.3.1

应用 Application

生产或消费数据的函数或数据结构

3.3.2

应用对象 application objects

通过网络间和网络设备内来管理和提供信息交换的运行时的多个对象类

3.3.3

应用过程 application process

网络上分布式应用的一部分, 其在设备上可以确切寻址

3.3.4

应用关系 application relationship

在两个或多个应用实体调用间, 为了交换信息和协调运行而形成的合作关系。这种关系可通过交换应用协议数据单元或预配置活动来激活

3.3.5

属性 attribute

对象的外部可见的特征或特性的描述

注: 对象的属性包含对象的变量部分信息。通常情况下, 它们提供状态信息或监管对象的操作。属性也可能会影响对象的行为。属性分为类属性和实例属性。

3.3.6

基本型从站 basic slave

只支持数据物理寻址的从站设备

3.3.7

行为 **behavior**

一个对象如何响应特定事件的表现

3.3.8

位 **Bit**

由 1 和 0 组成的信息单元, 是可发送的最小数据单元

3.3.9

通道 **channel**

表示从站的单个物理或逻辑管理对象, 用于控制数据的传送

3.3.10

客户端 **Client**

- 1) 使用另一个对象的服务来执行任务的对象
- 2) 服务器所响应的消息的发起者

3.3.11

时钟同步 **clock synchroization**

时间主站发起一系列交互来同步所有时间接收器的时钟

3.3.12

通信对象 **Communication object**

通过网络管理和提供信息交换的运行时的组件

3.3.13

连接 **connection**

在相同或不同设备中的两个应用对象之间的逻辑绑定

3.3.14

消费 **consume**

从提供者接收数据的行为

3.3.15

消费者 **consumer**

从提供者接收数据的节点或汇点 (sink)

3.3.16

传输路径 **conveyance path**

APDU 通过应用关系的单向流

3.3.17

周期 **Cyclic**

以定期和重复模式进行重复的事件

3.3.18

数据 **Data**

泛指现场总线上传送的任何信息

3.3.19

数据一致性 **data consistency**

实现客户端与服务器之间及其内部对输入输出数据对象的一致性传输和访问的方法

3.3.20

数据类型 data type

某种数据类型的值与编码之间的关系。IEC61131-3 的数据类型定义适用于本规范

3.3.21

数据类型对象 data type object

在对象字典中指示为数据类型的条目

3.3.22

默认网关 default gateway

至少有两个接口在不同的 IP 子网, 并作为子网路由器的设备

3.3.23

设备 Device

连接到至少由一个通信部件(网络部件)组成的现场总线的物理实体, 并且该物理实体可能包含一个控制部件和/或一个终端部件(变送器, 执行器等)

3.3.24

设备行规 device profile

设备相关信息以及提供相似设备间的兼容性的功能的集合

3.3.25

诊断信息 diagnosis information

在服务器上出于维护目的的所有可用的数据

3.3.26

分布式时钟 distributed clocks

同步从站并维持一个全局时基的方法

3.3.27

误差 Error

由计算、观察或测量而来的值或状态与指定或理论上正确的值或状态之间的差异

3.3.28

错误类 error class

相关错误定义与对应错误码的通用类

3.3.29

错误码 error code

在一个错误类中一个特定类型错误的标识

3.3.30

事件 Event

状态发生变化的实例

3.3.31

现场总线内存管理单元 fieldbus memory management unit

现场总线内存管理单元拥有在逻辑地址和物理内存间建立一个或多个对应关系的功能

3.3.32

现场总线内存管理单元实体 **fieldbus memory management unit entity**

现场总线内存管理单元的单个元素: 一个相干的逻辑地址空间和一个相干的物理内存位置之间的对应关系

3.3.33

帧 **Frame**

DLPDU 的同义语

3.3.34

完整型从站 **full slave**

同时支持数据的物理寻址和逻辑寻址的从站设备

3.3.35

索引 **Index**

在应用进程中一个对象的地址

3.3.36

接口 **Interface**

由功能特性、信号特性或其他合适的特性划分的两个功能单元之间的共享边界

3.3.37

小端模式 **little endian**

一种数据表示方法, 对于大于 8 位的数, 先发送低 8 位

3.3.38

主站 **Master**

控制网络上的数据传送, 通过发送报文来初始化对从站的媒体访问, 并构成控制系统的接口的设备

3.3.39

映射 **Mapping**

两个对象间的对应关系, 使得一个对象成为另一对象的一部分

3.3.40

映射参数 **mapping parameters**

用于定义应用对象与过程数据对象之间通信的一组值

3.3.41

媒体 **Medium**

在两点或多点间传输通信信号的电缆、光纤或其他介质

注: "media"一词只用其复数形式"medium"

3.3.42

报文 **Message**

用于传送信息的一系列有序的八位位组

注: 通常用于在应用层对等实体之间传递信息

3.3.43

网络 **Network**

所有的媒体、连接器、中继器、路由器、网关和相关节点通信部件, 用以实现一组通信设备互连

3.3.44

节点 Node

- a) 出现在一个本地链路上的单个 DL 实体
- b) 一个网络的链路端点或两个及以上链路的交点[参考 IEC 61158-2]

3.3.45

对象 object

设备内某一特定组件的抽象描述

注: 一个对象可以是:

- a) 一个设备能力的抽象描述, 它由以下任一或所有部分组成:
 - 1) 数据 (随时间改变的信息);
 - 2) 组态 (行为的参数);
 - 3) 方法 (用数据和组态可以做的事情);
- b) 相关数据 (以变量的形式) 和操作这些数据的方法 (程序) 的集合。这些方法具有明确定义的接口和行为。

3.3.46

对象字典 object dictionary

通过包含数据类型对象、通信对象和应用对象描述的索引和子索引寻址的数据结构

3.3.47

过程数据 process data

以处理为目的的数据对象, 其中包含指定周期传输或非周期传输的应用对象

3.3.48

过程数据对象 process data object

通过包含一个或多个过程数据实体的映射参数来描述的结构

3.3.49

生产者 producer

向一个或多个消费者发送数据的节点或源

3.3.50

资源 Resource

处理能力或信息容量

3.3.51

段 Segment

现场总线被其特性阻抗终止的主干电缆段

3.3.52

服务器 Server

为另一个 (客户端) 对象提供服务的对象

3.3.53

服务 Service

对象和/或对象类的执行是建立在另一个对象和/或对象类的请求基础上的操作或功能

3.3.54

从站 Slave

只有在被主站或前面的从站启动后, 才能执行媒体访问的 DL 实体

3.3.55

子索引 sub-index

在对象字典中的一个对象的子地址

3.3.56

同步管理器 Sync Manager

控制单元的集合, 用来协调对同时使用的对象的访问

3.3.57

同步管理器通道 Sync Manager channel

用来协调对同时使用的对象的访问的单个控制单元

3.3.58

交换机 Switch

在 IEEE 802.1D 中定义的 MAC 桥

3.4 通用符号和缩写

3.4.1	AL-	Application layer(as a prefix)	应用层 (作为前缀)
3.4.2	ALE	AL-entity(the local active instance of the application layer)	应用层实体 (应用层的本地活动实例)
3.4.3	AL	AL-layer	应用层
3.4.4	APDU	AL-protocol-data-unit	应用层协议数据单元
3.4.5	ALM	AL-management	应用层管理
3.4.6	ALME	AL-management Entity(the local active instance of AL-management)	应用层管理实体 (AL管理的本地活动实例)
3.4.7	ALMS	AL-management service	应用层管理服务
3.4.8	ALS	AL-service	应用层服务
3.4.9	ALSDU	AL-service-data-unit	应用层服务数据单元
3.4.10	DL	Data-link-layer	数据链路层
3.4.11	FIFO	First-in first-out(queueing method)	先进先出 (队列方式)
3.4.12	OSI	Open systems interconnection	开放系统互连
3.4.13	PhL	Ph-layer	物理层
3.4.14	QoS	Quality of service	服务质量

3.5 附加的符号和缩略语

3.5.1	ASE	Application service element	应用服务元素
3.5.2	AR	Application relationship	应用关系
3.5.3	CAN	Controller Area Network	控制器局域网
3.5.4	CiA	CAN in automation	自动化中的CAN
3.5.5	CoE	CAN application protocol over EtherCAT services	基于EtherCAT服务的CAN应用协议
3.5.6	CSMA/CD	Carrier sense multiple access with collision detection	带冲突检测的载波监听多路访问
3.5.7	DC	Distributed clocks	分布式时钟
3.5.8	DNS	Domain name system (server for name resolution in IP networks)	域名服务器 (IP网络中用于名称解析的服务器)
3.5.9	E ² PROM	Electrically erasable programmable read only memory	电可擦可编程只读存储器
3.5.10	EoE	Ethernet tunneled over EtherCAT services	基于EtherCAT服务的以太网隧道

3.5.11	ESC	EtherCAT slave controller	EtherCAT从站控制器
3.5.12	FCS	Frame check sequence	帧校验序列
3.5.13	FMMU	Fieldbus memory management unit	现场总线内存管理单元
3.5.14	FoE	File access with EtherCAT services	基于EtherCAT服务的文件访问
3.5.15	HDR	Header	帧头
3.5.16	ID	Identifier	标识符
3.5.17	IETF	Internet engineering task force	因特网工程任务组
3.5.18	IP	Internet protocol	因特网协议
3.5.19	LAN	Local area network	局域网
3.5.20	MAC	Medium access control	媒体访问控制
3.5.21	OD	Object dictionary	对象字典
3.5.22	PDI	Physical device internal interface (a set of elements that allows access to DL services from the AL)	物理设备内部接口 (允许DL用户访问DL服务的一组元素)
3.5.23	PDO	Process data object	过程数据对象
3.5.24	RAM	Random access memory	随机存取存储器
3.5.25	Rx	Receive	接收
3.5.26	SDO	Service data object	服务数据对象
3.5.27	SII	Slave information interface	从站信息接口
3.5.28	SM	Synchronization manager	同步管理器
3.5.29	SoE	Servo drive profile with EtherCAT services	基于EtherCAT服务的伺服驱动行规
3.5.30	SyncM	Synchronization manager	同步管理器
3.5.31	TCP	Transmission control protocol	传输控制协议
3.5.32	Tx	Transmit	发送
3.5.33	UDP	User datagram protocol	用户数据包协议
3.5.34	VoE	Profile specific services	行规特定服务
3.5.35	WKC	Working counter	工作计数器

3.6 约定

3.6.1 一般概念

以下服务均已在 ETG.1000.3 中进行了详细说明。服务规范中对 EtherCAT DL 所提供的服务进行了定义。这些服务到 ISO/IEC 8802-3 的映射在本部分中描述。

本标准采用 ISO/ IEC10731 中所记叙的的描述性约定。

3.6.2 保留位和八位位组的编码约定

术语 **Reserved** 可用于描述八位位组的位或整个八位位组。在发送端, 所有保留位或保留八位位组都应设为 0, 并且不得在接收端进行测试, 除非明确说明要求检查或被状态机检查。

术语 **Reserved** 也可用于指明一个参数范围内的特定值是用于将来扩展用的。在这种情况下, 保留值不应用在发送端, 并且不得在接收端进行测试, 除非明确说明要求检查或由状态机检查。

3.6.3 特定字段八位位组的公共译码约定

APDUs 可能包含载有原始和压缩信息的特定字段。这些字段应按图 1 所示的顺序编码。

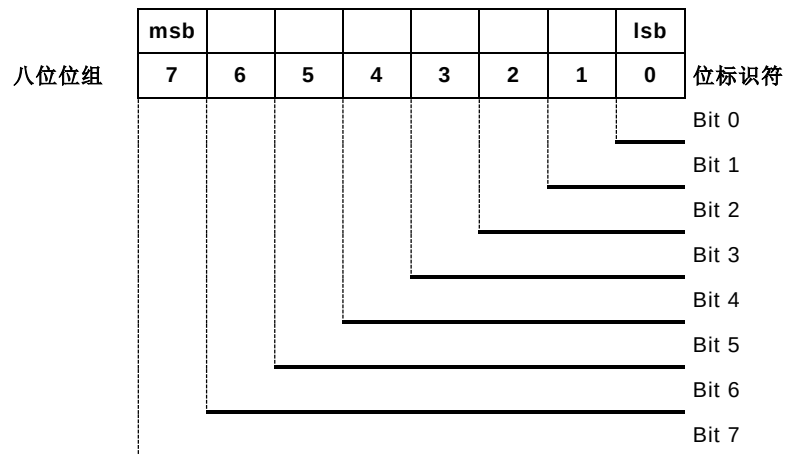


图 1 – 特定字段的公共结构

位可以组合成位组。每一个位或位组都应通过位标识符寻址（如 Bit 0, Bit 1 到 4）。八位位组内的位置应依照上图。位或位组可以用别名标识, 也可标注为保留。各个位的分组应无间隙升序。一组位的值可表示为二进制、十进制或十六进制。此值仅对该位组有效, 如果所有的 8 位均在组中的话, 则只能代表整个八位位组。十进制或十六进制的值应以二进制值的形式传输, 这样, 组中编号最高的位代表位组的 MSB。

例 1: 特定字段八位位组的描述和关系

Bit 0: Reserved

Bit 1-3: Reason_Code, 十进制值 2 意味着一般错误。

Bit 4-7: 固定设置为 1

根据上面的描述, 构造的八位位组如下所示:

(msb) Bit 7 = 1,

Bit 6 = 1,

Bit 5 = 1,

Bit 4 = 1,

Bit 3 = 0,

Bit 2 = 1,

Bit 1 = 0,

(lsb) Bit 0 = 0.

此位组表示八位位组的值为 0xF4

3.6.4 摘要语法约定

与 PDU 结构相关的各 AL 语法元素在表 1 中由一个示例进行描述。

帧部分(Frame part)中的元素在规范中具体定义。

数据字段(Data field)是元素的名称。

数据类型(Data Type)指出终端符号的类型。

值/描述(Value/Description)包含了常数值或参数的意义。

表 1 – PDU 元素的描述示例

帧部分	数据字段	数据类型	值/描述
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO Request
SDO	Size Indicator	Unsigned1	0x00: 数据大小(1..4)未定义 0x01: 数据大小在 Data Set Size 中定义
	Transfer Type	Unsigned1	0x01: 急送
	Data Set Size	Unsigned2	0x00: 4 个八位位组数据 0x01: 3 个八位位组数据 0x02: 2 个八位位组数据 0x03: 1 个八位位组数据
	Complete Access	Unsigned1	0x00
	Command Specifier	Unsigned3	0x01: 启动下载请求

在图 2 所示的列表中, 由 C 语言符号 (GB/T 15272-1994) 描述信息的属性类型。BYTE 和 WORD 是 unsigned char 和 unsigned short 类型。

```
typedef struct
{
    Unsigned8      Type;
    Unsigned8      Revision;
    Unsigned16     Build;
    Unsigned8      NoOfSuppFmmuChannels;
    Unsigned8      NoOfSuppSyncManChannels;
    Unsigned8      RamSize;
    Unsigned8      Reserved1;
    unsigned       FmmuBitOperationNotSupp: 1;
    unsigned       Reserved2: 7;
    unsigned       Reserved3: 8;
} TDLINFORMATION;
```

图 2 – 类型描述实例

对象的属性的描述如表 2 所示。

子索引 (Subindex) 描述对象的单个元素。

描述 (Description) 表示属性的名称字符串。

数据类型 (Data Type) 指出元素的类型。

M/O/C 表示该属性是强制性的(M)、可选(O)或取决于其他属性(C)设置。

访问类型显示对这个元素的访问权。R 表示读访问, W 表示写访问。可以延伸显示访问适用的 AL 状态。。

PDO 映射表示将该属性映射到 TxPDO 或 RxPDO 的可能性或表示该参数是不可映射的。

值 (Value) 包含常数值量和/或参数的含义。

表 2 – 属性描述实例

子索引	描述	数据类型	M/O/C	访问	PDO 映射	值
0	Number of entries	UNSIGNED8	M	R	No	4
1	Vendor ID	UNSIGNED32	M	R	No	由 ETG 惟一指定
2	Product Code	UNSIGNED32	M	R	No	由供应商惟一指定
3	Revision Number	UNSIGNED32	M	R	No	由供应商惟一指定 Bit 0-15: 设备的次版本 Bit 16-31: 设备的主版本

3.6.5 状态机约定

协议序列以状态机的方式描述。

在状态图中状态表示为框, 状态转换表示为箭头, 状态名和状态图转换与在状态转换表中的名称相对应。

状态转移表结构如下, 见表 3。

第一行包含转换的名称。

在第二行定义当前状态。

第三列包含一个可选事件, 接着是以 “/” 开始的条件, 最后是以 “=>” 作为起始的动作。

最后一行包含下一个状态。

如果事件发生并条件满足, 转换激发, 即: 执行动作, 并进入下一个状态。

状态机布局如表 3 所示。状态机的各个元素的含义如表 4 所示。

表 3 – 状态机描述元素

#	当前状态	事件/条件 => 动作	下一个状态

表 4 – 状态机元素的说明

描述元素	含义
当前状态 下一个状态	状态的名称
#	状态转换的名称或编号。
事件	事件的名称或描述。
/条件	Boolean 表达式。“\”不是条件的一部分。
=>动作	任务服务或函数调用的列表。“=>”不是动作的一部分

在状态机中使用的约定如表 5 所示

表 5 – 状态机中使用的约定

约定	含义
=	左侧项的值被右侧项的值替换, 如果右侧项的值是参数, 来源于原语, 可看作一个输入事件
axx	如果 a 是一个字母, 则表示参数的名称 例如: Identifier = reason 意思是 reason 参数值被赋值给名称为 Identifier 的参数。
"xxx"	表示固定的可见字符串。 例如: Identifier = "abc" 意思是"abc"值被赋值给一个名称为 Identifier 的参数
nnn	如果所有的元素都是数值, 这个项表示以十进制表示的一个数值常量
0xnn	如果所有的元素 nn 都是数值, 这个项表示以十六进制表示的一个数值常量
==	指示左侧项等于右侧项的逻辑条件
<	指示左侧项小于右侧项的逻辑条件
>	指示左侧项不等于右侧项的逻辑条件
!=	逻辑条件指示左侧项不等于右侧项
&&	逻辑“与”
	逻辑“或”
!	逻辑“非”
+ - * /	算数运算符
;	表达式的定界符

我们强烈建议读者参考属性定义、本地功能以及 FDL-PDU 定义相关的条以理解协议机。我们假设读者对于这些定义都有足够的认识, 使用时不需要任何额外的解释。

以 C 语言符号(GB/T 15272)定义的其他结构可用来描述条件和动作。

4应用层协议规范

4.1操作原理

EtherCAT 是一种旨在最大限度地利用全双工以太网带宽的实时以太网技术。媒体访问控制采用主站/从站原则, 主站节点(通常是控制系统)发送以太网帧给从站节点, 从站节点从这些帧中提取和插入数据。

从以太网的角度看, 一个 **EtherCAT** 网段就是一个单个的以太网设备, 它接收和发送标准的 **ISO/IEC8802-3** 以太网帧。但这种以太网设备并不局限于带有下游微处理器的单个以太网控制器, 也可以包含大量的 **EtherCAT** 从站设备。这些从站设备直接处理到来的以太网帧, 从中读取数据和/或插入自己的数据, 并把帧传给下一个从站设备。网段内的最后一个从站设备沿着设备链反向发送完全处理的以太网帧, 并通过第一个从站设备把收集的信息返回给主站, 主站接收信息做为以太网响应帧。

此方法采用以太网全双工的模式: 双方向的通信都是独立执行的, 主站设备和由一个或多个从站设备构成的 **EtherCAT** 网段直接通信不需要使用交换机。

工业通信系统必须满足不同要求的数据传输特性。参数数据以非周期方式被传输, 且数量大, 即时间要求相对不是很重要, 传输通常是由控制系统触发的。诊断数据也是以非周期方式传输, 是由事件驱动的, 但是这个时间要求较高, 其传输由外围设备触发。

另一方面, 过程数据通常是以不同的周期时间来循环传输的, 过程数据通信对时间的要求最严格。**EtherCAT** 支持多种服务和协议来满足这些不同的要求。

4.2节点参考模型

4.2.1 映射到 OSI 的基本参考模型

EtherCAT 的描述使用了 **ISO/IEC 7498** 信息处理系统—开放系统互连—基本参考模型 (**OSI**) 的原理, 方法和模型。**OSI** 模型提供了一种通信标准分层的方法, 其中各层可以独立开发和修改。**EtherCAT** 规范定义了完整的自顶向下的 **OSI** 协议栈和一些栈用户的功能。**OSI** 中间的 3~6 层功能被并入到了 **EtherCAT** 数据链路层或 **EtherCAT** 应用层。同样地, **EtherCAT** 应用层提供了和现场总线应用层通用的用户特性, 这简化了用户操作, 见图 3。

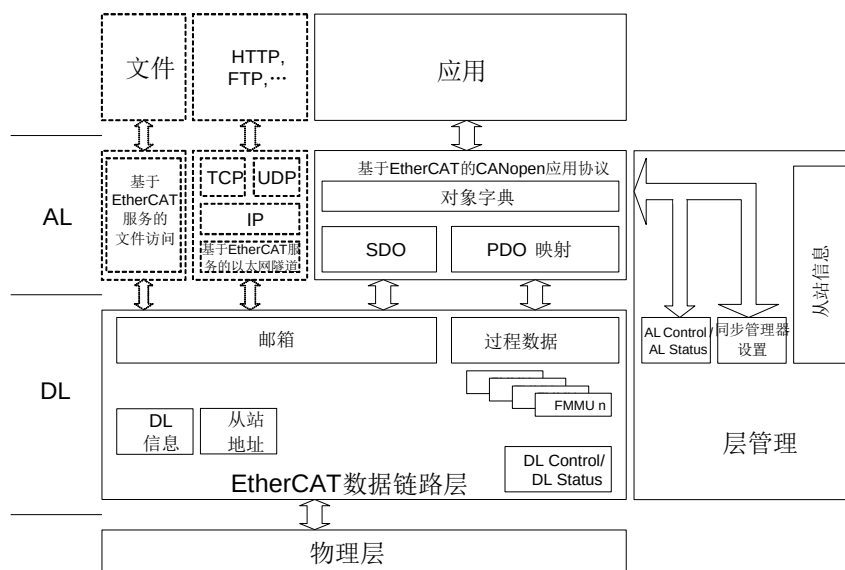


图 3 – 从站节点参考模型

4.2.2 数据链路层的特征

数据链路层为通过 EtherCAT DL 连接的设备之间的数据通讯提供基本时间关键的支持。“时间关键”一词用来描述带时窗的应用，在此时窗内，要求完成一个或多个有明确定义确定度的规定动作。在时窗内没有完成指定的动作，有可能造成需要该动作的应用的失败，甚至会影响设备、厂房及人身安全。

数据链路层的任务包括计算、比较、生成帧校验序列，并通过从以太网帧中提取或插入数据来实现通讯。这些任务依据在被预先定义的内存位置中的数据链路层参数来实现。在物理内存中通过邮箱配置或过程数据部分使得应用层能够使用应用数据。

另外，数据链路层还有一些协调主从站交互的数据结构，例如 AL 控制，状态和事件，及同步管理器设定等。

4.2.3 应用层结构

应用层包含以下元素

- 实时实体（必备的）
- 处理 TCP/UDP/IP 和相关的协议的实体（可选的）
- 文件访问工具（可选的）
- 管理单元（必备的）

应用层使用由 EtherCAT 数据链路层提供的服务传递应用层服务数据。

5FAL 语法说明

5.1 编码原则

应用层使用到的 DL 对象在 ETG.1000.5 里有详细说明。

5.2 数据类型和编码规则

5.2.1 数据类型和编码规则的一般描述

为了能够交换有意义的数据, 数据格式和含义必须被生产者和消费者们知道。本规范通过数据类型的概念模型化以上需求。

编码规则定义了数据类型的数值的描述以及传输语法的描述。数值以位序列描述。位序列以八位位组为单位传输。数值的数据类型以小节字序风格编码, 如表 6。

数据类型编码规则应该同时适用 DL 服务和协议以及 AL 服务和协议的规定。在 ISO/IEC 8802-3 中指定以太网帧的编码规则。以太网的 DLSDU 是一个八位位组串。八位位组的传送顺序取决于 MAC 和 PhL 的编码规则。

5.2.2 Boolean 值的编码

- Boolean 值的编码应是基本的。内容八位位组 (ContentsOctets) 由单个的八位位组所组成。
- 如果 Boolean 值为 FALSE, 内容八位位组应为 0。如果 Boolean 值为 TRUE, 内容八位位组应为 0xff。

5.2.3 Time Of Day 值 (带日期/不带日期) 的编码

- Time Of Day 值 (带日期/不带日期) 的编码是基本的。
- 其内容八位位组应和数据值的八位位组的值相等, 见图 4:

bits	7	6	5	4	3	2	1	0	
octets									从 00:00 开始的 ms 数
1	0	0	0	0	2^{27}	2^{26}	2^{25}	2^{24}	
2	2^{23}	2^{22}	2^{21}	2^{20}	2^{19}	2^{18}	2^{17}	2^{16}	
3	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
4	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	从 1984 年 1 月 1 日开始的日期计数, 只表示日期
5	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	
6	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
msb									

图 4 – Time Of Day 的编码

5.2.4 Time Difference 值（带日期/不带日期）的编码

- a) Time Difference（带日期/不带日期）的编码是基本的。
b) 其内容八位位组应和数据值的八位位组的值相等，见图 5：

bits	7	6	5	4	3	2	1	0	
octets									ms
1	2 ³¹	2 ³⁰	2 ²⁹	2 ²⁸	2 ²⁷	2 ²⁶	2 ²⁵	2 ²⁴	
2	2 ²³	2 ²²	2 ²¹	2 ²⁰	2 ¹⁹	2 ¹⁸	2 ¹⁷	2 ¹⁶	
3	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	
4	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	天，只表示日期
5	2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	
6	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	
msb									

图 5 – Time Difference 的编码

5.2.5 位序列传输语法

为了传送一个位序列，要将其重新编入一个八位位组序列中。在 GB/T 15272 中十六进制表示法同样可以用于八位位组。使 $b = b_0 \dots b_{n-1}$ 成为一个位序列。符号 k 是一个非负整数，使得 $8(k-1) < n \leq 8k$ ，那么 b 被转换为 k 个八位位组，如表 6。最高编号的八位位组的位 b_i ， $i \geq n$ 是不关注位的。

八位位组 1 先被传送，八位位组 k 最后被传送。因此传送位序列是依次穿过网络的（八位位组内的传送顺序由 ISO/IEC 8802-3 决定）：

$b_7, b_6, \dots, b_0, b_{15}, \dots, b_8, \dots$

表6 – 位序列的传输语法

八位位组编号	1.	2.	k.
	$b_7 \dots b_0$	$b_{15} \dots b_8$	$b_{8k-1} \dots b_{8k-8}$

例

Bit 9	...	Bit 0
10b	0001b	1100b
0x2	0x1	0xC
		= 0x21C

位序列 $b = b_0 \dots b_9 = 0011\ 1000\ 01_b$ 表示一个值为 0x21C 的 10 位无符号数。它可以在 2 个八位位组里进行传输：首先 0x1C 和 然后 0x02。

5.2.6 Unsigned Integer 值的编码

基本数据类型 Unsignedn 的值是非负整数, 值的范围是 $0, \dots, 2^n-1$ 。数据表示长度为 n 的位序列。

位序列

$$b = b_0 \dots b_{n-1}$$

被赋值

$$\text{Unsignedn}(b) = b_{n-1} \times 2^{n-1} + \dots + b_1 \times 2^1 + b_0 \times 2^0$$

位序列从左边最低有效字节(八位位组)开始。

示例: 值 266 = 0x10A, 数据类型是 Unsigned16, 分成 2 个八位位组发送, 第一个是 0x0A 然后是 0x01

Unsignedn 数据类型的传送见表 7。也使用 Unsigned 数据类型 Unsigned1~Unsigned7 以及 Unsigned9~Unsigned15。在这种情况下, 下一个元素将从第一个自由位的位置开始, 见 3.6.3。

表7 – Unsignedn 数据类型的传输语法

八位位组编号	1.	2.	3.	4.	5.	6.	7.	8.
Unsigned8	b7..b0							
Unsigned16	b7..b0	b15..b8						
Unsigned32	b7..b0	b15..b8	b23..b16	b31..b24				
Unsigned64	b7..b0	b15..b8	b23..b16	b31..b24	b39..b32	b47..b40	b55..b48	b63..b56

5.2.7 Signed Integer 值的编码

基本数据类型 Integern 的值是整数, 值的范围是 -2^{n-1} 到 $2^{n-1}-1$ 。数据表示长度为 n 的位序列。位序列

$$b = b_0 \dots b_{n-1}$$

被赋值

$$\text{Integern}(b) = b_{n-2} \times 2^{n-2} + \dots + b_1 \times 2^1 + b_0 \times 2^0 \text{ if } b_{n-1} = 0$$

并且, 执行 2 的补码

$$\text{Integern}(b) = -\text{Integern}(\text{b}) - 1 \text{ if } b_{n-1} = 1$$

注意位序列从左边最低有效位开始。

示例: 值 -266 = 0xFE6 的数据类型为 Integer16, 分成 2 个八位位组发送, 第一个是 0xF6 然后是 0xFE。

Integern 数据类型的传输见表 8。也使用 Integer 数据类型 Integer1~Integer7 以及 Integer9~Integer15。在这种情况下, 下一个元素将从第一个自由位的位置开始, 见 3.6.3。

表 8 – Integern 数据类型的传输语法

八位位组编号	1.	2.	3.	4.	5.	6.	7.	8.
Integer8	b7..b0							
Integer16	b7..b0	b15..b8						
Integer32	b7..b0	b15..b8	b23..b16	b31..b24				
Integer64	b7..b0	b15..b8	b23..b16	b31..b24	b39..b32	b47..b40	b55..b48	b63..b56

5.2.8 Floating Point 值的编码

Float32 ::= OCTET STRING SIZE (4) -- IEC 60559 单精度

Float64 ::= OCTET STRING SIZE (8) -- IEC 60559 双精度

5.2.9 Visible String 值的编码

- 可变长度的 Visible String 值的编码应是基本的。
- 该数据类型没有长度字段, 也没有结束标志, 长度被隐式编码。
- 内容八位位组应是一个八位位组序列。最左边的字符串元素被编码在第一个八位位组里, 接下来的在第二个, 以此类推直到最右边的一个字符串元素在最后一个八位位组里被编码。

5.2.10 Unicode String 值的编码

- 可变长度的 Unicode String 值的编码应是基本的。
- 该数据类型没有长度字段, 长度被隐式编码。
- 内容八位位组应是一个无符号整数序列。最左边的字符串元素的编码在第一个无符号整数里进行, 接下来的在第二个, 以此类推直到最右边的一个字符串元素在最后一个无符号整数里被编码。

5.2.11 Octet String 值的编码

- 可变长度的 Octet String 值的编码应是基本的。
- 该数据类型没有长度字段, 长度被隐式编码。
- 内容八位位组应是一个八位位组序列。最左边的字符串元素的编码在第一个八位位组里进行, 接下来的在第二个, 以此类推直到最右边的一个字符串元素在最后一个八位位组里被编码。

5.3AR 编码

5.2.12 AL 控制请求 (指示)

图 6 描述 AL 控制请求的属性类型。

```
typedef struct
{
    unsigned    State:          4;
    unsigned    Acknowledge:    1;
    unsigned    Reserved:       3;
    unsigned    ApplSpecific:    8;
} TALCONTROL;
```

图 6 – AL 控制请求结构

AL 控制 (AL Control) 请求被映射到对 DL-user 控制寄存器对象和 R2 的 DL 写服务 (ETG.1000.3 中规定)。表 9 描述 AL 控制请求编码。

表 9 – AL 控制描述

参数	DL 用户寄存器	数据类型	值
State	R1	Unsigned4	1: Init 2: Pre-Operational 3: Bootstrap 4: Safe-Operational 8: Operational
Acknowledge	R1	Unsigned1	0: AL 状态寄存器的参数 Change 不被改变 1: AL 状态寄存器的参数 Change 将被复位
Reserved	R1	Unsigned3	应为 0
Application Specific	R2	Unsigned8	

5.2.13 AL 控制 (AL Control) 响应 (证实)

AL 控制响应被映射到针对 DL-user 控制寄存器对象和 R4、R5、R6 的 DL 读服务 (ETG.1000.3 中规定)。图 7 描述 AL 控制响应的属性类型。

```
typedef struct
{
    unsigned    State:          4;
    unsigned    Change:         1;
    unsigned    Reserved1:      3;
    unsigned    ApplSpecific:    8;
    unsigned    Reserved2:      16;
    unsigned    AlStatusCode:    16;
} TALSTATUS;
```

图 7 – AL 控制响应结构

表 10 描述 AL 控制响应编码。

表 10 – AL 控制响应

参数	DL-user 寄存器	数据类型	值
State	R3	Unsigned4	1: Init 2: Pre-Operational 3: Bootstrap 4: Safe-Operational 8: Operational
Change	R3	Unsigned1	0: 状态转换成功。 1: 状态转换不成功。
Reserved	R3	Unsigned3	
Application Specific	R4	Unsigned8	
Reserved	R5	Unsigned16	
Application Specific	R6	Unsigned16	见 表 11

表 11 – AL 状态编码

编码	描述	当前状态 (或状态变化)	结果状态
0x0000	无错误	任意	当前状态 + E
0x0001	未特指的错误	任意	任意 + E
0x0002	无内存	任意	任意 + E
0x0011	无效的请求状态改变	I -> S, I -> O, P -> O O -> B, S -> B, P -> B	当前状态 + E
0x0012	未知的请求状态	任意	当前状态 + E
0x0013	引导不被支持	I -> B	I + E
0x0014	无有效固件	I -> P	I + E
0x0015	无效的邮箱配置	I -> B	I + E
0x0016	无效的邮箱配置	I -> P	I + E
0x0017	无效的同步管理器配置	P -> S, S -> O	当前状态 + E
0x0018	无有效输入	O, S -> O,	S + E
0x0019	无有效输出	O, S -> O	S + E
0x001A	同步错误	O, S -> O	S + E
0x001B	同步管理器看门狗	O, S	S + E
0x001C	无效的同步管理器类型	O, S, P -> S	S + E
0x001D	无效的输出生配置	O, S, P -> S	S + E
0x001E	无效的输入配置	O, S, P -> S	P + E
0x001F	无效的看门狗配置	O, S, P -> S	P + E
0x0020	从站需要冷启动	任意	当前状态 + E
0x0021	从站需要 INIT	B, P, S, O	当前状态 + E
0x0022	从站需要 PREOP	S, O	S + E, O + E
0x0023	从站需要 SAFEOP	O	O + E
0x0024	无效的输入映射	P -> S	P + E
0x0025	无效的输出生映射	P -> S	P + E
0x0026	设置不一致	P -> S	P + E
0x0027	不支持 FreeRun	P -> S	P + E
0x0028	不支持 SyncMode	P -> S	P + E
0x0029	FreeRun 需要 3 个缓存区的模式	P -> S	P + E
0x002A	背景看门狗	S, O	P + E
0x002B	无有效的输入和输出	O, S -> O	S + E
0x002C	致命的同步错误	O	S + E
0x002D	无同步错误	S -> O	S + E
0x0030	无效的 DC SYNCH 配置	O, S -> O, P -> S	P + E, S + E
0x0031	无效的 DC 锁定配置	O, S -> O, P -> S	P + E, S + E
0x0032	PLL 错误	O, S -> O	S + E
0x0033	DC 同步 IO 错误	O, S -> O	S + E
0x0034	DC 同步超时错误	O, S -> O	S + E
0x0035	DC 无效同步周期时间	P -> S	P + E

0x0036	DC Sync0 周期时间	P -> S	P + E
0x0037	DC Sync1 周期时间	P -> S	P + E
0x0041	MBX_AOE	B, P, S, O	当前状态 + E
0x0042	MBX_EOE	B, P, S, O	当前状态 + E
0x0043	MBX_COE	B, P, S, O	当前状态 + E
0x0044	MBX_FOE	B, P, S, O	当前状态 + E
0x0045	MBX_SOE	B, P, S, O	当前状态 + E
0x004F	MBX_VOE	B, P, S, O	当前状态 + E
0x0050	EEPROM 无访问	任意	任意 + E
0x0051	EEPROM 错误	任意	任意 + E
0x0060	从站本地重启	任意	I
other codes < 0x8000	保留		
0x8000 – 0xFFFF	特定供应商		

5.2.14 AL 状态改变 (AL State Changed)

AL 状态改变被映射到对 AL 状态和 AL 状态代码对象的 DL 读服务。图 8 描述 AL 状态改变的属性类型。

```
typedef struct
{
    unsigned    State:          4;
    unsigned    Change:         1;
    unsigned    Reserved1:      3;
    unsigned    ApplSpecific:   8;
    unsigned    Reserved2:     16;
    unsigned    AlStatusCode:  16;
} TALSTATUS;
```

图 8 – AL 状态变化结构

表 12 规定 AL 状态改变编码。

表 12 – AL 状态改变

参数	DL-user 寄存器	数据类型	值
State	R3	Unsigned4	1: Init 2: Pre-Operational 3: Bootstrap 4: Safe-Operational 8: Operational
Change	R3	Unsigned1	应为 1
Reserved	R3	Unsigned3	
Application Specific	R4	Unsigned8	
Reserved	R5	Unsigned16	
AL Status Code	R6	Unsigned16	见表 11

5.2.15 AL AR 属性

AL AR 属性可由 DL 读或写服务或由本地读写服务访问。

图 9 描述 PDI 控制 (PDI Control) 的属性类型。

```
typedef struct
{
    unsigned    PDType:                8;
    unsigned    StrictALControl:       1;
    unsigned    Reserved:              7;
} TPDICONTROL;
```

图 9 – PDI 控制类型描述

表 13 指定 PDI 控制编码。从 SII 启动时, 将被载入 PDI 控制。

表 13 – PDI 控制

参数	DL-user 寄存器	数据类型	DL 访问	本地访问	值/描述
PDI Type	R7	Unsigned8	R	R	类型特定的 (见 ETG1000 3 DL 信息参数)
Strict AL Control	Copy	Unsigned1	R	R	0x00: AL 管理将通过一个应用控制器执行。 0x01: AL 管理将被模拟 (AL 状态直接遵照 AL 控制)

PDI 配置编码是控制器特定的, 见表 14, 在启动时由 SII 设置。

表 14 – PDI 配置

参数	DL-user 寄存器	数据类型	DL 访问	本地访问	值/描述
Application Specific	R8	unsigned8	R	R	

图 10 描述同步配置的属性类型。

```
typedef struct
{
    unsigned    SignalCondSync0:       2;
    unsigned    EnableSignalSync0:     1;
    unsigned    EnableInterruptSync0:  1;
    unsigned    SignalCondSync1:       2;
    unsigned    EnableSignalSync1:     1;
    unsigned    EnableInterruptSync1:  1;
} TSYNCCFG;
```

图 10 – 同步配置类型描述

同步配置编码见表 15。在启动时, 同步配置从 SII 载入。

表 15 – 同步配置

参数	DL-user 寄存器	数据类型	DL 访问	本地访问	值/描述
Signal Conditioning Sync0	R8	unsigned2	R	R	控制器特定
Enable Signal Sync0	R8	unsigned1	R	R	0x00: 禁用 0x01: 启用
Enable Interrupt Sync0	R8	unsigned1	R	R	0x00: 禁用 0x01: 启用
Signal Conditioning Sync1	R8	unsigned2	R	R	控制器特定
Enable Signal Sync1	R8	unsigned1	R	R	0x00: 禁用 0x01: 启用
Enable Interrupt Sync1	R8	unsigned1	R	R	0x00: 禁用 0x01: 启用

5. 4SII 编码

从站信息接口区域编码见表 16 和表 17。地址表示一个字地址（例：0 是第一个字，1 是第二个字）。

表 16 – 从站信息接口区域

参数	地址	数据类型	值/描述
PDI Control	0x0000	Unsigned16	PDI 控制寄存器初始值(0x140-0x141)
PDI Configuration	0x0001	Unsigned16	PDI 配置寄存器初始值(0x150-0x151)
SyncImpulseLen	0x0002	Unsigned16	以 10ns 为倍数的同步脉冲
PDI Configuration2	0x0003	Unsigned16	PDI 配置寄存器 R8 最重要字的初始值(0x152-0x153)
Configured Station Alias	0x0004	Unsigned16	地址别名
Reserved	0x0005	BYTE[4]	应为 0
Checksum	0x0007	Unsigned16	低字节包含 word0 到 word6 作为无符号数除以多项式 x^8+x^2+x+1 后的余数 (初始值 0xFF)
Vendor ID	0x0008	Unsigned32	CAN-对象 0x1018, 子索引 1
Product Code	0x000A	Unsigned32	CAN-对象 0x1018, 子索引 2
Revision Number	0x000C	Unsigned32	CAN-对象 0x1018, 子索引 3
Serial Number	0x000E	Unsigned32	CAN-对象 0x1018, 子索引 4
Reserved	0x0013	BYTE[8]	应为 0
Bootstrap Receive Mailbox Offset	0x0014	Unsigned16	引导状态的接收邮箱偏移量（主站到从站）
Bootstrap Receive Mailbox Size	0x0015	Unsigned16	引导状态的邮箱接收大小（主站到从站）
Bootstrap Send Mailbox Offset	0x0016	Unsigned16	引导状态的发送邮箱偏移（从站到主站）
Bootstrap Send Mailbox Size	0x0017	Unsigned16	引导状态的发送邮箱大小（从站到主站）
Standard Receive Mailbox Offset	0x0018	Unsigned16	标准状态的接收邮箱偏移（主站到从站）

参数	地址	数据类型	值/描述
Standard Receive Mailbox Size	0x0019	Unsigned16	标准状态的接收邮箱大小 (主站到从站)
Standard Send Mailbox Offset	0x001A	Unsigned16	标准状态的发送邮箱偏移 (从站到主站)
Standard Send Mailbox Size	0x001B	Unsigned16	标准状态的发送邮箱大小 (从站到主站)
Mailbox Protocol	0x001C	Unsigned16	支持的邮箱协议, 见表 18
Reserved	0x002B	BYTE[66]	应为 0
Size	0x003E	Unsigned16	E ² PROM 的大小 KBit-1
Version	0x003F	Unsigned16	版本为 1

表 17 – 从站信息接口类别

参数	地址	数据类型	值/描述
First Category Header	0x0040	Unsigned15	类别类型 (Category Type) 在表 19 中定义
	0x0040	Unsigned1	供应商特定
	0x0041	Unsigned16	后面的类别字的大小 x
First Category Data	0x0042	Category dependent	类别数据
Second Category Header	0x0042 + x	Unsigned15	类别类型在表 19 中定义
	0x0042 + x	Unsigned1	供应商特定
	0x0043 + x	Unsigned16	后面的类别字的大小
Second Category Data	0x0044 + x	Category dependent	类别数据
...			以此类推, 直到最后一个类别

表 18 – 支持的邮箱协议类型

协议	值	描述
EoE	0x0002	基于 EtherCAT 的以太网 (使用数据链路服务的隧道)
CoE	0x0004	基于 EtherCATCAN 应用协议 (访问 SDO)
FoE	0x0008	基于 EtherCAT 文件访问
SoE	0x0010	基于 EtherCAT 伺服驱动行规
VoE	0x0020	基于 EtherCAT 供应商特定协议

表 19 – 种类类型

协议	值	描述
NOP	00	无信息
STRINGS	10	本类别数据的其他类别结构的字符串储存见表 20
DataTypes	20	将来使用的数据类型
General	30	本类别数据的一般信息结构见表 21
FMMU	40	本类别数据使用的 FMMU 结构见表 22
SyncM	41	本类别数据的同步管理器的配置结构见表 23
TXPDO	50	本类别数据的 TxPDO 结构描述见表 24
RXPDO	51	本类别数据的 RxPDO 结构描述见表 24
DC	60	将来使用的分布式时钟
End	0xffff	

表 20 – String 类别的结构

参数	字节地址	数据类型	值/描述
nStrings	0x0000	Unsigned8	字符串数量
str1_len	0x0001	Unsigned8	字符串 1 的长度
str_1	0x0002	BYTE [str1_len]	字符串 1 的数据
str2_len	0x0002+str1_len	Unsigned8	字符串 2 的长度
str_2	0x0003+str1_len	BYTE [str2_len]	字符串 2 的数据
....			
strn_len	0x000z	Unsigned8	字符串 n 的长度
str_n	0x000z+1	BYTE [strn_len]	字符串 n 的数据
PAD_Byte	0x000y	BYTE	如果类别长度是奇数, 则需要填充

表 21 – General 类别的结构

参数	字节地址	数据类型	值/描述
GroupIdx	0x0000	Unsigned8	组信息 (供应商特定) - STRINGS 索引
ImgIdx	0x0001	Unsigned8	图像名称 (供应商特定) - STRINGS 索引
OrderIdx	0x0002	Unsigned8	设备订货号 (供应商特定) - STRINGS 索引
NameIdx	0x0003	Unsigned8	设备名称信息 (供应商特定) - STRINGS 索引
Reserved	0x0004	Unsigned8	保留
CoE Details	0x0005	Unsigned8	Bit 0: 使能 SDO Bit 1: 使能 SDO 信息 Bit 2: 使能 PDO 赋值 Bit 3: 使能 PDO 配置 Bit 4: 使能在启动时上载 Bit 5: 使能 SDO 完全访问
FoE Details	0x0006	Unsigned8	Bit 0: 使能 FoE
EoE Details	0x0007	Unsigned8	Bit 0: 使能 EoE
SoEChannels	0x0008	Unsigned8	保留
DS402Channels	0x0009	Unsigned8	保留
SysmanClass	0x000a	Unsigned8	保留
Flags	0x000b	Unsigned8	Bit 0: 使能 SafeOp Bit 1: 使能 notLRW
CurrentOnEBus	0x000c	Signed16	EBUS 电流消耗, 以 mA 为单位, 负值表示以电流供电
PAD_Byte1	0x000b	BYTE[2]	保留
Physical Port	0x0010	Unsigned16	物理接口说明: 0x00: 不使用 0x01: MII 0x02: 保留 0x03: EBUS 每个端口由 4 位表示: 3:0: Port 0 7:4: Port 1 11:8: Port 2 15:9: Port 3
PAD_Byte2	0x0012	BYTE[14]	保留

表 22 – FMMU 类别的结构

参数	字节地址	数据类型	值/描述
FMMU0	0x0000	Unsigned8	0x00: FMMU0 未使用 0x01: FMMU0 用于输出 0x02: FMMU0 用于输入 0x03: FMMU0 用于 SM 状态 (读取邮箱) 0xFF: FMMU0 未使用
FMMU1	0x0001	Unsigned8	0x00: FMMU1 未使用 0x01: FMMU1 用于输出 0x02: FMMU1 用于输入 0x03: FMMU1 用于 SM 状态 (读取邮箱) 0xFF: FMMU1 未使用
	...		如果超过 2 个 FMMU 被使用, 则以此类推

表 23 – 每个元素的 SyncM 类别的结构

参数	字节地址	数据类型	值/描述
Physical Start Address	0x0000	WORD	数据起始点 (见同步管理通道的物理起始地址)
Length	0x0002	WORD	
Control Register	0x0004	Unsigned8	定义操作的模式 (见同步管理通道的控制寄存器)
Status Register	0x0005	BYTE	忽略
Enable Synch Manager	0x0006	Unsigned8	使能同步管理通道 Bit 0: 使能 Bit 1: 固定内容 (配置工具信息-同步管理通道有固定的内容) Bit 2: 虚拟同步管理通道 (虚拟同步管理通道-不使用硬件) Bit 3: 仅 OP 状态 (同步管理通道应该仅在 OP 状态被使能) Bit 7:4: 保留
Sync Manager Type	0x0007	BYTE	同步管理器类型 0x00 = 没使用或未知 0x01 = 用于邮箱输出 0x02 = 用于邮箱输入 0x03 = 用于进程数据输出 0x04 = 用于进程数据输入

注: 从偏移量 0x006 和 0x007 得到的数据不能被用于相应 ESC 寄存器的寄存器内容。

表 24 – 每个 PDO 的 TXPDO 和 RXPDO 的结构

参数	地址	数据类型	值/描述
PDO Index	0x0000	Unsigned16	RxPDO: 0x1600 到 17FF, TxPDO: 0x1A00 到 1BFF
nEntry	0x0002	Unsigned8	条目个数
SyncM	0x0003	Unsigned8	相关同步管理器
Synchronization	0x0004	Unsigned8	到 DC 同步的引用
NameIdx	0x0005	Unsigned8	对象名称- STRINGS 索引
Flags	0x0006	WORD	将来使用
Entry 1	0x0008	8 BYTES	重复在表 25 中定义的条目
...			
Entry nEntry	nEntry*8	8 BYTES	重复在表 25 中定义的条目

表 25 – PDO Entry 的结构

参数	入口结构偏移量	数据类型	值/描述
Entry Index	0x0000	Unsigned16	条目的索引
Subindex	0x0002	Unsigned8	子索引
Entry Name Idx	0x0003	Unsigned8	条目的名称- STRINGS 索引
Data Type	0x0004	Unsigned8	条目的数据类型 (CoE 对象词典中索引)
BitLen	0x0005	Unsigned8	条目的数据长度
Flags	0x0006	WORD	将来使用

5.5 等时同步 PDI 编码

分布式时钟同步和锁存的属性类型见图 11。

```
typedef struct
{
    BYTE            Reserved1;
    unsigned        CyclicOperationEnable:    1;
    unsigned        SYNC0Activate:            1;
    unsigned        SYNC1Activate:            1;
    unsigned        Reserved2:                 5;
    WORD            SYNC0Pulse;
    BYTE            Reserved5[10];
    unsigned        Interrupt1Status:          1;
    unsigned        Reserved2:                 7;
    unsigned        Interrupt2Status:          1;
    unsigned        Reserved3:                 7;
    DWORD           CyclicOperationStartTime;
    BYTE            Reserved4[12];
    DWORD           SYNC0CycleTime;
    DWORD           SYNC1CycleTime;
    unsigned        Latch0PosEdge:             1;
    unsigned        Latch0NegEdge:             1;
    unsigned        Reserved5:                 14;
    unsigned        Latch1PosEdge:             1;
    unsigned        Latch1NegEdge:             1;
    unsigned        Reserved6:                 14;
    BYTE            Reserved7[4];
    unsigned        Latch0PosEvt:              1;
    unsigned        Latch0NegEvt:              1;
    unsigned        Reserved8:                 6;
    unsigned        Latch1PosEvt:              1;
    unsigned        Latch1NegEvt:              1;
    unsigned        Reserved9:                 6;
    DWORD           Latch0PosEdgeValue;
    BYTE            Reserveda[4];
    DWORD           Latch0NegEdgeValue;
    BYTE            Reservedb[4];
    DWORD           Latch1PosEdgeValue;
    BYTE            Reservedc[4];
    DWORD           Latch1NegEdgeValue;
    BYTE            Reservedd[4];
} TDCISOCHRON;
```

图 11 – 分布式时钟同步和锁存类型描述

分布式时钟同步参数编码见表 26。这些参数可以映射到 DL DC 用户参数的 P1~P6。事件 SYNC0 和 SYNC1 被映射到 DL 事件。

表 26 – 分布式时钟同步参数

参数	用户参数	数据类型	EtherCAT DL 访问类型	PDI 访问 类型	值/描述
Cyclic Operation Enable	P1	Unsigned1	RW	R	0: 禁用 1: 启用
SYNC0 activate	P1	Unsigned1	RW	R	0: 无效 1: 生成 SCNC0 脉冲
SYNC1 activate	P1	Unsigned1	RW	R	0: 无效 1: 生成 SCNC1 脉冲
SYNC Pulse	P2	Unsigned16	R	R	从 SII 中获取
Interrupt 0 Status	P3	Unsigned1	R	R	0: 未激活 1: 激活
Reserved	P3	Unsigned7	R	R	
Interrupt 1 Status	P3	Unsigned1	R	R	0: 未激活 1: 激活
Reserved	P3	Unsigned7	R	R	
Cyclic Operation Start Time	P4	Unsigned32	RW	R	当系统时间低 32 位达到这个值时 (单位: ns), 开始产生中断
SYNC0 Cycle Time	P5	DWORD	RW	R	SYNC0 的周期时间
SYNC1 Cycle Time	P6	DWORD	RW	R	SYNC1 的周期时间

分布式时钟锁存数据编码见表 27

表 27 – 分布式时钟锁存数据

参数	用户参数	数据类型	EtherCAT DL 访问类型	PDI 访问 类型	值/描述
Latch0 positive Edge	P7	Unsigned1	RW	R	0: 连续 1: 单个
Latch0 negative Edge	P7	Unsigned1	RW	R	0: 连续 1: 单个
Reserved	P7	Unsigned6	R	R	
Latch1 positive Edge	P7	Unsigned1	RW	R	0: 连续 1: 单个
Latch1 negative Edge	P7	Unsigned1	RW	R	0: 连续 1: 单个
Reserved	P7	Unsigned6	R	R	
Latch0 positive Event	P8	Unsigned1	RW	R	0: 没有事件 1: 事件存储
Latch0 negative Event	P8	Unsigned1	RW	R	0: 没有事件 1: 事件存储
Reserved	P8	Unsigned6	R	R	
Latch1 positive Event	P8	Unsigned1	RW	R	0: 没有事件 1: 事件存储
Latch1 negative Event	P8	Unsigned1	RW	R	0: 没有事件 1: 事件存储
Reserved	P8	Unsigned6	R	R	
Latch 0 positive Edge Value	P9	DWORD	R	R	锁存0 正值事件
Latch 0 negative Edge Value	P10	DWORD	R	R	锁存0 负值事件
Latch 1 positive Edge Value	P11	DWORD	R	R	锁存1 正值事件
Latch 1 negative Edge Value	P12	DWORD	R	R	锁存1 负值事件

5.6 CoE 编码

5.6.1 PDU 结构

图 12 描述了 CoE 的一般属性类型。

```
typedef struct
{
    unsigned    NumberLo:    8;
    unsigned    NumberHi:    1;
    unsigned    Reserved:    3;
    unsigned    Service:     4;
} TCOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TCOEHEADER    CoeHeader;
    BYTE          Data[MBX_DATA_SIZE-2];
} TCOEMBX;
```

图 12 – CoE 一般结构

表 28 规定 CoE 的编码。

表 28 – CoE 元素

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 指源站地址; 如果从站是客户机, 指目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值都是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	取决于 CoE 服务
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x01: Emergency 0x02: SDO Request 0x03: SDO Response 0x04: TxPDO 0x05: RxPDO 0x06: TxPDO remote request 0x07: RxPDO remote request 0x08: SDO Information

5.6.2 SDO

5.6.2.2 SDO 下载急送 (SDO Download Expedited)

5.6.2.2.1 SDO 下载急送请求

图 13 描述了 SDO 下载急送请求的属性类型。

```
typedef struct
{
    unsigned    SizeIndicator:    1;
    unsigned    TransferType:    1;
    unsigned    DataSetSize:    2;
    unsigned    CompleteAccess:  1;
    unsigned    Command:         3;
    BYTE        IndexLo;
    BYTE        IndexHi;
    BYTE        SubIndex;
} TINITSDOHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TCOEHEADER    CoeHeader;
    TINITSDOHEADER SdoHeader;
    BYTE          Data[4];
} TINITSDODOWNLOADEXPREQMBX;
```

图 13 – SDO 下载急送请求结构

SDO 下载急送请求编码的规定见表 29。

表 29 – SDO 下载急送请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO Request
SDO	Size Indicator	Unsigned1	0x01: Data Set Size 中规定了 Data 大小
	Transfer Type	Unsigned1	0x01: 急速传输
	Data Set Size	Unsigned2	0x00: 4 个八位位组数据 0x01: 3 个八位位组数据 0x02: 2 个八位位组数据 0x03: 1 个八位位组数据
	Complete Access	Unsigned1	0x00: 由索引和子索引进行寻址的条目将被下载。 0x01: 上传完整的对象, 子索引应为 0 (包含子索引 0) 或 1 (不包括子索引 0)
	Command Specifier	Unsigned3	0x01: 下载请求
	Index	WORD	对象的索引
	Subindex	BYTE	对象的子索引, 如果 Complete Access = 0x01, 该值为 0 或 1
	Data	BYTE[4]	对象的数据

5.6.2.2.SDO 下载急送 响应

图 14 描述了 SDO 下载急送响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TINITSDOHEADER  SdoHeader;
} TINITSODOWNLOADEXPRESMBX;
```

图 14 – SDO 下载急送响应结构

SDO 下载急送响应编码的规定见表 30。

表 30 – SDO 下载急送响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x03: SDO Response
SDO	Size Indicator	Unsigned1	0x00
	Transfer Type	Unsigned1	0x00
	Data Set Size	Unsigned2	0x00
	Complete Access	Unsigned1	0x00: 由索引和子索引进行寻址的条目将被下载。 0x01: 上传完整的对象, 子索引应为 0 (包含子索引 0) 或 1 (不包括子索引 0)
	Command Specifier	Unsigned3	0x03: 下载响应
	Index	WORD	对象的索引
	Subindex	BYTE	对象的子索引, 如果 Complete Access = 0x01, 该值为 0 或 1
	reserved	DWORD	

5.6.2.3 SDO 常规下载 (SDO Download Normal)

5.6.2.3.1 SDO 常规下载请求

图 15 描述了 SDO 常规下载请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TINITSDOHEADER  SdoHeader;
    DWORD           CompleteSize;
    BYTE            Data[MBX_DATA_SIZE-10];
} TINITSDODOWNLOADNORMREQMBX;
```

图 15 – SDO 常规下载请求结构

SDO 常规下载请求编码的规定见表 31。

表 31 – SDO 常规下载请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n >= 0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO Request
SDO	Size Indicator	Unsigned1	0x01
	Transfer Type	Unsigned1	0x00: 常规传输
	Data Set Size	Unsigned2	0x00
	Complete Access	Unsigned1	0x00: 由索引和子索引进行寻址的条目将被下载。 0x01: 上传完整的对象, 子索引应为 0 (包含子索引 0) 或 1 (不包含子索引 0)
	Command Specifier	Unsigned3	0x01: 下载请求
	Index	WORD	对象的索引
	Subindex	BYTE	对象的子索引, 如果 Complete Access = 0x01, 该值为 0 或 1
	Complete Size	DWORD	对象的完整数据大小
	Data	BYTE[n-10]	如果((Length-10) >= Complete Size): 对象的数据 如果((Length-10) < Complete Size): 对象的第一部分数据, 随后是 Download SDO Segment。

5.6.2.3 SDO 常规下载响应

SDO 常规下载响应与 SDO 下载急送响应的属性类型和编码相同 (见 5.6.2.1.2)。

5.6.2.4 下载 SDO 段 (Download SDO Segment)

5.6.2.4.1 下载 SDO 段请求

图 16 描述了下载 SDO 段请求的属性类型。

```
typedef struct
{
    unsigned    MoreFollows:    1;
    unsigned    SegDataSize:    3;
    unsigned    Toggle:         1;
    unsigned    Command:         3;
} TSDOSEGHEADER;
```

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOSEGHEADER   SdoHeader;
    BYTE            Data[MBX_DATA_SIZE-3];
} TDOWNLOADSDOSEGREQMBX;
```

图 16 – 下载 SDO 段请求结构

下载 SDO 段请求编码见表 32。

表 32 – 下载 SDO 段请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n >= 0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO 响应
SDO	More Follows	Unsigned1	0x00: 后面还有 Download SDO Segment 0x01: 最后一个 Download SDO Segment
	SegData Size	Unsigned3	定义了最后 7 个数据八位位组 (总是被发送) 包含多少数据 (只适用于 Length 为 0x0a 的情况, 否则为 0x00) 0x00: 7 个八位位组数据 0x01: 6 个八位位组数据 0x02: 5 个八位位组数据 0x03: 4 个八位位组数据 0x04: 3 个八位位组数据 0x05: 2 个八位位组数据 0x06: 1 个八位位组数据 0x07: 0 个八位位组数据
	Toggle	Unsigned1	在每次 Download SDO Segment 请求时翻转, 从 0x00 开始
	Command specifier	Unsigned3	0x00: 下载段请求。
	Data	BYTE[n-3]	对象的数据部分

5.6.2.4.2 下载 SDO 段 (Download SDO Segment) 响应

图 17 描述了下载 SDO 段响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOSEGHEADER   SdoHeader;
} TDOWNLOADSDOSEGRESMBX;
```

图 17 – 下载 SDO 段响应结构

下载 SDO 段响应的编码见表 33。

表 33 – 下载 SDO 段响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x03: SDO Response
SDO	Reserved	Unsigned4	0x00
	Toggle	Unsigned1	与对应 Download SDO Segment 请求相同
	Command Specifier	Unsigned3	0x01: 下载段响应
	Reserved	BYTE[7]	

5.6.2.5 SDO 上传急送 (SDO Upload Expedited)

5.6.2.5.1 SDO 上传急送请求

图 18 描述 SDO 上传急送请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TINITSDOHEADER   SdoHeader;
} TINITSDOUPLOADEXPREQMBX;
```

图 18 – SDO 上传急送请求结构

SDO 上传急送请求的编码见表 34。

表 34 – SDO 上传急送请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO 响应
SDO	Reserved	Unsigned4	0x00
	Complete Access	Unsigned1	0x00: 由索引和子索引进行寻址的条目将被上传。 0x01: 上传完整的对象, 子索引应为 0 (包含子索引 0) 或 1 (不包含子索引 0)
	Command Specifier	Unsigned3	0x02: 上传请求
	Index	WORD	对象的索引
	Subindex	BYTE	对象的子索引, 如果 Complete Access = 0x01, 该值为 0 或 1
	Reserved	DWORD	

5.6.2.5.SDO 上传急送响应

图 19 描述 SDO 上传急送响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TINITSDOHEADER  SdoHeader;
    BYTE            Data[4];
} TINITSDOUPLOADEXPREQMBX;
```

图 19 – SDO 上传急送响应结构

SDO 上传急送响应的编码见表 35。

表 35 – SDO 上传加速响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x03: SDO Response
SDO	Size Indicator	Unsigned1	0x01: 在 Data Set Size 中规定了 Data 大小
	Transfer Type	Unsigned1	0x01: 急送传输
	Data Set Size	Unsigned2	0x00: 4 个八位位组数据 0x01: 3 个八位位组数据 0x02: 2 个八位位组数据 0x03: 1 个八位位组数据
	Complete Access	Unsigned1	0x00: 由索引和子索引进行寻址的条目将被上传。 0x01: 上传完整的对象, 子索引应为 0 (包含子索引 0) 或 1 (不包含子索引 0)
	Command Specifier	Unsigned3	0x02: 上传响应
	Index	WORD	对象的索引
	Subindex	BYTE	对象的子索引, 如果 Complete Access = 0x01, 该值为 0 或 1
	Data	BYTE[4]	对象的数据

5.6.2.6 SDO 常规上传 (SDO Upload Normal)

5.6.2.6.1 SDO 常规上传请求

SDO 常规上传请求的属性类型和编码与 SDO 急送上传请求相同 (见 5.6.2.4.1)。

5.6.2.6.2 SDO 常规上传响应

图 20 描述 SDO 常规上传响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TINITSDOHEADER  SdoHeader;
    DWORD           CompleteSize;
    BYTE            Data[MBX_DATA_SIZE-10];
}
```

```
} TINITSDOUPLOADNORMRESMBX;
```

图 20 – SDO 常规上传响应结构

SDO 常规上传响应的编码见表 36。如果数据参数的八位位组个数小于或等于 4，则可使用 5.6.2.4.2 中规定的响应。

表 36 – SDO 常规上传响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n >= 0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x03: SDO Response
SDO	Size Indicator	Unsigned1	0x01
	Transfer Type	Unsigned1	0x00: 常规传输
	Data Set Size	Unsigned2	0x00
	Complete Access	Unsigned1	0x00: 由索引和子索引进行寻址的条目将被上传。 0x01: 上传完整的对象, 子索引应为 0 (包含子索引 0) 或 1 (不包含子索引 0)
	Command Specifier	Unsigned3	0x02: 上传响应
	Index	WORD	对象的索引
	Subindex	BYTE	对象的子索引, 如果 Complete Access = 0x01, 该值为 0 或 1
	Complete Size	DWORD	对象的完整数据大小
	Data	BYTE[n-10]	如果 ((Length-10) >= Complete Size): 对象的数据 如果 ((Length-10) < Complete Size): 首先是对象的数据部分, 随后是上传 SDO 段

5.6.2.7 上传 SDO 段 (Upload SDO Segment)

5.6.2.7.1 上传 SDO 段请求

图 21 描述上传 SDO 段请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
```

```
TSDOSEGHEADER      SdoHeader;
} TUPLOADSDOSEGREQMBX;
```

图 21 –上传 SDO 段请求结构

上传 SDO 段请求的编码见表 37。

表 37 –上传 SDO 段请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO Request
SDO	Reserved	Unsigned4	0x00
	Toggle	Unsigned1	在每个 Upload SDO Segment 请求都翻转, 从 0x00 开始
	Command Specifier	Unsigned3	0x03: 上传段请求
	Reserved	BYTE[7]	

5.6.2.7.2 上传 SDO 段响应

图 22 描述上传 SDO 段响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOSEGHEADER    SdoHeader;
    BYTE             Data[MBX_DATA_SIZE-3];
} TUPLOADSDOSEGRESMBX;
```

图 22 –上传 SDO 段响应结构

上传 SDO 段响应的编码见表 38。

表 38 – 上传 SDO 段响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n >= 0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x03: SDO Response
SDO	More Follows	Unsigned1	0x00: 后面还有 Upload SDO Segment 0x01: 最后一个 Upload SDO Segment
	SegData Size	Unsigned3	定义了最后 7 个数据八位位组 (总被发送) 包含多少数据 (只适用于 Length 为 0x0a, 否则为 0x 00) 0x00: 7 个八位位组数据 0x01: 6 个八位位组数据 0x02: 5 个八位位组数据 0x03: 4 个八位位组数据 0x04: 3 个八位位组数据 0x05: 2 个八位位组数据 0x06: 1 个八位位组数据 0x07: 0 个八位位组数据
	Toggle	Unsigned1	与相应的 Upload SDO Segment 请求相同。
	Command specifier	Unsigned3	0x00: 上传段的响应
	Data	BYTE[n-3]	对象的数据部分

5.6.2.8 中止 SDO 传输 (Abort SDO Transfer)

5.6.2.8.1 中止 SDO 传输请求

图 23 描述中止 SDO 传输请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TINITSDOHEADER  SdoHeader;
    DWORD           AbortCode;
} TABORTSDOTTRANSFERREQMBX;
```

图 23 – 中止 SDO 传输请求结构

中止 SDO 传输请求的编码见表 39。

表 39 –中止 SDO 传输请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户端, 这里指源站地址; 如果从站是客户端, 这里指目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x02: SDO 请求
SDO	Size Indicator	Unsigned1	0x00
	Transfer Type	Unsigned1	0x00
	Data Set Size	Unsigned2	0x00
	Reserved	Unsigned1	0x00
	Command Specifier	Unsigned3	0x04: 中止传输请求
	Index	WORD	对象的索引
	Subindex	BYTE	对象的子索引
	Abort Code	DWORD	中止代码见表 40

5.6.2.8.2 SDO 中止代码

SDO 中止代码见表 40。

表 40 –SDO 中止代码

值	含义
0x05 03 00 00	Toggle 位不变化
0x05 04 00 00	SDO 协议超时
0x05 04 00 01	客户机/服务器命令限定符无效或未知
0x05 04 00 05	内存溢出
0x06 01 00 00	不支持的对象访问
0x06 01 00 01	试图读取一个只写对象
0x06 01 00 02	试图写入一个只读对象
0x06 02 00 00	该对象在对象目录中不存在
0x06 04 00 41	该对象不能被映射到 PDO
0x06 04 00 42	被映射的对象的数量和长度将超过 PDO 长度
0x06 04 00 43	一般参数不兼容的原因
0x06 04 00 47	在设备中一般的内部不兼容性
0x06 06 00 00	由于硬件错误而导致访问失败
0x06 07 00 10	数据类型不匹配, 服务参数的长度不匹配。
0x06 07 00 12	数据类型不匹配, 服务参数的长度太长。
0x06 07 00 13	数据类型不匹配, 服务参数的长度太短。
0x06 09 00 11	子索引不存在
0x06 09 00 30	参数值超出范围 (只对写访问)
0x06 09 00 31	写入的参数值太大
0x06 09 00 32	写入的参数值太小
0x06 09 00 36	最大值小于最小值
0x08 00 00 00	一般错误
0x08 00 00 20	数据不能被传输或存储到应用
0x08 00 00 21	由于本地控制, 数据不能被传输或存储到应用
0x08 00 00 22	由于目前设备状态, 数据不能被传输或存储到应用程序
0x08 00 00 23	对象字典动态生成失败或当前没有对象字典

5.6.3 SDO 信息

5.6.3.1 概述

在以下服务中, 在“SDO Info Service Data”部分中的所有内容应作为一个块处理, 并只发送一次。

如果服务被分段, 那么最后一个分段的长度可以小于等于 0x0A。

5.6.3.2 SDO 信息 (SDO Information) 服务

图 24 描述 SDO 信息服务的属性类型。

```
typedef struct
{
    unsigned      OpCode:          7;
    unsigned      InComplete:      1;
    unsigned      Reserved:        8;
    WORD          FragmentsLeft;
} TSDOINFOHEADER;
```

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOINFOHEADER  SdoInfoHeader;
} TSDOINFOSERVICE;
```

图 24 – SDO 信息服务结构

SDO 信息服务的编码见表 41。

表 41 – SDO 信息服务

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n > 0x06: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
			0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x01: Get OD List 请求 0x02: Get OD List 响应 0x03: Get Object Description 请求 0x04: Get Object Description 响应 0x05: Get Object Description 请求 0x06: Get Object Description 响应 0x07: SDO Info Error 请求
	Incomplete	Unsigned1	0x00: 最后一个 SDO Information 分段 0x01: 后面还有 SDO Information 分段
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	后面分段的数量
SDO Info Service Data	Data	BYTE[n-6]	SDO 信息服务数据

5.6.3.3 获取 OD 列表 (Get OD List)

5.6.3.3.1 获取 OD 列表请求

表 25 描述获取 OD 列表请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOINFOHEADER  SdoInfoHeader;
```

```
WORD ListType;  
} TGETODLISTREQ;
```

图 25 – Get OD 列表请求结构

获取 OD 列表请求的编码见表 42。

表 42 – 获取 OD 列表请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x08: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x01: Get OD List 请求
	Incomplete	Unsigned1	应为 0
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	应为 0
SDO Info Service Data	List Type	WORD	0x00: 在 5 个不同的列表中得到对象的数量 0x01: 对象字典中的所有对象, 应在响应中传递 0x02: 只有在 RxPDO 中可映射的对象在响应中传递 0x03: 只有在 TxPDO 中可映射的对象在响应中传递 0x04: 只有存储用于设备替换的对象在响应中传递 0x05: 只有用作启动参数的对象在响应中传递

5.6.3.3.2 获取 OD 列表响应

图 26 描述获取 OD 列表响应的属性类型。

```
typedef struct  
{  
    TMBXHEADER MbxHeader;  
    TCOEHEADER CoeHeader;  
    TSDOINFOHEADER SdoInfoHeader;  
    WORD ListType;  
} TGETODLISTRES;
```

图 26 – 获取 OD 列表响应结构

获取 OD 列表响应的编码见表 43。

表 43 – 获取 OD 列表响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n >= 0x08: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x02: Get OD List 响应
	Incomplete	Unsigned1	0x00: 最后一个 SDO Information 分段。 0x01: 后面还有 SDO Information 分段。
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	后面分段的数量
SDO Info Service Data	List Type	WORD	0x00: 列表的长度应在响应中传递 0x01: 对象字典中的所有对象, 应在响应中传递 0x02: 只有 RxPDO 中可映射的对象在响应中传递 0x03: 只有 TxPDO 中可映射的对象在响应中传递 0x04: 只有存储用于设备替换的对象在响应中传递 0x05: 只有用作启动参数的对象在响应中传递
	Index List	WORD [(n-8)/2]	对象索引的列表 如果列表类型为 0, 那么用 5 个 WORD 来分别表示每个列表类型的长度

5.6.3.4 OD 列表段 (OD List Segment)

如果 Get OD 列表响应的 SDO 服务数据信息必须被分段, 那么 SDO 信息服务数据被分段, OD 列表段服务应传输这些段。

5.6.3.5 获取对象描述 (Get Object Description)

5.6.3.5.1 获取对象描述请求

图 27 描述获取对象描述请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOINFOHEADER  SdoInfoHeader;
    WORD             Index;
```

```
} TGETOBJDESCREQ;
```

图 27 – 获取对象描述请求结构

获取对象描述请求的编码见表 44。

表 44 – 获取对象描述请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x08: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x03: Get Object Description 请求
	Incomplete	Unsigned1	应为 0
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	应为 0
SDO Info Service Data	Index	WORD	请求对象描述的索引

5.6.3.5.2 获取对象描述响应

图 28 描述 Get Object Description 响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOINFOHEADER  SdoInfoHeader;
    WORD             Index;
    WORD             DataType;
    BYTE             MaxSubindex;
    BYTE             ObjCode;
} TGETOBJDESCRES;
```

图 28 – 获取对象描述响应结构

获取对象描述响应的编码见表 45。

表 45 – 获取对象描述响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n > 0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
			0x00
	Service	Unsigned4	0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x04: Get Object Description 响应
	Incomplete	Unsigned1	0x00: 最后一个 SDO Information 段
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	后面的段数量
SDO Info Service Data	Index	WORD	对象描述的索引
	Data Type	WORD	数据类型列表的引用
	Max Subindex	BYTE	对象的最大子索引
	Object Code	BYTE	对象代码 7: Variable 变量 8: Array 数组 9: Record 记录
	Name	char[n-12]	对象名称

5.6.3.6 获取条目描述 (Get Entry Description)

5.6.3.6.1 获取条目描述请求

图 29 描述 Get Entry Description 请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOINFOHEADER  SdoInfoHeader;
    WORD            Index;
    BYTE            Subindex;
    BYTE            ValueInfo;
} TGETENTRYDESCREQ;
```

图 29 – 获取条目描述请求的结构

获取条目描述请求的编码见表 46。

表 46 – 获取条目描述请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x05: Get Entry Description 请求
	Incomplete	Unsigned1	应为 0
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	应为 0
SDO Info Service Data	Index	WORD	被请求对象描述的索引
	Subindex	BYTE	被请求对象描述的子索引
	ValueInfo	BYTE	在响应中该 ValueInfo 应包含哪些元素 Bit 0: 保留 Bit 1: 保留 Bit 2: 保留 Bit 3: 单元类型 Bit 4: 默认值 Bit 5: 最小值 Bit 6: 最大值

5.6.3.6.2 获取条目描述响应

图 30 描述获取条目描述响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TCOEHEADER      CoeHeader;
    TSDOINFOHEADER  SdoInfoHeader;
    WORD            Index;
    BYTE            Subindex;
    BYTE            ValueInfo;
    WORD            DataType;
    WORD            BitLength;
    WORD            ObjAccess;
} TGETENTRYDESCRES;
```

图 30 – 获取条目描述响应结构

获取条目描述响应的编码见表 47。

表 47 – 获取条目描述响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n >= 0x10: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x06: Get Entry Description 响应
	Incomplete	Unsigned1	0x00: 最后一个 SDO Information 段 0x01: 后面还有 SDO Information 段
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	后面的段数量
SDO Info Service Data	Index	WORD	请求的对象描述的索引
	Subindex	BYTE	请求的对象描述的子索引
	ValueInfo	BYTE	在响应中 ValueInfo 包含哪些元素 Bit 0: 保留 Bit 1: 保留 Bit 2: 保留 Bit 3: 单元类型 Bit 4: 默认值 Bit 5: 最小值 Bit 6: 最大值
	Data Type	WORD	对象数据类型的索引
	Bit Length	WORD	对象的位长度 如果长度为 0xFFFF, 对象长度大于 64Kbit。为了得到对象的长度, 上传该对象
	Object Access	WORD	Bit 0: Pre-Operational 状态下的读访问 Bit 1: Safe-Operational 状态下的读访问 Bit 2: Operational 状态下的读访问 Bit 3: Pre-Operational 状态下的写访问 Bit 4: Safe-Operational 状态下的写访问 Bit 5: Operational 状态下的写访问 Bit 6: 在 RxPDO 中对象可映射 Bit 7: 在 TxPDO 中对象可映射 Bit 8: 对象可备份 Bit 9: 在 RxPDO 对象可用于设置 Bit 10-15: 保留

帧部分	数据字段	数据类型	值/描述
	Data	BYTE[n-16]	如果单位类型包含在响应中, 则随后是对象的单位类型 (DWORD) 如果默认值包含在响应中, 则随后是对象条目的默认值 (相同数据类型的对象值) 如果最小值包含在响应中, 则随后是对象条目的最小值 (相同数据类型的对象值) 如果最大值包含在响应中, 则随后是对象条目的最大值 (相同数据类型的对象值) 如果长度大于所描述的响应参数, 则随后是描述 (字符数组)

5.6.3.7 条目描述段 (Entry Description Segment)

条目描述段 (Entry Description Segment) 与获取条目描述 (Get Entry Description) 响应的属性类型和编码相同。

5.6.3.8 SDO 信息错误 (SDO Info Error)

图 31 描述了 SDO 信息错误请求的属性类型。

```
typedef struct
{
    TMBXHEADER          MbxHeader;
    TCOEHEADER          CoeHeader;
    TSDOINFOHEADER      SdoInfoHeader;
    DWORD               AbortCode;
} TABORTSDOTRANSFERREQMBX;
```

图 31 – SDO 信息错误请求结构

SDO 信息错误请求的编码见表 48。

表 48 – SDO 信息错误请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x08: SDO Information
SDO Info Header	Opcode	Unsigned7	0x07: SDO 信息错误请求
	Incomplete	Unsigned1	应为 0
	Reserved	Unsigned8	0x00
	Fragments Left	WORD	应为 0
	Abort Code	DWORD	中止代码见表 40

5.6.4 紧急 (Emergency)

5.6.4.1 紧急请求

表 49 定义了紧急请求的编码。

表 49 – 紧急请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n = 0x0A: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	0x00
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x01: Emergency
Emergency	Error Code	WORD	错误代码
	Error Register	BYTE	错误寄存器
	Data	BYTE[5]	错误代码 0000-9FFF: 制造商特定错误字段 错误代码 A000-EFFF: 诊断数据 错误代码 F000-FFFF: 制造商特定错误字段
	Reserved	BYTE[n-10]	

5.6.4.2 紧急错误代码 (Emergency Error Code)

紧急错误代码见表 50。

表 50 – 紧急错误代码

错误代码 (十六进制)	含义
00xx	错误复位或无错误
10xx	一般错误
20xx	电流
21xx	电流, 设备输入侧
22xx	电流, 设备内侧
23xx	电流, 设备输出侧
30xx	电压
31xx	干线电压
32xx	设备内部电压
33xx	输出电压
40xx	温度
41xx	环境温度
42xx	设备温度
50xx	设备硬件
60xx	设备软件
61xx	内部软件
62xx	用户软件
63xx	数据集
70xx	附加模块
80xx	监测
81xx	通信
82xx	协议错误
8210	因长度错误而不处理 PDO
8220	超出 PDO 长度
90xx	外部错误
A0xx	ESM 转换错误
F0xx	附加功能
FFxx	设备特定

5.6.4.3 ESM 转换错误

5.6.4.3.1 错误代码

ESM 转换错误代码见表 51。

表 51 – 错误代码

错误代码(十六进制)	含义
A000	PRE-OPERATIONAL 到 SAFE-OPERATIONAL 的转换不成功
A001	SAFE-OPERATIONAL 到 OPERATIONAL 的转换不成功

5.6.4.3.2 诊断数据

ESM 转换诊断数据结构见表 52。

表 52 – 诊断数据

数据[0]	数据[1..4]	含义
0x00 + channel*4	Sync Manager Length Error	同步管理器通道的长度不匹配
0x01 + channel*4	Sync Manager Address Error	同步管理器通道的物理起始地址不匹配
0x02 + channel*4	Sync Manager Settings Error	同步管理器通道的设置不匹配

5.6.4.3.3 同步管理器长度错误

同步管理器长度错误的编码见表 53。

表 53 – 同步管理器长度错误

数据 [1..4]	数据类型	值/描述
Minimum Length	WORD	同步管理器通道的长度参数的最小值
Maximum Length	WORD	同步管理器通道的长度参数的最大值

5.6.4.3.4 同步管理器地址错误

同步管理器地址错误的编码见表 54。

表 54 – 同步管理器地址错误

数据 [1..4]	数据类型	值/描述
Minimum Address	WORD	同步管理器通道的物理起始地址参数的最小值
Maximum Address	WORD	同步管理器通道的物理起始地址参数的最大值

5.6.4.3.5 同步管理器设置错误

同步管理器设置错误的编码见表 55。

表 55 – 同步管理器设置错误

数据 [1..4]	数据类型	值/描述
Expected Buffer Type	Unsigned2	同步管理器通道的缓冲区类型参数的预期值
Expected Direction	Unsigned2	同步管理器通道的方向参数的预期值
Reserved	Unsigned1	0x00 (为将来使用保留)
Expected AL Event Enable	Unsigned1	同步管理器通道的 AL 事件使能 (AL Event Enable) 参数的期望值
Reserved	Unsigned10	0x00 (为将来使用保留)
Expected Channel Enable	Unsigned1	同步管理通道的通道使能 (Channel Enable) 参数的预期值
Reserved	Unsigned15	0x00 (为将来使用保留)

5.6.5 过程数据

5.6.5.1 RxPDO

通过邮箱进行 RxPDO 传输的协议见表 56。

表 56 – 通过邮箱进行 RxPDO 传输

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n = 0x02: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	相关的 RxPDO 号
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x05: RxPDO
PDO	Data	BYTE[n-2]	过程输出数据

5.6.5.2 TxPDO

通过邮箱进行 TxPDO 传输的编码见表 57。

表 57 – 通过邮箱进行 TxPDO 传输

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	n = 0x02: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	相关的 TxPDO 号
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x04: TxPDO
PDO	Data	BYTE[n-2]	过程输入数据

5.6.5.3 RxPDO 远程传输请求

RxPDO 远程传输请求的代码见表 58。

表 58 – RxPDO 远程传输请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x02: 邮箱服务数据的长度
	Address	WORD	如果主站是客户端, 这里指源站地址; 如果从站是客户端, 这里指目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	相关的 RxPDO 号
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x07: RxPDO Remote Transmission Request

5.6.5.4 TxPDO 远程传输请求

TxPDO 远程传输请求的代码见表 59。

表 59 – TxPDO 远程传输请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x02: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x03: CoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 起始值, 7 后面的值是 1)
	Reserved	Unsigned1	0x00
CoE Header	Number	Unsigned9	相关的 TxPDO 号
	Reserved	Unsigned3	0x00
	Service	Unsigned4	0x06: TxPDO Remote Transmission Request

5.6.6 命令

命令对象结构见表 60。每个命令对象应具有数据类型 0x0025。声明为命令对象的任何对象可使用该结构。

表 60 – 命令对象结构

子索引	描述	数据类型	值/描述
0	Number of entries	UNSIGNED8	3
1	Command	OCTET_STRING	字节 0-n: 请求数据 对命令数据的写访问将执行命令
2	Status	UNSIGNED8	0: 上一个命令完成, 无错误, 无回复 1: 上一个命令完成, 无错误, 有回复 2: 上一个命令完成, 有错误, 无回复 3: 上一个命令完成, 有错误, 有回复 4-99: 为将来使用保留 100-200: 表示命令执行的程度 (用%表示, 100 = 0 %, 200 = 100 %) 201-254: 为将来使用保留 255: 正在执行命令 (如果不支持百分比显示)
3	Reply	OCTET-STRING	字节 0-n: 响应数据

5.6.7 对象字典

5.6.7.1 对象字典结构

对象字典的结构见表 61。

表 61 – 对象字典结构

索引(十六进制)	对象字典区
0x0000-0x0FFF	数据类型区
0x1000-0x1FFF	CoE 通信区
0x2000-0x5FFF	制造商特定区
0x6000-0xFFFF	行规区

5.6.7.2 对象代码定义 (Object Code Definition)

对象代码定义条目的结构见表 62。

表 62 – 对象代码定义

对象代码	对象名称
0002	DOMAIN
0005	DEFTYPE
0006	DEFSTRUCT
0007	VAR
0008	ARRAY
0009	RECORD

5.6.7.3 数据类型区域

基本数据类型区见表 63。

表 63 – 基本数据类型区

索引(十六进制)	对象类型	名称
0001	DEFTYPE	BOOLEAN
0002	DEFTYPE	INTEGER8
0003	DEFTYPE	INTEGER16
0004	DEFTYPE	INTEGER32
0005	DEFTYPE	UNSIGNED8
0006	DEFTYPE	UNSIGNED16
0007	DEFTYPE	UNSIGNED32
0008	DEFTYPE	REAL32
0009	DEFTYPE	VISIBLE_STRING
000A	DEFTYPE	OCTET_STRING
000B	DEFTYPE	UNICODE_STRING
000C	DEFTYPE	TIME_OF_DAY
000D	DEFTYPE	TIME_DIFFERENCE
000E		Reserved
000F	DEFTYPE	DOMAIN
0010	DEFTYPE	INTEGER24
0011	DEFTYPE	REAL64
0012	DEFTYPE	INTEGER40
0013	DEFTYPE	INTEGER48
0014	DEFTYPE	INTEGER56
0015	DEFTYPE	INTEGER64
0016	DEFTYPE	UNSIGNED24
0017		Reserved
0018	DEFTYPE	UNSIGNED40
0019	DEFTYPE	UNSIGNED48
001A	DEFTYPE	UNSIGNED56
001B	DEFTYPE	UNSIGNED64
001C-001F		reserved

扩展数据类型区的定义见表 64。

表 64 – 扩展数据类型区

索引(十六进制)	对象	名称
0020		保留
0021	DEFSTRUCT	PDO_MAPPING
0022		保留
0023	DEFSTRUCT	IDENTITY
0024		保留
0025	DEFSTRUCT	COMMAND_PAR
0026-0028		保留
0029	DEFSTRUCT	SYNC_PAR
002A-002F		保留
0030	DEFTYPE	BIT1
0031	DEFTYPE	BIT2
0032	DEFTYPE	BIT3
0033	DEFTYPE	BIT4
0034	DEFTYPE	BIT5
0035	DEFTYPE	BIT6
0036	DEFTYPE	BIT7
0037	DEFTYPE	BIT8
0038-003F		保留
0040-005F	DEFSTRUCT	制造商特定的复杂数据类型
0060-007F	DEFTYPE	设备行规 0 特定的标准数据类型
0080-009F	DEFSTRUCT	设备行规 0 特定的复杂数据类型
00A0-00BF	DEFTYPE	设备行规 1 特定的标准数据类型
00C0-00DF	DEFSTRUCT	设备行规 1 特定的复杂数据类型
00E0-00FF	DEFTYPE	设备行规 2 特定的标准数据类型
0100-011F	DEFSTRUCT	设备行规 2 特定的复杂数据类型
0120-013F	DEFTYPE	设备行规 3 特定的标准数据类型
0140-015F	DEFSTRUCT	设备行规 3 特定的复杂数据类型
0160-017F	DEFTYPE	设备行规 4 特定的标准数据类型
0180-019F	DEFSTRUCT	设备行规 4 特定的复杂数据类型
01A0-01BF	DEFTYPE	设备行规 5 特定的标准数据类型
01C0-01DF	DEFSTRUCT	设备行规 5 特定的复杂数据类型
01E0-01FF	DEFTYPE	设备行规 6 特定的标准数据类型
0200-021F	DEFSTRUCT	设备行规 6 特定的复杂数据类型
0220-023F	DEFTYPE	设备行规 7 特定的标准数据类型
0240-025F	DEFSTRUCT	设备行规 7 特定的复杂数据类型
0260-07FF		保留

枚举数据类型区占用了索引 0x800 到 0xFFFF。每个项都有一个规定了所占用位数的数据类型（如 BIT3 或 UNSIGNED16）和一个条目列表，该列表规定了整数值（数据类型是 unsigned32）和枚举可视字符串，如表 65 所示。

表 65 – 枚举定义

子索引	定义	数据类型	M/O/C	访问	PDO 映射	值
0	Number of entries	UNSIGNED8	O	R	No	枚举值 n 的值
	Padding	UNSIGNED8				0 填充并维持后面对象的偶数八位位组边界
1	Enum1	OCTET STRING	O	R	No	UNSIGNED32 为整数值 VISIBLE STRING 为枚举字符串
	...					
1	Enumn	OCTET STRING	O	R	No	UNSIGNED32 为整数值 VISIBLE STRING 为枚举字符串

5.6.7.4 CoE 通信区

表 66 给出了 CoE 通信对象字典区的元素。

表 66 – CoE 通信区

索引 (十六进制)	对象类型	名称	类型	M/O/C
1000	VAR	Device Type	UNSIGNED32	M
1001		Error Register	UNSIGNED8	O
1002		Reserved		
....				
1007		Reserved		
1008	VAR	Manufacturer Device Name	VisibleString	O
1009	VAR	Manufacturer Hardware Version	VisibleString	O
100A	VAR	Manufacturer Software Version	VisibleString	O
100B		Reserved		
....				
1017		Reserved		
1018	RECORD	Identity Object	Identity (0x23)	M
1019		Reserved		
....				
15FF		Reserved		
1600	RECORD	1 st receive PDO Mapping	PDO Mapping (0x21)	C
1601	RECORD	2 nd receive PDO Mapping	PDO Mapping	C
....				
17FF	RECORD	512 th receive PDO Mapping	PDO Mapping	C
1800		Reserved		
....				
19FF		Reserved		
1A00	RECORD	1 st transmit PDO Mapping	PDO Mapping (0x21)	C
1A01	RECORD	2 nd transmit PDO Mapping	PDO Mapping	C

...	...	...	...	
1BFF	RECORD	512 th transmit PDO Mapping	PDO Mapping	C
1C00	ARRAY	Sync Manager Communication Type	UNSIGNED8	M
1C01		Reserved		
...	...	...	...	
1C0F		Reserved		
1C10	ARRAY	Sync Manager 0 PDO Assignment	UNSIGNED16	C
1C11	ARRAY	Sync Manager 1 PDO Assignment	UNSIGNED16	C
1C12	ARRAY	Sync Manager 2 PDO Assignment	UNSIGNED16	C
1C13	ARRAY	Sync Manager 3 PDO Assignment	UNSIGNED16	C
1C14	ARRAY	Sync Manager 4 PDO Assignment	UNSIGNED16	C
...	...	...	...	
1C2F	ARRAY	Sync Manager 31 PDO Assignment	UNSIGNED16	C
1C30	RECORD	Sync Manager 0 Synchronization		O
...	...	...	...	
1C4F	RECORD	Sync Manager 31 Synchronization		O
1C50		Reserved		
...	...	...	...	
1FFF		Reserved		

5.6.7.4.1 设备类型 (Device Type)

表 67 规定了 Device Type 对象字典条目 (0x1000)。

表 67 – Device Type

属性	值
Name	设备类型 (Device Type)
Object Code	VAR
Data Type	UNSIGNED32
Category	必备的
Access	Ro
PDO Mapping	No
Value	Bit 0-15: 使用的设备行规, 如果没有使用标准的设备行规, 则为 0x0000 Bit 16-31: 与使用的设备行规相关的附加信息

5.6.7.4.2 错误寄存器 (Error Register)

表 68 规定 Error Register 对象字典条目 (0x1001)。

表 68 – Error Register

对象	值
Name	Error Register
Object Code	VAR
Data Type	UNSIGNED8
Category	可选的
Access	R0
PDO Mapping	
Value	Bit 0: 一般错误 Bit 1: 电流错误 Bit 2: 电压错误 Bit 3: 温度错误 Bit 4: 通信错误 Bit 5: 设备行规特定错误 Bit 6: 保留 Bit 7: 制造商特定错误

5.6.7.4.3 制造商设备名称 (Manufacturer Device Name)

表 69 规定 Manufacturer Device Name 对象字典条目 (0x1008)。

表 69 – Manufacturer Device Name

属性	值
Name	Manufacturer Device Name
Object Code	VAR
Data Type	VISIBLE_STRING
Category	可选的
Access	R0
PDO Mapping	No
Value	设备的名字, 用非零终止的字符串表示

5.6.7.4.4 制造商硬件版本 (Manufacturer Hardware Version)

表 70 规定 Manufacturer Hardware Version 对象字典条目 (0x1009)。

表 70 – Manufacturer Hardware Version

属性	值
Name	Manufacturer Hardware Version
Object Code	VAR
Data Type	VISIBLE_STRING
Category	Optional
Access	R0
PDO Mapping	No
Value	设备的硬件版本, 用非零终止的字符串表示

5.6.7.4.5 制造商软件版本 (Manufacturer Software Version)

表 71 规定 Manufacturer Software Version 对象字典条目 (0x100A)。

表 71 – Manufacturer Software Version

属性	值
Name	Manufacturer Software Version
Object Code	VAR
Data Type	VISIBLE_STRING
Category	可选的
Access	R
PDO Mapping	No
Value	设备的软件版本, 用非零终止的字符串表示

5.6.7.4.6 标识对象 (Identity Object)

表 72 规定 Identity Object 字典条目 (索引 0x1018)。

表 72 – Identity Object

子索引	描述	数据类型	M/O/C	访问	PDO 映射	值
0	条目个数	UNSIGNED8	M	R	No	4
1	制造商 ID	UNSIGNED32	M	R	No	由 ETG 分配的惟一值
2	产品代码	UNSIGNED32	M	R	No	由制造商分配的惟一值
3	版本号	UNSIGNED32	M	R	No	由制造商分配的惟一值
4	序列号	UNSIGNED32	M	R	No	由制造商为这个设备分配的惟一值 如果没有给出序列号, 则为 0

5.6.7.4.7 接收 PDO 映射 (Receive PDO Mapping)

表 73 规定 Receive PDO Mapping 对象字典条目 (索引 0x1600 – 0x17FF)。

表 73 – Receive PDO Mapping

子索引	描述	数据类型	M/O/C	访问	PDO 映射	值
0	该 PDO 中的对象个数	UNSIGNED8	M	R/RW (注)	No	0 – 254 如果支持可变映射, 则可写
1	被映射的第一个输出对象	UNSIGNED32	C	R/RW (注)	No	Bit 0-7: 以比特为单位的被映射对象的长度(对于 PDO 中的间隙:应具有间隙的比特长度) Bit 8-15: 被映射对象的子索引 (PDO 中有间隙时为 0) Bit 16-31: 被映射对象的索引 (对于 PDO 中的间隙应为 0)
..						
n	被映射的最后一个输出对象	UNSIGNED32	C	R/RW (注)	No	
a 如果支持可变 PDO, 则可写。						

5.6.7.4.8 发送 PDO 映射 (Transmit PDO Mapping)

表 74 规定 Transmit PDO Mapping 对象字典条目 (索引 0x1A00 – 0x1BFF)。

表 74 – Transmit PDO Mapping

子索引	描述	数据类型	M/O/C	访问	PDO 映射	值
0	该 PDO 中的对象个数	UNSIGNED8	M	R/RW ^a	No	0 – 254 如果支持可变映射, 则可写
1	被映射的第一个输出对象	UNSIGNED32	C	R/RW ^a	No	Bit 0-7: 以比特为单位的映射对象的长度(PDO 中的间隙:应具有间隙的比特长度) Bit 8-15: 映射对象的子索引 (对于 PDO 中的间隙应为 0) Bit 16-31: 映射对象的索引 (对于 PDO 中的间隙应为 0)
..						
n	被映射的最后一个输出对象	UNSIGNED32	C	R/RW ^a	No	
a 如果支持可变 PDO 映射, 则可写。						

5.6.7.4.9 同步管理器通信类型 (Sync Manager Communication Type)

表 75 规定 Sync Manager Communication Type 对象字典条目 (索引 0x1C00)。

表 75 – Sync Manager Communication Type

子索引	描述	数据类型	M/O/C	访问	PDO 映射	值
0	使用的同步管理器通道个数	UNSIGNED8	M	R	No	4 ~ 32
1	通信类型同步管理器 0	UNSIGNED8	C	R	No	可配置为 ^a : 0: 未使用 1: 邮箱接收 (主站到从站) 2: 邮箱发送 (从站到主站) 3: 过程数据输出 4: 过程数据输入(从站到主站)
2	通信类型同步管理器 1	UNSIGNED8	C	R	No	可配置为 ^a : 0: 未使用 1: 邮箱接收 (主站到从站) 2: 邮箱发送 (从站到主站) 3: 过程数据输出 4: 过程数据输入(从站到主站)
3	通信类型同步管理器 2	UNSIGNED8	C	R	No	可配置为 ^a : 0: 未使用 1: 邮箱接收 (主站到从站) 2: 邮箱发送 (从站到主站) 3: 过程数据输出 4: 过程数据输入(从站到主站)
4	通信类型同步管理器 3	UNSIGNED8	C	R	No	可配置为 ^a : 0: 未使用 1: 邮箱接收 (主站到从站) 2: 邮箱发送 (从站到主站) 3: 过程数据输出 4: 过程数据输入(从站到主站)
5 – n	通信类型同步管理器 4 – (n-1)	UNSIGNED8	C	R	No	可配置为 ^a : 0: 未使用 1: 邮箱接收 (主站到从站) 2: 邮箱发送 (从站到主站) 3: 过程数据输出 4: 过程数据输入(从站到主站)
a Sync Manager communication type 可以用于以下方式: Communication Type Sync Manager 0: 1 邮箱接收 Communication Type Sync Manager 1: 2 邮箱发送 Communication Type Sync Manager 2: 3 过程数据输出(如果不支持输出, 可用于输入) Communication Type Sync Manager 3: 4 过程数据输入 如果不支持邮箱, 则可用以下方式: Communication Type Sync Manager 0: 3 过程数据输出(如果不支持输出, 可用于输入) Communication Type Sync Manager 1: 4 过程数据输入						

5.6.7.4.10 同步管理器 PDO 分配 (Sync Manager PDO Assignment)

5.6.7.4.10.1 同步管理器通道

表 76 规定 Sync Manager Channel 0~31 对象字典条目 (索引 0x1C10-0x1C2F)。如果 Sync Manager Channel 被用作邮箱同步管理器, 则相应的对象将不可用, 或子索引 0 应为 0。

表 76 – Sync Manager Channel 0~31

子索引	描述	数据类型	M/O/C	访问	PDO 映射	值
0	分配的 TxPDO 个数	UNSIGNED8	C	R/RW (注)	No	0 – 254
1 – n	分配的 PDO 的 PDO 映射对象索引	UNSIGNED16	C	R/RW (注)	No	下列值之一: 0x1600: RxPDO 1 0x1601: RxPDO 2 ... 0x17FF: RxPDO 512 或: 0x1A00: TxPDO 1 0x1A01: TxPDO 2 ... 0x1BFF: TxPDO 512
(注) a 如果支持可变 PDO, 则可写。						

5.6.7.4.11 同步管理器同步 (Sync Manager Synchronization)

表 77 定义 Sync Manager Synchronization 对象字典条目 (索引 0x1C30~0x1C4F)。

表 77 – Sync Manager Synchronization

子索引	描述	数据类型	M/O/C	访问	PDO 映射	值
0	同步参数的个数	UNSIGNED8	O	R	No	1 ~ 3
1	同步类型	UNSIGNED16	O	RW	No	• 0: 不同步 • 1: Synchron – 与同步管理器的 AL 事件同步 • 2: DC Sync0 – 与 AL 事件 Sync0 同步 • 3: DC Sync1 – 与 AL 事件 Sync1 同步 • 32: SyncSm0 – 与 SM0 的 AL 事件同步 • 33: SyncSm1 – 与 SM1 的 AL 事件同步 • ... • 63: SyncSm31 – 与 SM31 的 AL 事件同步
2	周期	UNSIGNED32	O	RW	No	两个事件之间的时间 (单位: ns)
3	便宜时间	UNSIGNED32	O	RW	No	相关 AL 事件和关联的动作之间的时间 (单位: ns)

5.7 EoE 编码

5.7.1 Initiate EoE

5.7.1.1 Initiate EoE 请求

图 32 描述了 Initiate EoE 请求的属性类型。

```
typedef struct
{
    unsigned    FrameType:        4;
    unsigned    Port:             4;
    unsigned    LastFragment:     1;
    unsigned    TimeAppended:     1;
    unsigned    TimeRequested:    1;
    unsigned    Reserved:         5;
    unsigned    FragmentNumber:   6;
    unsigned    CompleteSize:     6;
    unsigned    FrameNumber:      4;
} TEOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TEOEHEADER    EoeHeader;
    BYTE          Data[MAX_EOE_DATA_SIZE];
} TINIEOEREQ;
```

图 32 – EoE 一般结构

表 78 规定 Initiate EoE 请求的编码。

表 78 – Initiate EoE 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	$N = 0x24 + y \cdot 0x20$: 邮箱服务数据的长度 ($y=0 \sim 0x2F$)
	Address	WORD	如果主站是客户机, 则为源站地址; 如果从站是客户机, 则为目的站地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x02: EoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
EoE Header	FrameType	Unsigned4	0x02
	Port	Unsigned4	0x00: 发送到非特定端口 0x01-0x0F: 选择特定端口
	Last Fragment	Unsigned1	0x00: 后面至少还有一个 EoE Fragment 服务 0x01: 整个以太网帧都位于数据部分
	Time Appended	Unsigned1	0x00: 最后一个分段中的 EoE 数据之后不添加时间戳 0x01: 最后一个分段中的 EoE 数据之后添加时间戳
	Time Request	Unsigned1	0x00: 不要求发送时间的时间戳 0x01: 要求有发送时间的时间戳
	Reserved	Unsigned5	
	Fragment Number	Unsigned6	0x00
	Complete Size	Unsigned6	(以太网数据帧的完整大小 + 31)/32
	Frame Number	Unsigned4	以太网帧的编号
	EoE Data	BYTE[N-4]	以太网帧 (不包括前导码、SFD、FCS) 第一部分, 长度为 (N-4) 个八位位组 (如果包含时间戳, 则再减去 4 个八位位组)
(optional)	TimeStamp	Unsigned32	以 ns 为单位, 从 DA 开始的帧接收时间

5.7.1.2 Initiate EoE 响应

图 33 描述了 Initiate EoE 响应的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TEOEHEADER      EoeHeader;
    TimeStamp        Unsigned32;
} TINIEOERES;
```

图 33 – EoE 时间戳结构

表 79 规定 Initiate EoE 响应的编码。

表 79 – Initiate EoE 响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N = 0x08: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源地址; 如果从站是客户机, 则为目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x02: EoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
EoE Header	FrameType	Unsigned4	0x03
	Port	Unsigned4	0x00: 发送到非特定端口 0x01-0x0F: 选择指定的端口
	Last Fragment	Unsigned1	0x00
	Time Appended	Unsigned1	0x01: 添加时间戳
	Time Request	Unsigned1	0x00: 不要求发送时间的时间戳
	Reserved	Unsigned5	
	Fragment Number	Unsigned6	0x00
	Complete Size	Unsigned6	0x00
	Frame Number	Unsigned4	以太网帧的编号
	TimeStamp	Unsigned32	以 ns 为单位, 从 DA 开始的数据帧发送时间

5.7.2 EoE Fragment 请求

图 34 描述了 EoE Fragment 请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TEOEHEADER      EoeHeader;
    BYTE            Data[MAX_EOE_DATA_SIZE];
} TEOEFRAGREQ;
```

图 34 – EoE Fragment 请求结构

表 80 规定 EoE Fragment 请求的编码。

表 80 – EoE Fragment 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N > 0x04: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源地址; 如果从站是客户机, 则为目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x02: EoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
EoE Header	FrameType	Unsigned4	0x00
	Port	Unsigned4	0x00: 发送到非特定端口 0x01-0x0F: 选择特定端口
	Last Fragment	Unsigned1	0x00: 至少还有一个 EoE Fragment 服务 0x01: 当前以太网数据帧的最后的的数据部分 (包括时间戳)
	Time Appended	Unsigned1	0x00: 最后一个分段中的 EoE 数据后不添加时间戳 0x01: 最后一个分段中的 EoE 数据后添加时间戳
	Time Request	Unsigned1	0x00: 不请求发送时间的时间戳 0x01: 请求有发送时间的时间戳
	Reserved	Unsigned5	
	Fragment Number	Unsigned6	0x01-0x2F: 以太网帧分段的个数
	Offset	Unsigned6	以太网帧分段的偏移量
	Frame Number	Unsigned4	以太网帧的编号
	EoE Data	BYTE[N-4]	以太网帧 (不包含前导码、SFD、FCS) 的第一部分, 长度为 (N-4) 个八位位组 (如果包含时间戳还要减去 4 个八位位组)
(optional)	TimeStamp	Unsigned32	以 ns 为单位, 从 DA 开始的数据帧接收时间

5.7.3 EoE 数据元素

表 81 规定 EoE 数据的编码。

表 81 – EoE 数据

帧部分	数据字段	数据类型	值/描述
Ethernet	Dest MAC	BYTE[6]	符合 ISO/IEC 8802-3 的目标 MAC 地址
	Src MAC	BYTE[6]	符合 ISO/IEC 8802-3 的源 MAC 地址
(optional)	VLAN Tag	BYTE[4]	0x81、0x00 与由 IEEE 802.1Q 规定的两个八位位组 标签控制信息 (Tag Control Information)
	Ether Type	BYTE[2]	由 IEEE 分配
User Frame	Data		用户数据 (八位位组串) 或者 EoE 参数
	Padding	BYTE[n]	根据 ISO/IEC 8802-3 的规定，如果 DLPDU 长度少于 64 个八位位组，需要补足 64 个八位位组。

5.7.4 Set IP Parameter

5.7.4.1 Set IP Parameter 请求

图 35 描述了 Set IP Parameter 请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TEOEHEADER      EoeHeader;
    BYTE             Data[MAX_EOE_DATA_SIZE];
} TEOEFRAGREQ;
```

图 35 – Set IP Parameter 请求结构

表 82 规定 Set IP Parameter 请求的编码。

表 82 – Set IP Parameter 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N > 0x08: 邮箱服务数据长度
	Address	WORD	如果主站是客户机, 则为源地址; 如果从站是客户机, 则为目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x02: EoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
EoE Header	FrameType	Unsigned4	0x02
	Port	Unsigned4	0x00: 发送到非特定端口 0x01-0x0F: 选择特定的端口
	Last Fragment	Unsigned1	0x01: 最后的数据部分
	Time Appended	Unsigned1	0x00: 不添加时间戳
	Time Request	Unsigned1	0x00: 不请求发送时间的时间戳
	Reserved	Unsigned5	
	Fragment Number	Unsigned6	0x00
	Offset	Unsigned6	0x00
	Frame Number	Unsigned4	0x00
	Reserved	Unsigned1	
EoE Parameter	MAC included	Unsigned1	符合 ISO/IEC 8802-3 的 MAC 地址
	IP address included	Unsigned1	符合 IETF RFC 791 的 IP 地址
	Subnet Mask included	Unsigned1	符合 IETF RFC 791 的子网掩码
	Default Gateway included	Unsigned1	符合 IETF RFC 791 的默认网关地址
	DNS Server IP Address included	Unsigned1	符合 IETF RFC 791 的 DNS 服务器的 IP 地址
	DNS Name included	Unsigned1	符合 IETF RFC 791 的 DNS 名称
	reserved	Unsigned26	
	MAC	BYTE[6]	符合 ISO/IEC 8802-3 的 MAC 地址
	IP address	BYTE[4]	符合 IETF RFC 791 的 IP 地址
	Subnet Mask	BYTE[4]	符合 IETF RFC 791 和 IETF RFC 826 的子网掩码
(conditional)	Default Gateway	BYTE[4]	符合 IETF RFC 791 的默认网关地址
	DNS Server IP Address	BYTE[4]	符合 IETF RFC 791 的 DNS 服务器的 IP 地址
	DNS Name	char[32]	符合 IETF RFC 791 的 DNS 名称
	Reserved	Unsigned1	
	Reserved	Unsigned1	

5.7.4.2 Set IP Parameter 响应

图 36 描述了 Set IP Parameter 响应的属性类型。

```
typedef struct
{
    unsigned      FrameType:      4;
    unsigned      Port:           4;
    unsigned      LastFragment:   1;
    unsigned      TimeAppended:   1;
    unsigned      TimeRequested:  1;
    unsigned      Reserved:       5;
    unsigned      Result:         16;
} TEOEPARAHEADER;

typedef struct
{
    TMBXHEADER      MbxHeader;
    TEOEPARAHEADER  EoeHeader;
} TEOEFRAGREQ;
```

图 36 – Set IP Parameter 响应结构

表 83 规定 Set IP Parameter 响应的编码。

表 83 – Set IP Parameter 响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N=0X04: 邮箱服务数据长度
	Address	WORD	如果主站是客户机, 则为源地址; 如果从站是客户机, 则为目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x02: EoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
EoE Header	FrameType	Unsigned4	0x03
	Port	Unsigned4	0x00: 发送到非特定端口 0x01-0x0F: 选择特定的端口
	Last Fragment	Unsigned1	0x01: 最后的数据部分
	Time Appended	Unsigned1	0x00: 不添加时间戳
	Time Request	Unsigned1	0x00: 不要求发送时间的时间戳
	Reserved	Unsigned5	
	Result	Unsigned16	见表 84

表 84 – EoE Result 参数

结果代码	含义
0x0000	成功
0x0001	未定义的错误
0x0002	不支持的帧类型
0x0201	无 IP 支持
0x0401	无过滤器支持

5.7.5 Set Address Filter

5.7.5.1 Set Address Filter 请求

图 37 描述了 Set Address Filter 请求的属性类型。

```
typedef struct
{
    TMBXHEADER      MbxHeader;
    TEOEHEADER      EoeHeader;
    BYTE            Data[MAX_EOE_DATA_SIZE];
} TEOEFRAGREQ;
```

图 37 – Set Address Filter 请求结构

表 85 规定 Set Address Filter 请求的编码。

表 85 – Set Address Filter 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N > 0x06: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源地址; 如果从站是客户机, 则为目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x02: EoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
EoE Header	FrameType	Unsigned4	0x04
	Port	Unsigned4	0x00: 发送到非特定端口 0x01-0x0F: 选择特定的端口
	Last Fragment	Unsigned1	0x01: 最后的数据部分
	Time Appended	Unsigned1	0x00: 不添加时间戳
	Time Request	Unsigned1	0x00: 不要求发送时间的时间戳
	Reserved	Unsigned5	
	Fragment Number	Unsigned6	0x00
	Offset	Unsigned6	0x00
	Frame Number	Unsigned4	0x00
	Reserved	Unsigned1	符合 IETF RFC 791 的子网掩码
EoE Parameter	MAC filter count	Unsigned4	计数当前从站上以太网端口接受的 MAC 地址, MAC 地址须符合 ISO/IEC 8802-3。
	MAC filter mask	Unsigned2	计数当前从站上以太网端口接受的 MAC 地址掩码, MAC 地址须符合 ISO/IEC 8802-3。
	Inhibit Broadcast	Unsigned1	过滤广播报文
	Reserved	Unsigned8	
	List of MAC Address	List of BYTE[6]	符合 ISO/IEC 8802-3 的 MAC 地址。
	List of MAC Address Filter	List of BYTE[6]	符合 ISO/IEC 8802-3 的 MAC 地址。 比特被置位意味着目标 MAC 地址的该地址位与 MAC 地址列表中的相应条目进行比较
	Reserved	Unsigned8	

5.7.5.2 Set Address Filter 响应

图 38 描述了 Set Address Filter 响应的属性类型

```
typedef struct
{
    unsigned      FrameType:      4;
    unsigned      Port:           4;
    unsigned      LastFragment:   1;
    unsigned      TimeAppended:   1;
    unsigned      TimeRequested:  1;
    unsigned      Reserved:       5;
    unsigned      Result:         16;
} TEOEPARAHEADER;

typedef struct
{
    TMBXHEADER      MbxHeader;
    TEOEPARAHEADER  EoeHeader;
} TEOEFRAGREQ;
```

图 38 – Set Address Filter 响应结构

表 86 规定 Set Address Filter 响应的编码。

表 86 – Set Address Filter 响应

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N = 0x08: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源地址; 如果从站是客户机, 则为目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x02: EoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
EoE Header	FrameType	Unsigned4	0x05
	Port	Unsigned4	0x00: 发送到非特定端口 0x01-0x0F: 选择特定的端口
	Last Fragment	Unsigned1	0x01: 最后的数据部分
	Time Appended	Unsigned1	0x00: 不添加时间戳
	Time Request	Unsigned1	0x00: 不要求发送时间的时间戳
	Reserved	Unsigned5	
	Result	Unsigned16	见表 84

5.8 FoE 编码

5.8.1 FoE Read 请求

图 39 描述了 FoE Read 请求的属性类型。

```
typedef struct
{
    BYTE          OpCode;
    BYTE          Reserved;
} TFOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TFOEHEADER    FoeHeader;
    DWORD         Password;
    char          FileName[MAX_FILE_NAME_SIZE];
} TFOEREADREQ;
```

图 39 – FoE Read 请求结构

表 87 规定 FoE Read 请求的编码。

表 87 – FoE Read 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N > 0x06: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则为源地址; 如果从站是客户机, 则为目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x04: FoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
FoE Header	OpCode	BYTE	0x01: 读请求
	Reserved	BYTE	应为 0
Read Header	Password	DWORD	0: 不使用密码 1~0xFFFFFFFF: 密码
	File Name	char[n-6]	被读文件的名称

5.8.2 FoE Write 请求

图 40 描述了 FoE Write 请求的属性类型。

```
typedef struct
{
    BYTE          OpCode;
    BYTE          Reserved;
} TFOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TFOEHEADER    FoeHeader;
    DWORD         Password;
    char          FileName[MAX_FILE_NAME_SIZE];
} TFOEWRITEREQ;
```

图 40 – FoE Write 请求结构

表 88 规定 FoE Write 请求的编码。

表 88 – FoE Write 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N > 0x06: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则此地址是源地址, 如果从站是客户机的时, 则此地址是目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x04: FoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
FoE Header	OpCode	BYTE	0x02: 写请求
	Reserved	BYTE	应为 0
Write Header	Password	DWORD	0: 不使用密码 1~0xFFFFFFFF: 密码
	File Name	char[n-6]	被写文件的名称

5.8.3 FoE Data 请求

图 41 描述了 FoE Data 请求的属性类型。

```
typedef struct
{
    BYTE          OpCode;
    BYTE          Reserved;
} TFOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TFOEHEADER    FoeHeader;
    DWORD         PacketNo;
    BYTE          Data[MAX_DATA_SIZE];
} TFOEDATAREQ;
```

图 41 – FoE Data 请求结构

表 89 规定 FoE Data 请求的编码。

表 89 – FoE Data 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N > 0x06: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则此地址是源地址, 如果从站是客户机, 则此地址是目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x04: FoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
FoE Header	OpCode	BYTE	0x03: 数据请求
	Reserved	BYTE	应为 0
Data Header	Packet Number	DWORD	1~0xFFFFFFFF
	Data	BYTE[n-6]	文件数据

5.8.4 Ack 请求

图 42 描述了 Ack 请求的属性类型。

```
typedef struct
{
    BYTE          OpCode;
    BYTE          Reserved;
} TFOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TFOEHEADER    FoeHeader;
    DWORD         PacketNo;
} TFOEACKREQ;
```

图 42 – Ack 请求结构

表 90 规定 Ack 请求的编码。

表 90 – Ack 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	0x06: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则此地址是源地址, 如果从站是客户机, 则此地址是目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x04: FoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
FoE Header	OpCode	BYTE	0x04: 确认请求
	Reserved	BYTE	应为 0
Ack Header	Packet Number	DWORD	0: 写请求的确认 1~0xFFFFFFFF: 数据请求的确认

5.8.5 Error 请求

图 43 描述了 Error 请求的属性类型。

```
typedef struct
{
    BYTE          OpCode;
    BYTE          Reserved;
} TFOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TFOEHEADER    FoeHeader;
    DWORD         ErrorCode;
    char          ErrorText[MAX_ERROR_TEXT_SIZE];
} TFOEERRORREQ;
```

图 43 – Error 请求结构

表 91 规定 Error 请求的编码。

表 91 – Error 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N >= 0x06: 邮箱 服务数据的长度
	Address	WORD	如果主站是客户机, 则此地址是源地址, 如果从站是客户机, 则此地址是目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x04: FoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
FoE Header	OpCode	BYTE	0x05: 错误请求
	Reserved	BYTE	应为 0
Error Header	Error Code	DWORD	1~0xFFFFFFFF
	Error Text	char[n-6]	可选的错误描述

表 92 规定 Error 代码。

表 92 – Error 代码

错误代码	含义
0x8000	未定义
0x8001	没找到
0x8002	访问拒绝
0x8003	磁盘满
0x8004	非法
0x8005	数据包编号错误
0x8006	已经存在
0x8007	无用户
0x8008	仅引导
0x8009	不引导
0x800A	无权限
0x800B	程序错误

5.8.6 Busy 请求

图 44 描述了 Busy 请求的属性类型。

```
typedef struct
{
    BYTE          OpCode;
    BYTE          Reserved;
} TFOEHEADER;

typedef struct
{
    TMBXHEADER    MbxHeader;
    TFOEHEADER    FoeHeader;
    WORD          Done;
    WORD          Entire;
    char          BusyText[MAX_BUSY_TEXT_SIZE];
} TFOEBUSYREQ;
```

图 44 – Busy 请求结构

表 93 规定 Busy 请求的编码。

表 93 – Busy 请求

帧部分	数据字段	数据类型	值/描述
Mailbox Header	Length	WORD	N >= 0x06: 邮箱服务数据的长度
	Address	WORD	如果主站是客户机, 则此地址是源地址, 如果从站是客户机, 则此地址是目的地址
	Channel	Unsigned6	0x00 (为将来使用保留)
	Priority	Unsigned2	0x00: 最低优先级 ... 0x03: 最高优先级
	Type	Unsigned4	0x04: FoE
	Cnt	Unsigned3	邮箱服务计数器 (0 保留, 1 是起始值, 7 以后的下一个值为 1)
	Reserved	Unsigned1	0x00
FoE Header	OpCode	BYTE	0x06: Busy Request
	Reserved	BYTE	应为 0
Busy Header	Done	WORD	如果“Entire”等于 0, 则“Done”用百分比形式表示进度 (0~100%)。 如果“Entire”不等于 0, 则“Done”表示已传输数据的长度, 该数据与“Entire”具有相同的单位。
	Entire	WORD	如果“Entire”等于 0, 则“Done”用百分比形式表示文件传输进度 (0~100%) 如果“Entire”不等于 0, 则“Entire”表示指定单元中实际传输文件的长度, “Done”表示已传输数据的长度, 两个数据具有相同的单位 由这两个值可以得出百分比等式: $100 * \text{Done} / \text{Entire}$
	Busy Text	char[n-6]	可选的“忙”描述

6FAL 协议状态机

6.1 总体结构

6.1.1 概述

FAL 协议状态机结构如图 45 所示。

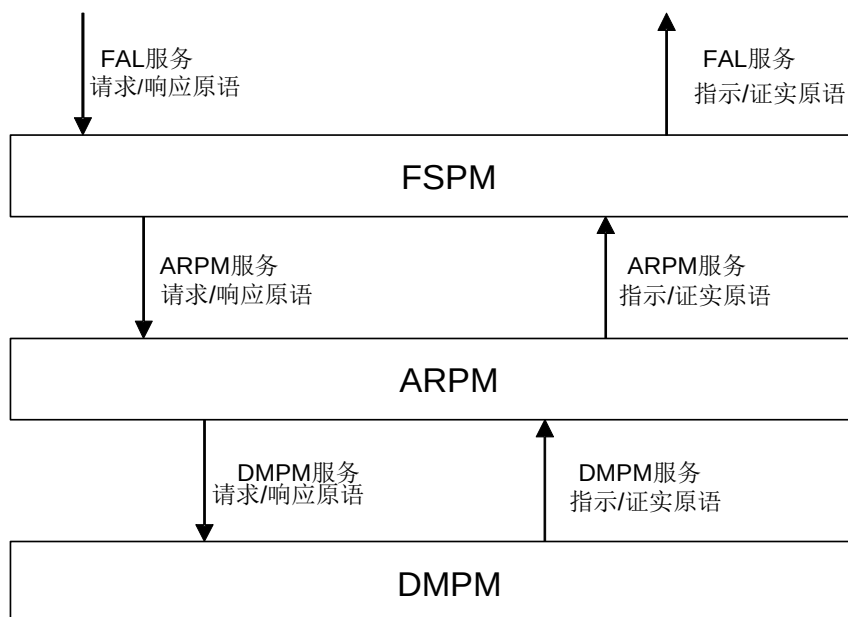


图 45 – 协议机间的关系

FAL 行为由三个集成的协议机规定。FSPM 是属于 FAL 类规范的 FAL 服务与特定的 AREP 之间的服务接口。

EtherCAT FAL 为从站提供一组协议机, 从而主站可以预知从站的行为。

FSPM 主要负责以下活动:

- 从 FAL 服务用户接受服务原语并将其转换为 FAL 内部原语。
- 根据隐式寻址机制选择 ARPM 状态机并将带服务参数 FAL 内部原语发送到 ARPM。
- 从 ARPM 接受 FAL 内部原语并将其转换为用于 FAL 服务用户的服务原语。
- 将 FAL 服务原语交付给 FAL 用户。

ARPM 为应用关系规定传输类型。

DMPM 规定到数据链路层的映射。因此, DMPM 定义了 LMPM 和 MAC 两种协议机。

6.1.2 现场总线服务协议机(Fieldbus Service Protocol Machines, FSPM)

FSPM 协调用于各种服务和应用关系的底层状态机。

FSPM 基本上是一个映射协议机。其主要任务是向负责相关服务的协议机传递该服务, 随后将证实以及响应转发给用户。另外, 该协议机中包含的基本冗余控制模式可使两个 AR 合并为一个具有更高可用性的单个实体。

6.1.3 应用关系协议机(Application Relationship Protocol Machines, ARPM)

ARPM 负责单个服务程序执行, 整体结构见图 46。过程数据交互直接由 DL 处理和 ESM 控制。应用有多种运行邮箱协议的方法。

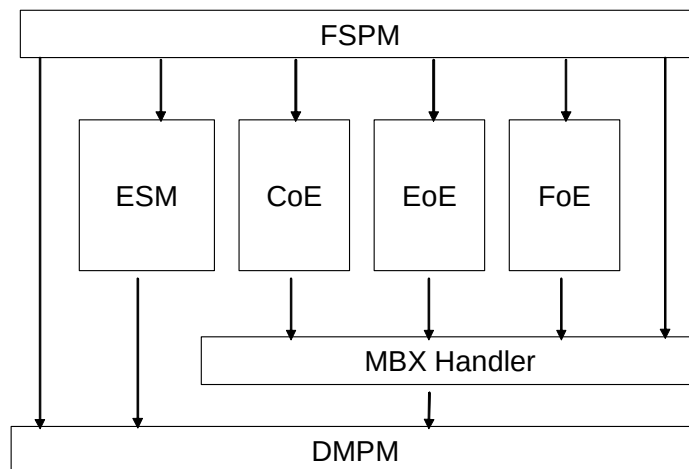


图 46 – AR 协议机

6.1.4 DLL 映射协议机(DLL Mapping Protocol Machines, DMPM)

DLL 映射协议机 (DMPM) 连接其他状态机与第二层。DMPM 对有关数据链路层用法的组态和错误处理的状态机进行协调。DMPM 将函数映射到第二层的 DLL 服务。DMPM 产生必要的第二层服务参数, 从第二层接收证实和指示, 并将其传递给适当的 DMPM 用户。

6.2 AP 上下关系状态机(AP-Context state machine)

在此协议里不定义 AP 上下关系状态机。

6.3 FAL 服务协议机(FAL service protocol machine, FSPM)

在 ETG.1000.5 中规定的服务直接被映射到 ARPM 的服务。

6.4 应用关系协议机(Application Relationship Protocol Machines, ARPMs)

6.4.1 AL 状态机

6.4.1.1 描述

ESM 负责在启动和工作期间协调主站与从站。状态的改变主要由主站与从站间的交互所导致。这些基本与写 AL 控制字有关。

在 DL 和 AL 初始化之后状态机进入“Init”状态。“Init”状态定义了应用层上主站与从站之间通信关系的初始点。在应用层上主站与从站之间无直接通信。主站使用“Init”状态来初始化一组配置寄存器。如果从站支持邮箱, 则相应的同步管理器配置也在“Init”状态进行。

如果从站支持可选邮箱并且邮箱已被设置, 则可进入“Pre-Operational”状态。主站和从站均可使用邮箱和适当的协议来交换应用特定的初始化信息和参数。在该状态下无过程数据通信。

如果从站支持输入且主站请求输入, 并且输入缓存区已被设置, 则可进入“Safe-Operational”状态。从站应用应传递实际输入数据而不处理输出数据, 从站的真正输出应被设置为处于安全状态。

如果输出缓存区已被设置且实际输出数据已被发送到从站（只要从站的输出将被使用），则可进入“Operational”状态。从站应用应传递实际输入数据，而主站应用应提供输出数据。

在可选“Bootstrap”状态，从站应用应能接受通过 FoE 协议下载永久性设置。

ESM 定义了四种应被支持的状态：

- Init,
- Pre-Operational,
- Safe-Operational,
- Operational。

除了以下例外，所有的状态转换都是可能的：从“Init”状态只可能转换到“Pre-Operational”状态；“Pre-Operational”状态不能直接转换到“Operational”状态。

通常，状态改变由主站请求。主站请求写 AL 控制寄存器，这会导致从站产生一个寄存器事件“AL 控制”指示。在一次成功的或失败的状态改变后，从站通过本地 AL 状态写服务来响应 AL 控制寄存器的改变。如果请求的状态改变失败，从站将错误标志置位。

EtherCAT 设备既可以是简单设备也可以是复杂设备，它们关于 AL 控制请求和 AL 控制响应的行为是不同的。当收到一个 AL 确认标志（Ack Flag）时，复杂从站将复位 AL 错误标志（Error Flag）（设备模拟无效），而简单从站将确认标志复制到 AL 错误标志（设备模拟有效）。

尽管行为不同，主站应能通过广播命令来初始化网络。因此，主站应被允许通过广播发送一个确认标志被置为 false 的 INIT 请求（AL 控制寄存器=0x0001）来复位所有设备。随后，复杂设备应复位错误标志。

在错误发生情况下，首先应将 AL 状态代码（AL Status Code）置位，然后将错误标志置位。在清零错误标志后，AL 状态代码也应被清零。如果错误标志被清零，主站应忽略 AL 状态代码。

当 AL 状态从 Op 转换到 SafeOp 发生错误时，输出同步管理器应被禁用。输入同步管理器应仅在出现导致无效的输入（输入错误或同步错误）的错误时才被禁用。

输出同步管理器在错误被确认且没有输出错误时应被重新使能。

输入同步管理器在错误被确认且没有输入错误时应被重新使能。

引导（Bootstrap）状态是可选的并且只与“Init”状态相互转换。引导状态的唯一目的就是下载设备固件。在引导状态邮箱是可用的，但限于通过 EtherCAT 服务（FoE）协议进行文件访问。

ESM 规定如图 47 所示。

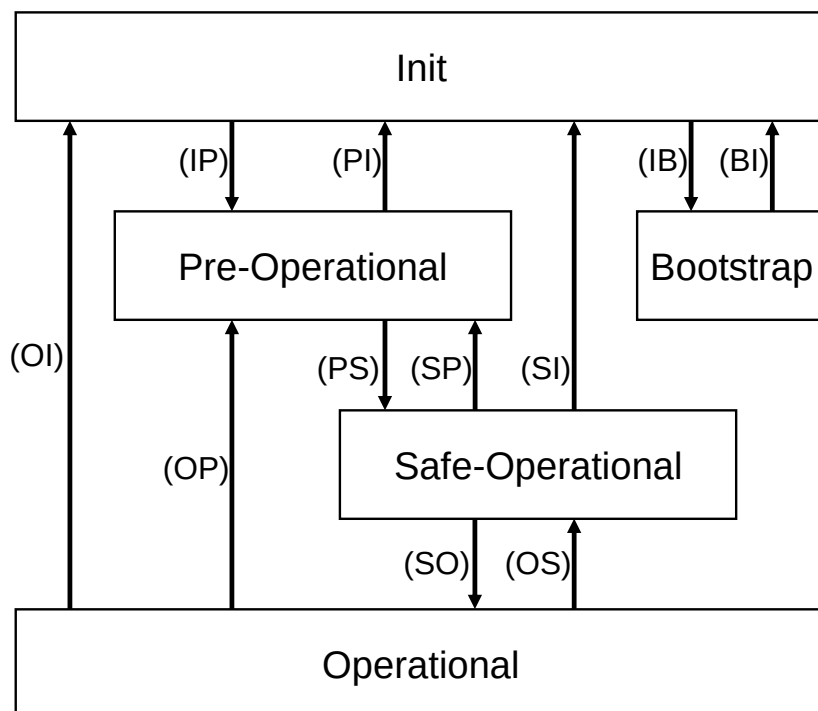


图 47 – ESM 图

本地管理服务与 ESM 内的状态转换相关, 如表 94 所示。如果有多个服务与状态转换相关, 从站应用将处理所有相关服务。

表 94 – 状态转换与本地管理服务

状态转换	本地管理服务
IP	开始邮箱通信 (Start Mailbox Communication)
PI	停止邮箱通信 (Stop Mailbox Communication)
PS	开始输入更新 (Start Input Update)
SP	停止输入更新 (Stop Input Update)
SO	开始输出更新 (Start Output Update)
OS	停止输出更新 (Stop Output Update)
OP	停止输出更新, 停止输入更新 (Stop Output Update, Stop Input Update)
SI	停止输出更新, 停止邮箱通信 (Stop Input Update, Stop Mailbox Communication)
OI	停止输出更新, 停止输入更新, 停止邮箱通信 (Stop Output Update, Stop Input Update, Stop Mailbox Communication)
IB	开始引导模式 (Start Bootstrap Mode)
BI	重启设备 (Restart Device)

6.4.1.2 ESM 状态

6.4.1.2.1 Init

“Init” 状态定义了应用层上主站与从站之间通信关系的初始点。在应用层上主站与从站之间无直接通信。主站使用 “Init” 状态来初始化一组 ESC 配置寄存器。如果从站支持邮箱服务, 则相应的同步管理器配置也在 “Init” 状态完成。

6.4.1.2.2 Pre-Operational

如果从站支持可选的邮箱, 则在“Pre-Operational”状态邮箱被激活。主站和从站均可使用邮箱和适当的协议来交换应用特定的初始化信息和参数。在该状态下没有过程数据通信。

6.4.1.2.3 Safe-Operational

在“Safe-Operational”状态, 从站应用应传递实际输入数据而不处理输出数据。输出应被设置为“安全状态”。

6.4.1.2.4 Operational

在“Operational”状态, 从站应用应传递实际输入数据且主站应用程序应传递实际输出数据。

6.4.1.2.5 Bootstrap

在可选“Bootstrap”状态, 从站应用应能接受通过 FoE 协议下载的新固件。

6.4.1.3 原语定义

6.4.1.3.1 在 DL 与 ESM 之间交换的原语

表 95 给出了由 ESM 发出并由 DL 接收的服务原语及其相关参数。

表 95 – ESM 发出到 DL 的原语

原语名称	相关参数	功能
AL State Change.req	AL Status Application Specific AL Status Code	见 ETG.1000.5 的服务定义
AL Control.rsp(+)	AL State AL Status Code	见 ETG.1000.5 的服务定义
AL Control.rsp(-)	AL State AL Status Code	见 ETG.1000.5 的服务定义

表 96 给出了由 DL 发出并由 ESM 接收的服务原语及其相关参数。

表 96 – DL 发出到 ESM 的原语

原语名称	相关参数	功能
AL Control.ind	AL Control State Ack Flag	见 ETG.1000.5 的服务定义

6.4.1.3.2 在应用与 ESM 之间交换的原语

表 97 给出了由 AL 发出并由 ESM 接收的服务原语及其相关参数。

表 97 – 应用发送到 ESM 的原语

原语名称	相关参数	功能
Stop Input		应用停止了过程数据的更新
SM Change		同步管理器配置变化 (可以在本地启用/禁用或通过通信方式发出)

表 98 给出了由 ESM 发出并由 AL 接收的服务原语及其相关参数。

表 98 – ESM 到应用的原语

原语名称	相关参数	功能
START MBX HANDLER		开始邮箱通信 INIT->PREOP: 通过启用同步管理器来开始邮箱通信
STOP MBX HANDLER		停止邮箱通信 PREOP->INIT: 通过禁用同步管理器来停止邮箱通信
START INPUT HANDLER		开始输入更新 PREOP->SAFEOP: 通过启用同步管理器来开始到 ESC 的输入更新 通过启用同步管理器来开始来自 ESC 的输出更新
STOP INPUT HANDLER		停止输入更新 SAFEOP->PREOP: 通过禁用同步管理器来停止到 ESC 的输入更新 通过禁用同步管理器来停止来自 ESC 的输出更新
START LOCAL OUTPUT HANDLER		开始本地输出更新 SAFEOP->OP: 设置输出为接收到的过程值
STOP LOCAL OUTPUT HANDLER		停止本地输出更新 OP->SAFEOP: 设置输出为安全值

6.4.1.3.3 ESM 变量

表 99 定义了 ESM 中用到的变量。

表 99 – ESM 变量

变量名	描述
bootSupported	从站支持引导 (Bootstrap) 模式
dcNotAccepted	设备不支持 DC 也不支持 DC 设置。如果 DC 被设置, 则设备将进入错误状态。这意味着同步控制寄存器 0x0981 应设为 0。
dcRequired	设备要求 DC 设置 (比特 0x0981:00 到 0x0981:02 应被相应设置)。不支持其他模式如 “(Freerun)” 和 “带 SM 事件的同步” 模式。
dcRunning	DC 运行且用于主站与从站间的同步。
dcSupported	设备至少支持一种使用分布式时钟的 Op 模式。

变量名	描述
localErrorCode	导致同步管理器通道被禁用的先前本地错误的根本原因
localErrorFlag	本地应用的标志, 指示导致同步管理器被禁用的本地错误。在发生导致 Op 状态转换为 SafeOp 状态的错误的情况下使用。
outputRunning	如果 WD_ENABLE = 0: 当至少接收一次输出时, 内部标志被置位。 如果 WD_ENABLE = 1: 当在看门狗时间内接收到输出时, 内部标志被置位。
outputRunningNotRequired	同步管理器看门狗不被使用且可被禁用。本地应用使用一个内部看门狗定时器并接受到 OP 状态的转换, OutputRunning=false (legacy 模式)。
pllRunning	本地应用与 DC 事件和主站应用同步。
safeOp2OpTimeoutTimer	从 SafeOp 状态到 Op 状态的转换超时
dcEventReceived	从站至少接收一次 Sync0/Sync1 事件
waitForPlRunning	等待直到本地 PLL 被主站周期锁定 (等待直到 pllRunning=TRUE)。该时间应短于要比 SafeOp2OpTimeoutTimer。
wdEnableRequired	同步管理器的看门狗必须被启用。
wdDisabled	同步管理器看门狗的状态 TRUE: WD 被禁用 FALSE: WD 被启用
wdEnabled	同步管理器看门狗的状态 TRUE: WD 被启用 FALSE: WD 被禁用

6.4.1.3.4ESM 宏

表 100 定义了 ESM 用到的宏。

表 100 – ESM 宏

宏名称	描述
SM_SETTINGS_0_1_MATCH	根据本地设置检查所请求的邮箱同步管理器设置的本地函数
SM_SETTINGS_2_TO_n_MATCH	根据本地设置检查所请求的过程数据同步管理器的本地函数 对于不使用邮箱的从站, 该值可为 SM 0 到 SM n。
DC_ACTIVATED	同步本地应用。 注: AssignActive 应被用来与 0x0701 进行逻辑与 (AND) 操作, 或者为 0x0300 或者为 0x0700 (按要求所有其他比特可被置位)。
DISABLE_SM_CHANNEL	同步管理器 2 应被禁用。仅当没有输出同步管理器或没有输入错误发生时, 在保持同步管理器 2 被启用时, 输入同步管理器才可被禁用。
ENABLE_SM_CHANNEL	同步管理器应被启用
WD_ENABLE	同步管理器的看门狗被启用(同步管理器寄存器基地址+0x04 的 Bit06)
START_WD	ESC 看门狗被启动。此外, 对本地主机控制器的看门狗定时器被启动。

6.4.1.3.5 ESM 功能

表 101 定义了 ESM 用到的函数。

表 101 – ESM 函数

函数名称	描述
Output Event	从 DL 发出的到 ESM 的原语: 指示新输出数据到达的事件
WD Expired	(本地) WD 超时
SM_Chg	从 DL 发出的到 ESM 的原语
AL_State Change.req (AL Status, AL Status Code)	从应用发出的到 ESM 的原语: 从站应用指示状态改变
PLL Running Event	指示主站现在与本地应用完成同步的本地事件 (输出从主站发出且在 DC 事件前被从站接收)。
SafeOp2OpTimeoutTimer Expired	指示本地状态改变超时的事件
Start SafeOp2OpTimeoutTimer	监视从 SafeOp 到 Op 超时的本地定时器被启动
AckSM_Chg	同步管理器改变事件应得到从站确认

6.4.1.3.6 原语参数

在 DL、ESM 和应用之间交互的原语参数, 见 ETG.1000.5。

6.4.1.4 ESM 状态表

表 102 给出了 ESM 状态机的完整描述。

表 102 – ESM 状态表

#	当前状态	事件 /条件 =>动作	下一个状态
1.1	INIT	AL_Control.ind (AL Control State, Ack Flag) /AL_ERROR_FLAG = 1 and .ACK_FLAG = 0 and AL_CONTROL_STATE = STATE_INIT => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_CONTROL_STATE = STATE_INIT AL_Control.rsp(+) (AL State, AL Status Code)	INIT
1.2	INIT	AL_Control.ind (AL Control State, Ack Flag) /AL_ERROR_FLAG = 1 and .ACK_FLAG = 0 and AL_CONTROL_STATE <> STATE_INIT => AL_Control.rsp(-) (AL State, AL Status Code)	INIT
2	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_INIT => AL_ERROR_FLAG = 0 AL_Control.rsp(+) (AL State, AL Status Code)	INIT
3	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_PREOP and SM_SETTINGS_0_AND_1_MATCH => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_PREOP START_MBX_HANDLER AL_Control.rsp(+) (AL State, AL Status Code)	PREOP
4	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_PREOP and not SM_SETTINGS_0_AND_1_MATCH => AL_STATE = STATE_INIT AL_STATUS_CODE = 0x16 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	INIT
5	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_BOOT and BOOT_SUPPORTED and SM_SETTINGS_0_AND_1_MATCH => AL_ERROR_FLAG = 0 AL_STATE = STATE_BOOT AL_STATUS_CODE = 0 START_MBX_HANDLER AL_Control.rsp(+) (AL State, AL Status Code)	BOOT

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
6	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_BOOT and BOOT_SUPPORTED and not SM_SETTINGS_0_AND_1_MATCH => AL_STATE = STATE_INIT AL_STATUS_CODE = 0x15 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	INIT
7	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_BOOT and not BOOT_SUPPORTED => AL_STATE = STATE_INIT AL_STATUS_CODE = 0x13 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	INIT
8	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and (AL_CONTROL_STATE = STATE_SAFEOP OR AL_CONTROL_STATE = STATE_OP) => AL_STATE = STATE_INIT AL_STATUS_CODE = 0x11 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	INIT
9	INIT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and (AL_CONTROL_STATE = unknownState) => AL_STATE = STATE_INIT AL_STATUS_CODE = 0x12 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	INIT
10	INIT	SM_Chg => ignore	INIT
11.1	PREOP	AL_Control.ind (AL Control State, Ack Flag) /AL_ERROR_FLAG = 1 and .ACK_FLAG = 0 and AL_CONTROL_STATE = STATE_INIT => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_INIT STOP_MBX_HANDLER AL_Control.rsp(= ±) (AL State, AL Status Code)	INIT
11.2	PREOP	AL_Control.ind (AL Control State, Ack Flag) /AL_ERROR_FLAG = 1 and .ACK_FLAG = 0 and AL_CONTROL_STATE <> STATE_INIT => AL_Control.rsp(-) (AL State, AL Status Code)	PREOP

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
12	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_INIT => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_INIT STOP_MBX_HANDLER AL Control.rsp(+) (AL State, AL Status Code)	INIT
13	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_PREOP => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL Control.rsp(+) (AL State, AL Status Code)	PREOP
14.1	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_SAFEOP and SM_SETTINGS_2_TO_n_MATCH and ((not DC_ACTIVATED and not dcRequired) or (DC_ACTIVATED and not dcNotAccepted and not dcSupported)) => dcRunning = FALSE AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_SAFEOP START_INPUT_HANDLER outputRunning = FALSE AL Control.rsp(+) (AL State, AL Status Code)	SAFEOP
14.2	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_SAFEOP and SM_SETTINGS_2_TO_n_MATCH and DC_ACTIVATED and not dcNotAccepted and dcSupported => dcRunning = TRUE AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_SAFEOP START_INPUT_HANDLER outputRunning = FALSE AL Control.rsp(+) (AL State, AL Status Code)	SAFEOP
14.3	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_SAFEOP and SM_SETTINGS_2_TO_n_MATCH and ((DC_ACTIVATED and dcNotAccepted) or (not DC_ACTIVATED and dcRequired)) => AL_STATUS_CODE = 0x30 AL_ERROR_FLAG = 1 AL_STATE = STATE_PREOP AL Control.rsp(-) (AL State, AL Status Code)	PREOP

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
17	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_SAFEOP and not SM_SETTINGS_2_TO_n_MATCH => AL_STATE = STATE_PREOP AL_STATUS_CODE = 0x17 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	PREOP
18	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and (AL_CONTROL_STATE = STATE_BOOT or AL_CONTROL_STATE = STATE_OP) => AL_STATE = STATE_PREOP AL_STATUS_CODE = 0x11 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	PREOP
19	PREOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and (AL_CONTROL_STATE = unknownState) => AL_STATE = STATE_PREOP AL_STATUS_CODE = 0x12 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	PREOP
20.1	PREOP	SM_Chg / AL_ERROR_FLAG = 0 and SM_SETTINGS_0_AND_1_MATCH => AckSM_Chg	PREOP
20.2	PREOP	SM_Chg / AL_ERROR_FLAG = 1 => ignore	PREOP
21	PREOP	SM_Chg /AL_ERROR_FLAG = 0 and not SM_SETTINGS_0_AND_1_MATCH => AL_STATUS_CODE = 0x16 AL_ERROR_FLAG = 1 AL_STATE = STATE_INIT STOP_MBX_HANDLER AckSM_Chg AL Status Changed.req (AL state, AL staus code)	INIT
22.1	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 1 and ACK_FLAG = 0) and AL_CONTROL = STATE_INIT => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_INIT STOP_MBX_HANDLER STOP_INPUT_HANDLER STOP_LOCAL_OUTPUT_HANDLER AL_Control.rsp(+) (AL State, AL Status Code)	INIT

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
22.2	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 1 and ACK_FLAG = 0) and AL_CONTROL_STATE <> STATE_INIT => AL_Control.rsp(-) (AL State, AL Status Code)	SAFEOP
23	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_INIT => AL_ERROR_FLAG = 0 AL_STATE = STATE_INIT STOP_MBX_HANDLER STOP_INPUT_HANDLER AL_Control.rsp(+) (AL State, AL Status Code)	INIT
24	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_PREOP => AL_ERROR_FLAG = 0 AL_STATE = STATE_PREOP STOP_INPUT_HANDLER AL_Control.rsp(+) (AL State, AL Status Code)	PREOP
25.1	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 1 and (not localErrorFlag and .ACK_FLAG = 1)) and AL_CONTROL_STATE = STATE_SAFEOP => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 ENABLE_SM_CHANNEL AL_Control.rsp(+) (AL State, AL Status Code)	SAFEOP
25.2	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /((AL_ERROR_FLAG = 0 or (localErrorFlag and .ACK_FLAG = 1)) and AL_CONTROL_STATE = STATE_SAFEOP => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_Control.rsp(+) (AL State, AL Status Code)	SAFEOP
26.1	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_OP and not WD_ENABLE and not outputRunningNotRequired and not wdEnabledRequired and (not dcRunning or pllRunning) and not localErrorFlag => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_OP ENABLE_SM_CHANNEL START_LOCAL_OUTPUT_HANDLER wdDisabled = TRUE AL_Control.rsp(+) (AL State, AL Status Code)	OP

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
26.2	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_OP and ((not WD_ENABLE and outputRunningNotRequired) or (WD_ENABLE and outputRunning)) and (not dcRunning or pllRunning) and not localErrorFlag => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_OP ENABLE_SM_CHANNEL START_LOCAL_OUTPUT_HANDLER wdDisabled = FALSE AL_Control.rsp(+) (AL State, AL Status Code)	OP
26.3	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_OP and not WD_ENABLE and not outputRunningNotRequired and not wdEnabledRequired and dcRunning and not pllRunning and not localErrorFlag => ENABLE_SM_CHANNEL wdDisabled = TRUE waitforPlIRunning = TRUE Start SafeOpToOpTimeoutTimer	SAFEOP
26.4	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_OP and ((not WD_ENABLE and outputRunningNotRequired) or (WD_ENABLE and outputRunning)) and dcRunning and not pllRunning and not localErrorFlag => ENABLE_SM_CHANNEL wdDisabled = FALSE waitforPlIRunning = TRUE Start SafeOpToOpTimeoutTimer	SAFEOP
26.5	SAFEOP	PLL Running Event /waitforPlIRunning => AL_STATE = STATE_OP AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 ENABLE_SM_CHANNEL START_LOCAL_OUTPUT_HANDLER AL_Control.rsp(+) (AL State, AL Status Code)	OP
26.6	SAFEOP	PLL Running Event /not waitforPlIRunning => ignore	SAFEOP
26.7	SAFEOP	SafeOp2OpTimeoutTimer Expired /waitforPlIRunning and not dcEventReceived => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = 0x2D AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	SAFEOP

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
26.8	SAFEOP	SafeOp2OpTimeoutTimer Expired /waitForPIIRunning and dcEventReceived => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = 0x32 AL_ERROR_FLAG = 1 AL Control.rsp(-) (AL State, AL Status Code)	SAFEOP
26.9	SAFEOP	SafeOp2OpTimeoutTimer Expired /not waitForPIIRunning => ignore	SAFEOP
27.1	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_OP and (not WD_ENABLE and not outputRunningNotRequired and wdEnabledRequired) or (not outputRunning and WD_ENABLE) and not localErrorFlag => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = 0x1B AL_ERROR_FLAG = 1 AL Control.rsp(-) (AL State, AL Status Code)	SAFEOP
27.2	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_OP and localErrorFlag => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = localErrorCode AL_ERROR_FLAG = 1 AL Control.rsp(-) (AL State, AL Status Code)	SAFEOP
29	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and AL_CONTROL_STATE = STATE_BOOT => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = 0x11 AL_ERROR_FLAG = 1 AL Control.rsp(-) (AL State, AL Status Code)	SAFEOP
30	SAFEOP	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and (AL_CONTROL_STATE = unknownState) => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = 0x12 AL_ERROR_FLAG = 1 AL Control.rsp(-) (AL State, AL Status Code)	SAFEOP

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
31.1	SAFEOP	SM_Chg / AL_ERROR_FLAG = 0 and SM_SETTINGS_0_AND_1_MATCH and SM_SETTINGS_2_TO_n_MATCH => AckSM_Chg	SAFEOP
31.2	SAFEOP	SM_Chg / AL_ERROR_FLAG = 1 => ignore	SAFEOP
32	SAFEOP	SM_Chg / AL_ERROR_FLAG = 0 and SM_SETTINGS_0_AND_1_MATCH and not SM_SETTINGS_2_TO_n_MATCH => AL_STATUS_CODE = 0x17 AL_ERROR_FLAG = 1 AL_STATE = STATE_PREOP STOP_INPUT_HANDLER AckSM_Chg AL Status Changed.req (AL state, AL staus code)	PREOP
33	SAFEOP	SM_Chg / AL_ERROR_FLAG = 0 and not SM_SETTINGS_0_AND_1_MATCH => AL_STATUS_CODE = 0x16 AL_ERROR_FLAG = 1 AL_STATE = STATE_INIT STOP_INPUT_HANDLER STOP_MBX_HANDLER AckSM_Chg AL Status Changed.req (AL state, AL staus code)	INIT
34.1	SAFEOP	AL_State Change.req (AL Status, AL Status Code) /AL_STATE = STATE_SAFEOP and AL_ERROR_FLAG = 1 => DISABLE_SM_CHANNEL AL_STATE = STATE_SAFEOP AL_ERROR_FLAG = TRUE localErrorFlag = TRUE localErrorCode = AL_STATUS_CODE AL Status Changed.req (AL state, AL status code)	SAFEOP
34.2	SAFEOP	AL_State Change.req (AL Status, AL Status Code) /AL_STATE = STATE_SAFEOP and AL_ERROR_FLAG = 0 and localErrorFlag => ENABLE_SM_CHANNEL localErrorFlag = FALSE localErrorCode = 0	SAFEOP
34.3	SAFEOP	AL_State Change.req (AL Status, AL Status Code) /AL_STATE = STATE_PREOP and AL_ERROR_FLAG = 1 => STOP_INPUT_HANDLER AL_STATE = STATE_PREOP AL_ERROR_FLAG = TRUE AL Status Changed.req (AL state, AL status code)	PREOP

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
34.4	SAFEOP	AL_State Change.req (AL Status, AL Status Code) /AL_STATE = STATE_INIT and AL_ERROR_FLAG = 1 => STOP_MBX_HANDLER STOP_INPUT_HANDLER AL_STATE = STATE_INIT AL_ERROR_FLAG = TRUE AL Status Changed.req (AL state, AL status code)	INIT
34.5	SAFEOP	AL_State Change.req (AL Status, AL Status Code) /((AL_STATE = STATE_INIT or STATE_PREOP) and AL_ERROR_FLAG = 0) or AL_STATE = STATE_OP or STATE_BOOT or unknown => ignore	SAFEOP
35.1	SAFEOP	Output Event /wdEnabled => outputRunning= TRUE START_WD	SAFEOP
35.2	SAFEOP	WD Expired /wdEnabled => outputRunning= FALSE	SAFEOP
35.3	SAFEOP	Output Event /not wdEnabled => ignore	SAFEOP
35.4	SAFEOP	WD Expired /not wdEnabled => ignore	SAFEOP
37	OP	AL_Control.ind (AL Control State, Ack Flag) /AL_CONTROL_STATE = STATE_INIT => AL_STATUS_CODE = 0 AL_ERROR_FLAG = 0 AL_STATE = STATE_INIT STOP_MBX_HANDLER STOP_INPUT_HANDLER STOP_LOCAL_OUTPUT_HANDLER AL Control.rsp(+) (AL State, AL Status Code)	INIT
38	OP	AL_Control.ind (AL Control State, Ack Flag) /AL_CONTROL_STATE = STATE_PREOP => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_PREOP STOP_INPUT_HANDLER STOP_LOCAL_OUTPUT_HANDLER AL Control.rsp(+) (AL State, AL Status Code)	PREOP
39	OP	AL_Control.ind (AL Control State, Ack Flag) /AL_CONTROL_STATE = STATE_SAFEOP => AL_ERROR_FLAG = 0 AL_STATUS_CODE = 0 AL_STATE = STATE_SAFEOP STOP_LOCAL_OUTPUT_HANDLER AL Control.rsp(+) (AL State, AL Status Code)	SAFEOP

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
40	OP	AL_Control.ind (AL Control State, Ack Flag) /AL_CONTROL_STATE = STATE_OP => AL_Control.rsp(+) (AL State, AL Status Code)	OP
42	OP	AL_Control.ind (AL Control State, Ack Flag) /AL_CONTROL_STATE = STATE_BOOT => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = 0x11 AL_ERROR_FLAG = 1 DISABLE_SM_CHANNEL STOP_LOCAL_OUTPUT_HANDLER AL_Control.rsp(-) (AL State, AL Status Code)	SAFEOP
43	OP	AL_Control.ind (AL Control State, Ack Flag) /AL_CONTROL_STATE = unknownState => AL_STATE = STATE_SAFEOP AL_STATUS_CODE = 0x12 AL_ERROR_FLAG = 1 DISABLE_SM_CHANNEL STOP_LOCAL_OUTPUT_HANDLER AL_Control.rsp(-) (AL State, AL Status Code)	SAFEOP
44	OP	SM_Chg / AL_ERROR_FLAG = 0 and SM_SETTINGS_0_AND_1_MATCH and SM_SETTINGS_2_TO_n_MATCH => AckSM_Chg	OP
45	OP	SM_Chg / AL_ERROR_FLAG = 0 and SM_SETTINGS_0_AND_1_MATCH and not SM_SETTINGS_2_TO_n_MATCH => AL_STATUS_CODE = 0x17 AL_ERROR_FLAG = 1 AL_STATE = STATE_PREOP STOP_LOCAL_OUTPUT_HANDLER STOP_INPUT_HANDLER AckSM_Chg AL Status Changed.req (AL state, AL staus code)	PREOP
46	OP	SM_Chg / AL_ERROR_FLAG = 0 and not SM_SETTINGS_0_AND_1_MATCH => AL_STATUS_CODE = 0x16 AL_ERROR_FLAG = 1 AL_STATE = STATE_INIT STOP_LOCAL_OUTPUT_HANDLER STOP_INPUT_HANDLER STOP_MBX_HANDLER AckSM_Chg AL Status Changed.req (AL state, AL staus code)	INIT

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
47.1	OP	AL State Change.req (AL Status, AL Status Code) /AL_STATE = STATE_SAFEOP and AL_ERROR_FLAG = 1 => STOP_LOCAL_OUTPUT_HANDLER DISABLE_SM_CHANNEL AL_STATE = STATE_SAFEOP localErrorFlag = TRUE localErrorCode = AL_STATUS_CODE AL Status Changed.req (AL state, AL status code)	SAFEOP
47.2	OP	AL State Change.req (AL Status, AL Status Code) /AL_STATE = STATE_PREOP and AL_ERROR_FLAG = 1 => STOP_INPUT_HANDLER STOP_LOCAL_OUTPUT_HANDLER AL_STATE = STATE_PREOP localErrorFlag = TRUE localErrorCode = AL_STATUS_CODE AL Status Changed.req (AL state, AL status code)	PREOP
47.3	OP	AL State Change.req (AL Status, AL Status Code) /AL_STATE = STATE_INIT and AL_ERROR_FLAG = 1 => STOP_MBX_HANDLER STOP_INPUT_HANDLER STOP_LOCAL_OUTPUT_HANDLER AL_STATE = STATE_PREOP localErrorFlag = TRUE localErrorCode = AL_STATUS_CODE AL Status Changed.req (AL state, AL status code)	INIT
47.4	OP	AL State Change.req (AL Status, AL Status Code) /((AL_STATE = STATE_SAFEOP or STATE_PREOP) and AL_ERROR_FLAG = 0) or AL_CONTROL_STATE = STATE_BOOT or unknown => ignore	OP
48	OP	Output event => Trigger WD Update Outputs	OP
49	OP	WD expired => AL_STATUS_CODE = 0x1B AL_ERROR_FLAG = 1 AL_STATE = STATE_SAFEOP STOP_LOCAL_OUTPUT_HANDLER DISABLE_SM_CHANNEL Output_Running = FALSE AL Status Changed.req (AL state, AL status code)	SAFEOP
51	BOOT	AL_Control.ind (AL Control State, Ack Flag) /AL_CONTROL_STATE = STATE_INIT => AL_ERROR_FLAG = 0 AL_STATE = STATE_INIT STOP_MBX_HANDLER AL_Control.rsp(+) (AL State, AL Status Code)	INIT

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
52	BOOT	AL_Control.ind (AL Control State, Ack Flag) / AL_CONTROL_STATE = STATE_BOOT => AL_ERROR_FLAG = 0 AL_Control.rsp(+) (AL State, AL Status Code)	BOOT
53.1	BOOT	AL_Control.ind (AL Control State, Ack Flag) /(AL_CONTROL_STATE = STATE_PREOP or AL_CONTROL_STATE = STATE_SAFEOP or AL_CONTROL_STATE = STATE_OP) => AL_STATUS_CODE = 0x11 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	INIT
53.2	BOOT	AL_Control.ind (AL Control State, Ack Flag) /(AL_CONTROL_STATE = STATE_PREOP or AL_CONTROL_STATE = STATE_SAFEOP or AL_CONTROL_STATE = STATE_OP) => ignore	BOOT
54.1	BOOT	AL_Control.ind (AL Control State, Ack Flag) /(AL_CONTROL_STATE = unknownState) => AL_STATUS_CODE = 0x12 AL_ERROR_FLAG = 1 AL_Control.rsp(-) (AL State, AL Status Code)	INIT
54.2	BOOT	AL_Control.ind (AL Control State, Ack Flag) /(AL_ERROR_FLAG = 0 or .ACK_FLAG = 1) and (AL_CONTROL_STATE = unknownState) => ignore	BOOT
55	BOOT	SM_Chg /SM_SETTINGS_0_AND_1_MATCH => ignore	BOOT
56.1	BOOT	SM_Chg /not SM_SETTINGS_0_AND_1_MATCH => AL_STATUS_CODE = 0x16 AL_ERROR_FLAG = 1 AL_STATE = STATE_INIT STOP_MBX_HANDLER AL Status Changed.req (AL state, AL status code)	INIT
56.2	BOOT	SM_Chg /not SM_SETTINGS_0_AND_1_MATCH => ignore	BOOT
57	BOOT	AL_State_Change.req (AL Status, AL Status Code) /AL_STATE = STATE_INIT and AL_ERROR_FLAG = 1 => STOP_MBX_HANDLER AL_STATE = STATE_INIT AL_ERROR_FLAG = TRUE AL Status Changed.req (AL state, AL status code)	INIT

表 102 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
58	BOOT	AL_State Change.req (AL Status, AL Status Code) /((AL_STATE = STATE_INIT and AL_ERROR_FLAG = 0) or AL_STATE = STATE_PREOP or STATE_SAFEOP or STATE_OP or STATE_BOOT or unknown => ignore	BOOT

6.4.1.5 ESM 函数

表 103 定义 ESM 的功能。

表 103 – ESM 函数

名称	功能
SM_Chg	指示 SM 设置变化的 DL 事件服务
Stop Inp	禁用输入更新的 AL 本地函数禁用
Output event	指示新输出数据到来的事件
WD expired	本地 WD 到时
SM_SETTINGS_OBJECT_SYNC_TYPE_MATCH	检查被请求的同步设置与本地特性是否相符的本地函数

6.4.2 邮箱处理程序状态机

6.4.2.1 描述

邮箱处理程序负责协调主站和从站的有关邮箱的操作。邮箱写被转发到特定状态机，响应被放到读邮箱。

因为没有特定的面向服务的状态，所以没有状态表。

邮箱处理程序的任务包括：

- 将写邮箱服务映射到协议处理器
- 将读邮箱的服务请求队列进行排队
- 对读邮箱的传输进行证实

协议处理程序可能是：

- CoE 状态机
- EoE 状态机
- PoE 状态机
- 行规特定状态机

6.4.2.2DL、邮箱处理程序和协议处理程序之间交换的原语

表 104 给出了由邮箱处理程序发出并由 DL 接收的服务原语及其相关参数。

表 104 – 邮箱处理程序向 DL 发出的原语

原语名称	相关参数	功能
Mailbox Read Upd.req	Length Address Channel Priority Type Cnt Service Data	见 ETG.1000.3 的服务定义

表 105 给出了由 DL 发出并由邮箱处理程序接收的服务原语及其相关参数。

表 105 – DL 向邮箱处理程序发出的原语

原语名称	相关参数	功能
Mailbox Write.ind	Length Address Channel Priority Type Cnt Service Data	见 ETG.1000.3 的服务定义
Mailbox Read Upd.cnf	Success	见 ETG.1000.3 的服务定义

表 106 给出了由协议处理程序发出并由邮箱处理程序接收的服务原语及其相关参数。“TYPE”前缀源于相应的 DL 服务原语的类型参数。

表 106 – 协议处理程序向邮箱处理程序发出的原语

原语名称	相关参数	功能
TYPE Mailbox Read Upd.req	Length Address Channel Priority Service Data	见 ETG.1000.3 的 Mailbox Read Upd 服务定义

表 107 给出了由邮箱处理程序发出并由协议处理程序接收的服务原语及其相关参数。“TYPE”前缀源于相应的 DL 服务原语的类型参数。

表 107 – 邮箱处理程序向协议处理程序发出的原语

原语名称	相关参数	功能
TYPE Mailbox Write.ind	Length Address Channel Priority Service Data	见 ETG.1000.3 的服务定义
TYPE Mailbox Read Upd.cnf	Success	见 ETG.1000.3 的 Mailbox Read Upd 服务定义

6.4.3 CoE 状态机

6.4.3.1 描述

CoE 状态机负责处理 CoE 服务。

主要任务是执行服务请求或提供以下形式的响应:

- 单一帧
- 帧序列 (信息服务)
- 带有更多后续帧指示的单个帧
- 由于服务的错误结束而中止

6.4.3.2 原语定义

6.4.3.2.1 邮箱处理程序和 CoESM 之间交换的原语

邮箱处理程序和 CoESM 之间交换的原语在 6.4.2.2 中描述。

6.4.3.2.2 应用和 CoESM 之间交换的原语

表 108 给出了由 AL 发出并由 CoESM 接收的服务原语及其相关参数。SDO 信息服务代表一组通过操作码 opcode 参数进行区分的服务 (Get OD list、Get object description 和 Get entry description)。

表 108 – 应用向 CoESM 发出的原语

原语名称	相关参数	功能
SDO Download Expedited.rsp	Success Address Index Subindex	见 ETG.1000.5 的 CoE 服务定义
SDO Download Normal.rsp	Success Address Index Subindex	见 ETG.1000.5 的 CoE 服务定义
Download SDO Segment.rsp	Success Address Toggle	见 ETG.1000.5 的 CoE 服务定义
SDO Upload Expedited.rsp	Success Address Index Subindex Size Data	见 ETG.1000.5 的 CoE 服务定义
SDO Upload Normal.rsp	Success Address Index Subindex Complete Size Size Data	见 ETG.1000.5 的 CoE 服务定义
Upload SDO Segment.rsp	Success Address Toggle More Follows Size Data	见 ETG.1000.5 的 CoE 服务定义
Abort SDO Transfer.req	Address Index Subindex Reason	见 ETG.1000.5 的 CoE 服务定义
SDO Info Service.rsp	Address Opcode Incomplete Fragments Left Size Data	见 ETG.1000.5 的 CoE 服务定义
SDO Info Seg Service.req	Address Opcode Incomplete Fragments Left Size Data	见 ETG.1000.5 的 CoE 服务定义
Emergency Service.req	Address Error Code Error Register Size Data	见 ETG.1000.5 的 CoE 服务定义

表 109 给出了由 CoESM 发出并由 AL 接收的服务原语及其相关参数。

表109 – CoESM 向应用发出的原语

原语名称	相关参数	功能
SDO Download Expedited.ind	Address Index Subindex Size Data	见 ETG.1000.5 的 CoE 服务定义
SDO Download Normal.ind	Address Index Subindex Complete Size Size Data	见 ETG.1000.5 的 CoE 服务定义
Download SDO Segment.ind	Address Toggle More Follows Size Data	见 ETG.1000.5 的 CoE 服务定义
SDO Upload Expedited.ind	Address Index Subindex	见 ETG.1000.5 的 CoE 服务定义
SDO Upload Normal.ind	Address Index Subindex	见 ETG.1000.5 的 CoE 服务定义
Upload SDO Segment.ind	Address Toggle	见 ETG.1000.5 的 CoE 服务定义
Abort SDO Transfer.req	Address Index Subindex Reason	见 ETG.1000.5 的 CoE 服务定义
SDO Info Service.ind	Address Opcode Incomplete Fragments Left Size Data	见 ETG.1000.5 的 CoE 服务定义

6.4.3.2.3 原语的参数

CoESM 和应用之间交换的原语参数在 ETG.1000.5 中描述。

6.4.3.3 CoESM 状态表

表 110 给出了 CoESM 状态机的完整描述。

表 110 – CoESM 状态表

#	当前状态	事件 /条件 =>动作	下一个状态
1	OFF	START MAILBOX => Seg = 0	IDLE
2	OFF	STOP MAILBOX =>	OFF
3	IDLE	START MAILBOX =>	IDLE
4	IDLE	STOP MAILBOX =>	OFF

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
5	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.Service != 2 && Service_Data.Service != 8 => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
6	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.Service == 2 && Service_Data.Command_Specifier > 3 => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
7	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length != 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 1 && Service_Data.Transfer_Type == 1 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
8	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 1 && Service_Data.Transfer_Type == 1 => Size = 4 - Service_Data.Data_Set_Size Index = Service_Data.Index SubIndex = Service_Data.SubIndex Data = Service_Data.Data SDO Download Expedited.ind (Address, Index, Subindex, Size, Data)	DWLE
9	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length <= 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 1 && Service_Data.Transfer_Type == 1 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
10	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length > 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 1 && Service_Data.Transfer_Type == 1 => Seg = (Service_Data.Complete_Size - (Length - 10)) Toggle = FALSE Size = Length - 10 Complete_Size = Service_Data.Complete_Size Index = Service_Data.Index SubIndex = Service_Data.SubIndex Data = Service_Data.Data SDO Download Normal.ind (Address, Index, Subindex, Complete Size, Size, Data)	DWLN
11	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length < 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 0 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
12	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length >=10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 0 => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = ABT_SEQ CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
13	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length != 6 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 2 && Service_Data.Transfer_Type == 1 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
14	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 6 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 2 && Service_Data.Transfer_Type == 1 => Index = Service_Data.Index SubIndex = Service_Data.SubIndex SDO Upload Expedited.ind (Address, Index, Subindex)	UPLE
15	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length != 6 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 2 && Service_Data.Transfer_Type == 1 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
16	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 6 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 2 && Service_Data.Transfer_Type == 1 => Index = Service_Data.Index SubIndex = Service_Data.SubIndex SDO Upload Normal.ind (Address, Index, Subindex)	UPLN
17	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length != 3 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 3 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
18	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 3 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 3 => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = ABT_SEQ CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
19	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length != 8 && Service_Data.Service == 8 && Service_Data.Opcode == 1,3 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
20	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 8 && Service_Data.Service == 8 && Service_Data.Opcode == 1,3 => Size = Length - 6 Opcode = Service_Data.Opcode Incomplete = Service_Data.Incomplete FragmentsLeft = Service_Data.FragmentsLeft SDO Info Service.ind (Address, Opcode, Incomplete, Fragments Left, Size, Data)	INF
21	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length != 10 && Service_Data.Service == 8 && Service_Data.Opcode == 5 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
22	IDLE	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 10 && Service_Data.Service == 8 && Service_Data.Opcode == 5 => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = ABT_SEQ SDO Info Service.ind (Address, Opcode, Incomplete, Fragments Left, Size, Data)	INF
23	IDLE	CoE Read Upd.cnf (success) => ignore	IDLE
24	IDLE	Emergency Service.req (Address, Error Code, Error Register, Size, Data) => Length = 10 Service_Data.Service = 1 Service_Data.Error Code = Error Code Service_Data.Error Register = Error Register Service_Data.Data = Data CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
25	IDLE	other application service primitives => ignore	IDLE
26	DWLE	START MAILBOX => ignore	DWLE
27	DWLE	STOP MAILBOX => CANCEL SERVICE	OFF
28	DWLE	Abort SDO Transfer.req (Address, Index, Subindex, Reason) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = Reason CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
29	DWLE	SDO Download Expedited.rsp (Success, Address, Index, Subindex) => Length = 6 Service_Data.Service = 3 Service_Data.Command_Specifier = 3 Service_Data.Transfer Type = 1 Service_Data.Index = Index Service_Data.Subindex = Subindex CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
30	DWLE	other application service primitives => ignore	DWLE
31	DWLN	START MAILBOX => ignore	DWLN
32	DWLN	STOP MAILBOX => CANCEL SERVICE	OFF
33	DWLN	Abort SDO Transfer.req (Address, Index, Subindex, Reason) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = Reason Seg = 0 CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
34	DWLN	SDO Download Normal.rsp (Success, Address, Index, Subindex) => Length = 6 Service_Data.Service = 3 Service_Data.Command_Specifier = 3 Service_Data.Transfer Type = 0 Service_Data.Index = Index Service_Data.Subindex = Subindex CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
35	DWLN	other application service primitives => ignore	DWLN
36	DWLS	START MAILBOX => ignore	DWLS
37	DWLS	STOP MAILBOX => CANCEL SERVICE	OFF
38	DWLS	Abort SDO Transfer.req (Address, Index, Subindex, Reason) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = Reason Seg = 0 CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
39	DWLS	Download SDO Segment.rsp (Success, Address, Toggle) => Length = 6 Service_Data.Service = 3 Service_Data.Command_Specifier = 3 Service_Data.Toggle, Toggle = !Toggle CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
40	DWLS	other application service primitives => ignore	DWLS
41	UPLE	START MAILBOX => ignore	UPLE
42	UPLE	STOP MAILBOX => CANCEL SERVICE	OFF
43	UPLE	Abort SDO Transfer.req (Address, Index, Subindex, Reason) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = Reason CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
44	UPLE	SDO Upload Expedited.rsp (Success, Address, Index, Subindex, Size, Data) => Length = 10 Service_Data.Service = 3 Service_Data.Command_Specifier = 2 Service_Data.Transfer Type = 1 Service_Data.Data Set Size = 4 - Size Service_Data.Index = Index Service_Data.Subindex = Subindex Service_Data.Data = Data CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
45	UPLE	other application service primitives => ignore	UPLE
46	UPLN	START MAILBOX => ignore	UPLN
47	UPLN	STOP MAILBOX => CANCEL SERVICE	OFF
48	UPLN	Abort SDO Transfer.req (Address, Index, Subindex, Reason) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = Reason CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
49	UPLN	SDO Upload Normal.rsp (Success, Address, Index, Subindex, Complete Size, Size, Data) => Seg = Size - Complete Size Toggle = FALSE Length = 10 + Size Service_Data.Service = 3 Service_Data.Command_Specifier = 2 Service_Data.Transfer Type = 0 Service_Data.Index = Index Service_Data.Subindex = Subindex Service_Data.Data = Data CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
50	UPLN	other application service primitives => ignore	UPLN
51	UPLS	START MAILBOX => ignore	UPLS
52	UPLS	STOP MAILBOX => CANCEL SERVICE	OFF
53	UPLS	Abort SDO Transfer.req (Address, Index, Subindex, Reason) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = Reason Seg = 0 CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
54	UPLS	Upload SDO Segment.rsp (Success, Address, Toggle, More Follows, Size, Data) => Seg = (Seg -Size) if (Size > 7) then Service_Data.SegDataSize = 0 else Service_Data.SegDataSize = 7-Size Length = Size + 3 + Service_Data.SegDataSize Service_Data.Service = 3 Service_Data.Command_Specifier = 3 Service_Data.Toggle = Toggle Service_Data.Data = Data Service_Data.More Follows = More Follows CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
55	UPLS	other application service primitives => ignore	UPLS
56	INF	START MAILBOX => ignore	UPLS
57	INF	STOP MAILBOX => CANCEL SERVICE	OFF

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
58	INF	Abort SDO Transfer.req (Address, Index, Subindex, Reason) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = Reason Seg = 0 CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
59	INF	SDO Info Service.rsp (Address, Opcode, Incomplete, Fragments Left, Size, Data) => if FragmentsLeft >0) then Seg = 0x80000000 else Seg = 0 Length = Size + 6 Service_Data.Service = 8 Service_Data.Opcode = Opcode Service_Data.Incomplete = Incomplete Service_Data.Data = Data Service_Data.Fragments Left = Fragments Left CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
60	INF	SDO Info Seg Service.req (Address, Opcode, Incomplete, Fragments Left, Size, Data) => if FragmentsLeft >0) then Seg = 0x80000000 else Seg = 0 Length = Size + 6 Service_Data.Service = 8 Service_Data.Opcode = Opcode Service_Data.Incomplete = Incomplete Service_Data.Data = Data Service_Data.Fragments Left = Fragments Left CoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
61	WUPD	START MAILBOX =>	WUPD
62	WUPD	STOP MAILBOX => RESET MAILBOX	OFF
63	WUPD	CoE Mailbox Read Upd.cnf (success) /Seg == 0 =>	IDLE
64	WUPD	CoE Mailbox Read Upd.cnf (success) /Seg == 0x80000000 && (Fragments Left == 0) => Seg = 0	IDLE
65	WUPD	CoE Mailbox Read Upd.cnf (success) /Seg == 0x80000000 && (Fragments Left > 0) =>	INF
66	WUPD	CoE Mailbox Read Upd.cnf (success) /Seg > 0 && Seg < 0x80000000 =>	WSEG
67	WUPD	CoE Mailbox Read Upd.cnf (success) /Seg < 0 && Seg > 0x80000000 =>	WSEG
68	WUPD	Timeout => RESET MAILBOX	OFF
69	ERR	START MAILBOX =>	ERR

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
70	ERR	STOP MAILBOX => RESET MAILBOX	OFF
71	ERR	ERR Read Upd.cnf (success) =>	IDLE
72	ERR	Timeout => RESET MAILBOX	OFF
73	WSEG	START MAILBOX =>	WSEG
74	WSEG	STOP MAILBOX =>	OFF
75	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.Service != 2 => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER Seg = 0 ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
76	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.Service == 2 && Service_Data.Command_Specifier != 0,3 => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER Seg = 0 ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
77	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length < 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 0 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE Seg = 0 ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
78	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length >= 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 0 && Seg < (Length - 3 - Service_Data.SegDataSize) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = ABT_SEQ COE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
79	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length >= 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 0 && Seg >= (Length - 3 - Service_Data.SegDataSize) && (Service_Data.More Follows != (0 < (Seg - (Length - 3 - Service_Data.SegDataSize)))) Toggle = Service_Data.Toggle) => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = ABT_SEQ COE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD

表 110 (续)

#	当前状态	事件 /条件 =>动作	下一个状态
80	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length >= 10 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 0 && Seg >= (Length - 3 - Service_Data.SegDataSize) && Service_Data.More Follows == (0 < (Seg -(Length - 3 - Service_Data.SegDataSize))) && !Toggle = Service_Data.Toggle => Seg = (Seg -(Length - 3 - Service_Data.SegDataSize)) Size = Length - 3 - Service_Data.SegDataSize Toggle = Service_Data.Toggle More Follows = Service_Data.More Follows Data = Service_Data.Data Download SDO Segment.ind (Address, Toggle, More Follows, Size, Data)	DWLS
81	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length != 6 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 3 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
82	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 6 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 3 && Toggle = Service_Data.Toggle => Length = 10 Service_Data.Service = 2 Service_Data.Command_Specifier = 4 Service_Data.Abort_Code = ABT_SEQ COE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
83	WSEG	CoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 6 && Service_Data.Service == 2 && Service_Data.Command_Specifier == 3 && !Toggle = Service_Data.Toggle => Toggle = Service_Data.Toggle Upload SDO Segment.ind (Address, Toggle)	UPLS
84	WSEG	Emergency Service.req (Address, Error Code, Error Register, Size, Data) => Length = 10 Service_Data.Service = 1 Service_Data.Error Code = Error Code Service_Data.Error Register = Error Register Service_Data.Data = Data COE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
85	WSEG	other application service primitives => ignore	IDLE

6.4.4 EoE 状态机

6.4.4.1 描述

EoE 状态机负责在 EtherCAT 上传送标准以太网帧。无论 EtherCAT PDU 是完整的以太网帧还是可合并成完整帧的以太网帧分段, 状态机都可接收和发送 EtherCAT PDU。重组后 (不在本状态机的范围内) 的以太网帧可视为等同于未经 EtherCAT 服务传输的以太网帧。

在 IEEE 802.1D 中定义了与 EoE 服务相关的以太网端口的进出规则。

一般时间戳的定义:

- 32 位, 1ns 分辨率
- DC 系统时间可以被使用
- 时间戳的触发条件是目的地址(DA)的开始

附加时间戳 (TA=1):

- 从站->主站: 时间戳包含精确的接收时间
- 主站->从站: 时间戳包含期望发送时间
- 如果从站有这样的特性, 从站通常需要附加时间戳
- 时间戳将帧数据扩展 32 位
- TA bit 只允许在最后一个段 (LF=1)
- 如果时间戳不能填入最后的段->增加一个段
- 在原最后的段中填入部分时间戳(LF=0,TA=0), 并在追加的段中发送剩余部分时间(LF=1,TA=1)

时间戳请求(TR = 1):

- 以精确的发送时间作出响应, FrameNo 与请求中的相同
- 响应发送的越快越好

6.4.4.2 原语定义

6.4.4.2.1 邮箱处理程序和 EoESM 之间交换的原语

邮箱处理程序和 EoESM 之间交换的原语在 6.4.2.2 中描述。

6.4.4.2.2 应用和 EoESM 之间交换的原语

表 111 给出了由 AL 发出并由 EoESM 接收的服务原语及其相关参数。

表 111 – 应用向 EoESM 发出的原语

原语名称	相关参数	功能
Initiate_EoE.req	Address, Port Time Appended Time Requested Frame Number Complete Size Last Fragment Size Data Time Stamp	见 ETG.1000.5 的 CoE 服务定义
Initiate_EoE.rsp	Address, Frame Number Time Stamp	见 ETG.1000.5 的 CoE 服务定义
EoE Fragment.req	Address, Port Time Appended Frame Number Offset Last Fragment Size Data Time Stamp	见 ETG.1000.5 的 CoE 服务定义
Set IP Parameter.rsp	Address, Reason	见 ETG.1000.5 的 CoE 服务定义
Set Address Filter.rsp	Address, Reason	见 ETG.1000.5 的 CoE 服务定义

表 112 给出了由 EoESM 发出并由 AL 接收的服务原语及其相关参数。

表 112 – EoESM 向应用发出的原语

原语名称	相关参数	功能
Initiate_EoE.ind	Address, Port Time Appended Time Requested Frame Number Complete Size Last Fragment Size Data Time Stamp	见 ETG.1000.5 的 CoE 服务定义
EoE Fragment.ind	Address, Port Time Appended Frame Number Offset Last Fragment Size Data Time Stamp	见 ETG.1000.5 的 CoE 服务定义
Set IP Parameter.ind	Address, MAC Address IP Address Subnet Mask Default Gateway DNS Server DNS Name	见 ETG.1000.5 的 CoE 服务定义
Set Address Filter.ind	Address, Broadcast Forwarding MAC Address Filters MAC Filter Masks	见 ETG.1000.5 的 CoE 服务定义

6.4.4.2.3 原语参数

EoESM 和应用之间交换的原语所用到的参数在 ETG.1000.5 中描述。

6.4.4.3 EoESM 状态表

表 113 包含 EoESM 状态机的完整描述。在任何状态下 Set IP Parameter 和 Set Address Filter 都以如下方式处理: 如果可能, 请求的参数将被改变。

表 113 – EoESM 状态表

#	当前状态	事件 /条件 =>行动	下一状态
1	OFF	START MAILBOX => SSeg, RSeg = 0	IDLE
2	OFF	STOP MAILBOX =>	OFF
3	IDLE	START MAILBOX =>	IDLE
4	IDLE	STOP MAILBOX => Terminate Segmented Services	OFF
5	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg == 0 && Length < 36 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
6	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg == 0 && Length >= 36 && (Service_Data.Fragment > 0 Service_Data.CompleteSize < 2) => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
7	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg == 0 && Length >= 36 && Service_Data.Fragment == 0 && Service_Data.CompleteSize >= 2 && Service_Data.LastFragment == 1 => TimeAppended = Service_Data.TimeAppended TimeRequested= Service_Data.TimeRequested Size = Length-4-TimeAppended*4 if (TimeAppend) Timestamp = Service_Data[Length -4..Length-1] Frame Number = Service_Data.Frame Number Complete Size = Service_Data.CompleteSize Fragment Number = Service_Data.Fragment LastFragment = 1 Data = Service_Data.EoE Data Initiate_EoE.ind (Address, Port, Time Appended, Time Requested, Frame Number, Complete Size, Last Fragment, Size, Data, Timestamp)	IDLE
8	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg == 0 && Length >= 36 && Service_Data.Fragment == 0 && Service_Data.CompleteSize >= 2 && Service_Data.LastFragment == 0 && ((Length -4) mod 32) == 0 => TimeAppended = Service_Data.TimeAppended TimeRequested= Service_Data.TimeRequested Size, SSeg = Length-4 Frame Number = Service_Data.Frame Number Fragment Number = Service_Data.Fragment Complete Size = Service_Data.CompleteSize LastFragment = 0 Data = Service_Data.EoE Data Initiate_EoE.req (Address, Port, Time Appended, Time Requested, Frame Number, Complete Size, Last Fragment, Size, Data, Timestamp)	IDLE

表 113 (续)

#	当前状态	事件 /条件 =>行动	下一状态
9	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg == 0 && Length >= 36 && Service_Data.Fragment == 0 && Service_Data.CompleteSize >= 2 && Service_Data.LastFragment == 0 && ((Length - 4) mod 32) != 0 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
10	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg != 0 && (Service_Data.Fragment == 0 Service_Data.Offset*32 != SSeg) => SSeg = 0 Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
11	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg != 0 && Service_Data.Fragment != 0 && Service_Data.Offset*32 == SSeg && Service_Data.LastFragment == 1 => SSeg = 0 TimeAppended = Service_Data.TimeAppended Size = Length-4-TimeAppended*4 if (TimeAppend) Timestamp = Service_Data[Length - 4..Length-1] Frame Number = Service_Data.Frame Number Offset = Service_Data.Offset Fragment Number = Service_Data.Fragment LastFragment = 1 Data = Service_Data.EoE Data EoE_Fragment.ind (Address, Port, Time Appended, Frame Number, Offset, Last Fragment, Size, Data, Timestamp)	IDLE
12	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg != 0 && Service_Data.Fragment != 0 && Service_Data.Offset*32 == SSeg && Service_Data.LastFragment == 0 && ((Length - 4) mod 32) == 0 => SSeg = SSEG + Length-4 Frame Number = Service_Data.Frame Number Offset = Service_Data.Offset Fragment Number = Service_Data.Fragment LastFragment = 0 Data = Service_Data.EoE Data EoE_Fragment.ind (Address, Port, Time Appended, Frame Number, Offset, Last Fragment, Size, Data, Timestamp)	IDLE
13	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) / Service_Data.FrameType == 0 && SSeg != 0 && Service_Data.Fragment != 0 && Service_Data.Offset*32 == SSeg && Service_Data.LastFragment == 0 && ((Length - 2) mod 32) != 0 => SSeg = 0 Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR

表 113 (续)

#	当前状态	事件 /条件 =>行动	下一状态
14	IDLE	Initiate_EoE.req (Address, Port, Time Appended, Time Requested, Frame Number, Complete Size, Last Fragment, Size, Data, Timestamp) => Service_Data.TimeAppended = TimeAppended Service_Data.TimeRequested = TimeRequested Length = Size+4+TimeAppended*4 if (TimeAppend) Service_Data[Length -4..Length-1] = Timestamp Service_Data.Frame Number = Frame Number Service_Data.Fragment Number = Fragment Service_Data.Complete Size = CompleteSize Service_Data.LastFragment = LastFragment Service_Data.Port = Port Service_Data.EoE Data = Data EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
15	IDLE	EoE_Fragment.req(Address, Port, TimeAppended, Frame Number, Offset, Last Fragment, Size, Data, Timestamp) => Service_Data.TimeAppended = TimeAppended Length = Size+4+TimeAppended*4 if (TimeAppend) Service_Data[Length -4..Length-1] = Timestamp Service_Data.Frame Number = Frame Number Service_Data.Fragment Number = Fragment Service_Data.Complete Size = CompleteSize Service_Data.LastFragment = LastFragment Service_Data.Port = Port Service_Data.EoE Data = Data EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
16	IDLE	EOE Read Upd.cnf (success) => ignore	IDLE
17	IDLE	other application service primitives => ignore	IDLE
18	IDLE	Initiate_EoE.rsp(Address, Frame Number, Timestamp) => Service_Data.FrameType = 1 Service_Data.TimeAppended = 1 Service_Data.TimeStamp = Timestamp EOE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
19	WUPD	START MAILBOX =>	WUPD
20	WUPD	STOP MAILBOX => RESET MAILBOX	OFF
21	WUPD	EOE Read Upd.cnf (success) =>	WUPD
22	WUPD	EOE Read.ind =>	IDLE
23	WUPD	Timeout => RESET MAILBOX	OFF
24	ERR	START MAILBOX=>	ERR
25	ERR	STOP MAILBOX => RESET MAILBOX	OFF
26	ERR	ERR Read Upd.cnf (success) =>	ERR

表 113 (续)

#	当前状态	事件 /条件 =>行动	下一状态
27	ERR	ERR Read.ind =>	IDLE
28	ERR	Timeout => RESET MAILBOX	OFF
29	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length > 8 (fits request) && Service_Data.FrameType == 2 => Extract IP Parameter from Service Data. IP Parameter Set IP Parameter.ind(Address, Mac Address, IP Address, Subnet Mask, Default Gateway, DNS Server, DNS Name)	IPPAR
30	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length not (fits request) && Service_Data.FrameType == 2 => Length = 4 Service_Data.FrameType = 3 Service_Data.Result = 2 Service_Data.Port = 0 EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
31	IPPAR	START MAILBOX => ignore	IPPAR
32	IPPAR	STOP MAILBOX => CANCEL SERVICE	OFF
33	IPPAR	Set IP Parameter.rsp(+) (Adress) => Length = 4 Service_Data.FrameType = 3 Service_Data.Result = 0 Service_Data.Port = 0 EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
34	IPPAR	Set IP Parameter.rsp(-) (Adress, Reason) => Length = 4 Service_Data.FrameType = 3 Service_Data.Result = Reason Service_Data.Port = 0 EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
35	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length > 6 (fits request) && Service_Data.FrameType == 4 => Extract Address Filter from Service Data SetAddress Filter.ind(Address, Broadcast forwarding, MAC Address Filter 1..16, MAC Filter Mask 1..4)	MFLT
36	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length not (fits request) && Service_Data.FrameType == 4 => Length = 4 Service_Data.FrameType = 5 Service_Data.Result = 2 Service_Data.Port = 0 EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
37	MFLT	START MAILBOX => ignore	MFLT

表 113 (续)

#	当前状态	事件 /条件 =>行动	下一状态
38	MFLT	STOP MAILBOX => CANCEL SERVICE	OFF
39	MFLT	SetAddress Filter.rsp(+) (Adress) => Length = 4 Service_Data.FrameType = 5 Service_Data.Result = 0 Service_Data.Port = 0 EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
40	MFLT	SetAddress Filter.rsp(-) (Adress, Reason) => Length = 4 Service_Data.FrameType = 5 Service_Data.Result = Reason Service_Data.Port = 0 EoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
41	IDLE	EoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.FrameType != 0,2,4 => ignore	IDLE

6.4.5 FoE 状态机

6.4.5.1 描述

FoE 状态机负责在 EtherCAT 上传输文件。该状态机也可通过传送命令接收和发送 EtherCAT PDU（写表示加载文件到从站，读表示加载文件到主站）。

6.4.5.2 原语定义

6.4.5.2.1 邮箱处理程序和 FoESM 之间交换的原语

FoESM 与邮箱处理器程序的原语见 6.4.2.2。

6.4.5.2.2 应用和 FoESM 之间交换的原语

表 114 给出由 AL 发出并由 FoESM 接收的服务原语及其相关参数。

表 114 – 应用向 FoESM 发出的原语

原语名称	相关参数	功能
FoE Data.req	Address, Packet Number Size Data	见 ETG.1000.5 的 CoE 服务定义
FoE Ack.req	Address, Packet Number	见 ETG.1000.5 的 CoE 服务定义
FoE Error.req	Address, Error Code, Error Text	见 ETG.1000.5 的 CoE 服务定义

原语名称	相关参数	功能
FoE Data.req	Address, Packet Number Size Data	见 ETG.1000.5 的 CoE 服务定义
FoE Ack.req	Address, Packet Number	见 ETG.1000.5 的 CoE 服务定义
FoE Error.req	Address, Error Code, Error Text	见 ETG.1000.5 的 CoE 服务定义

表 115 给出由 FoESM 发出并由 AL 接收的服务原语及其相关参数。

表 115 – FoESM 向应用发出的原语

原语名称	相关参数	功能
FoE Write.ind	Address, Password File Name	见 ETG.1000.5 的 CoE 服务定义
FoE Read.ind	Address, Password File Name	见 ETG.1000.5 的 CoE 服务定义
FoE Data.ind	Address, Packet Number Size Data	见 ETG.1000.5 的 CoE 服务定义
FoE Ack.ind	Address, Packet Number	见 ETG.1000.5 的 CoE 服务定义
FoE Error.ind	Address, Error Code, Error Text	见 ETG.1000.5 的 CoE 服务定义

6.4.5.2.3 原语参数

FoESM 与应用之间交换的原语所使用的参数在 ETG.1000.5 中描述。

6.4.5.3 FoESM 状态表

FoESM 状态机的完整描述见表 116。

表 116 – FoESM 状态表

#	当前状态	事件 /条件 =>行动	下一状态
1	OFF	START MAILBOX => Seg = 0	IDLE
2	OFF	STOP MAILBOX =>	OFF
3	IDLE	START MAILBOX =>	IDLE
4	IDLE	STOP MAILBOX =>	OFF
5	IDLE	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode == 0 Service_Data.OpCode > 4 => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
6	IDLE	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length <= 6 && Service_Data.OpCode == 2 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
7	IDLE	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length > 6 && Service_Data.OpCode == 2 => Password = Service_Data.Password Filename = Service_Data.FileName FoE Write.ind (Address, Password, Filename)	DWLS
8	IDLE	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length <= 6 && Service_Data.OpCode == 1 => Length = 4 Service_Data == 1, MBXERR_INVALIDSIZE ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
9	IDLE	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length > 6 && Service_Data.OpCode == 1 => Password = Service_Data.Password Filename = Service_Data.FileName FoE Read.ind (Address, Password, Filename)	UPLS
10	IDLE	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Length == 3 && (Service_Data.OpCode == 3 Service_Data.OpCode == 4) => Length = 6 Service_Data.OpCode = 5 Service_Data.Error_Code = ABT_SEQ FoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
11	IDLE	FoE Read Upd.cnf (success) => ignore	IDLE
12	IDLE	other application service primitives => ignore	IDLE
13	DWLS	START MAILBOX => ignore	DWLS
14	DWLS	STOP MAILBOX => CANCEL SERVICE	OFF

表 116 (续)

#	当前状态	事件 /条件 =>行动	下一状态
15	DWLS	FoE Error.req (Address, ErrorCode, ErrorText) => Seg = 0 Length = 6 + size(Error_Text) Service_Data.OpCode = 5 Service_Data.Error_Code = Error_Code Service_Data.Error_Text= Error_Text FoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
16	DWLS	FoE Ack.req (Address, Packet Number) => Length = 6 Service_Data.OpCode = 4 Service_Data.PacketNumber = Packet Number Seg = Packet Number + 1 FoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	DWUPD
17	DWLS	other application service primitives => ignore	DWLS
18	UPLS	START MAILBOX => ignore	UPLS
19	UPLS	STOP MAILBOX => CANCEL SERVICE	OFF
20	UPLS	FoE Error.req (Address, ErrorCode, ErrorText) => Seg = 0 Length = 6 + size(Error_Text) Service_Data.OpCode = 5 Service_Data.Error_Code = Error_Code Service_Data.Error_Text= Error_Text FoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
11	IDLE	FoE Read Upd.cnf (success) => ignore	IDLE
21	UPLS	FoE Data.req (Address, Packet Number, Size, Data) /Size = FullSize => Length = 6 + Size Service_Data.OpCode = 3 Service_Data.PacketNumber = Packet Number Service_Data.Data = Data Seg = Packet Number + 1 FoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	UWUPD
22	UPLS	FoE Data.req (Address, Packet Number, Size, Data) /Size< FullSize => Length = 6 + Size Service_Data.OpCode = 3 Service_Data.PacketNumber = Packet Number Service_Data.Data = Data Seg = Packet Number + 1 FoE Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	WUPD
23	UPLS	other application service primitives => ignore	UPLS
24	UWUPD	START MAILBOX =>	UWUPD

表 116 (续)

#	当前状态	事件 /条件 =>行动	下一状态
25	UWUPD	STOP MAILBOX => RESET MAILBOX	OFF
26	UWUPD	FoE Read Upd.cnf (success) =>	UWSEG
27	UWUPD	Timeout => RESET MAILBOX	OFF
28	WUPD	START MAILBOX =>	WUPD
29	WUPD	STOP MAILBOX => RESET MAILBOX	OFF
30	WUPD	FoE Read Upd.cnf (success) =>	IDLE
31	WUPD	Timeout => RESET MAILBOX	OFF
32	DWUPD	START MAILBOX =>	DWUPD
33	DWUPD	STOP MAILBOX => RESET MAILBOX	OFF
34	DWUPD	FoE Read Upd.cnf (success) =>	DWSEG
35	DWUPD	Timeout => RESET MAILBOX	OFF
36	ERR	START MAILBOX =>	ERR
37	ERR	STOP MAILBOX => RESET MAILBOX	OFF
38	ERR	ERR Read Upd.cnf (success) =>	IDLE
39	ERR	Timeout => RESET MAILBOX	OFF
40	DWSEG	START MAILBOX =>	DWSEG
41	DWSEG	STOP MAILBOX =>	OFF
42	DWSEG	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode != 3 => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER Seg = 0 FoE Error.ind (Address, ErrorCode = ABT_SEQ, ErrorText) ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR

表 116 (续)

#	当前状态	事件 /条件 =>行动	下一状态
43	DWSEG	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode == 3 && Service_Data.PackeNumber != Seq => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER Seg = 0 FoE Error.ind (Address, ErrorCode = ABT_SEQ, ErrorText) ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
44	DWSEG	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode == 3 && Service_Data.PackeNumber != Seq && Length = FullSize + 6 => Size = Length - 6 Packet Number = Service_Data.Packet Number Data = Service_Data.Data FoE Data.ind (Address, Packet Number, Size, Data)	DWLS
45	DWSEG	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode == 3 && Service_Data.PackeNumber != Seq && Length < FullSize + 6 => Size = Length - 6 Packet Number = Service_Data.Packet Number Data = Service_Data.Data FoE Data.ind (Address, Packet Number, Size, Data)	IDLE
46	DWSEG	other application service primitives => ignore	DWSEG
47	UWSEG	START MAILBOX =>	UWSEG
48	UWSEG	STOP MAILBOX =>	OFF
49	UWSEG	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode != 4 => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER Seg = 0 FoE Error.ind (Address, ErrorCode = ABT_SEQ, ErrorText) ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
50	UWSEG	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode == 4 && Service_Data.PackeNumber != Seq => Length = 4 Service_Data == 1, MBXERR_INVALIDHEADER Seg = 0 FoE Error.ind (Address, ErrorCode = ABT_SEQ, ErrorText) ERR Mailbox Read Upd.req (Length, Address, Channel, Priority, Service_Data)	ERR
51	UWSEG	FoE Mailbox Write.ind (Length, Address, Channel, Priority, Service Data) /Service_Data.OpCode == 4 && Service_Data.PackeNumber == Seq => Packet Number = Service_Data.Packet Number FoE Ack.req (Address, Packet Number)	UPLS
52	UWSEG	other application service primitives => ignore	UWSEG

6.5 DLL 映射协议机(DLL mapping protocol machine, DMPM)

ETG.1000.3 中定义的服务被直接应用在 ARPMs。

参考文献

IEC 61131-3, *Programmable controllers – Programming languages*

IEC/TR 61158-1 (Ed.2.0), *Industrial communication networks – Fieldbus specifications – Part 1: Overview and guidance for the IEC 61158 and IEC 61784 series*

IEC 61158-3-12, *Industrial communication networks – Fieldbus specifications - Part 3-12: data-link layer service definition – Type 12 elements*

IEC 61158-4-12, *Industrial communication networks – Fieldbus specifications - Part 4-12: data-link layer protocol specification – Type 12 elements*

IEC 61158-5-12, *Industrial communication networks – Fieldbus specifications - Part 5-12: Application layer service definition – Type 12 elements*

IEC 61784-2, *Industrial communication networks – Profiles – Part 2: Additional fieldbus profiles for real-time networks based on ISO/IEC 8802-3*

ISO/IEC 646, *Information technology – ISO 7-bit coded character set for information interchange*

ISO/IEC 8822, *Information technology – Open Systems Interconnection – Presentation service definition*

ISO/IEC 8859-1, *Information technology – 8-bit single-byte coded graphic character sets – Part 1: Latin alphabet No. 1*

ISO/IEC 10646, *Information technology – Universal Multiple-Octet Coded Character Set (UCS)*

ISO 8601, *Data elements and interchange formats – Information interchange – Representation of dates and times*

ISO 15745-1, *Industrial automation systems and integration – Open systems application integration framework – Part 1: Generic reference description*

IETF RFC 792, *Internet Control Message Protocol*; available at <<http://www.ietf.org>>

ETG.1000.2 *EtherCAT Specification Part 2: Physical layer specification and service definition*

ETG.1000.3 *EtherCAT Specification Part 3: Data Link Layer service definition*

ETG.1000.4 *EtherCAT Specification Part 4: Data-link layer protocol specification*

ETG.1000.5 *EtherCAT Specification Part 5: Application layer service definition*

ETG.1000.6 *EtherCAT Specification Part 6: Application layer protocol specification*

ETG.1100 *EtherCAT Specification Communication profiles*

附录

EtherCAT Technology Group (ETG)

EtherCAT Technology Group

Ostendstrasse 196
90482 Nuremberg, Germany

phone: +49 (9 11) 5 40 56 20

fax: +49 (9 11) 5 40 56 29

e-mail: info@ethercat.org

web: www.ethercat.org